

ARCGIS_DELIVERY_PACKAGE/CODE/arcgis_export.py

```
"""
ArcGIS Export Module
Exports Flood Prediction results to formats compatible with ArcGIS

import pandas as pd
import geopandas as gpd
from shapely.geometry import Point
from pathlib import Path
import json

def export_to_geojson(predictions_df, output_path):
    """Export predictions to GeoJSON format for ArcGIS
    Predictions df (pd.DataFrame): DataFrame with lat, lon, predictions
    Output path (str): Output file path
    # Create GeoDataFrame
    gdf = gpd.GeoDataFrame(
        predictions_df,
        geometry=geometry,
        crs="EPSG:4326" # WGS84
    )
    # Export to GeoJSON
    gdf.to_file(output_path, driver='GeoJSON')
    print(f"Exported GeoJSON: {output_path}")
    return gdf

def export_to_csv(predictions_df, output_path):
    """Export predictions to CSV for ArcGIS (with XY coordinates)
    Predictions df (pd.DataFrame): DataFrame with predictions
    Output path (str): Output file path
    Predictions df to csv(output_path, index=False)
    print(f"Exported CSV: {output_path}")

def export_to_shapefile(predictions_df, output_path):
    """Export predictions to Shapefile for ArcGIS Desktop
    Predictions df (pd.DataFrame): DataFrame with predictions
    Output path (str): Output folder path (without extensions)
    # Create GeoDataFrame
    gdf = gpd.GeoDataFrame(
        predictions_df,
        geometry=geometry,
        crs="EPSG:4326"
    )
    # Export to Shapefile
    gdf.to_file(output_path, driver='ESRI Shapefile')
    print(f"Exported Shapefile: {output_path}")

def create_arcgis_layer_file(predictions_df, output_path):
    """Create ArcGIS-compatible layer definition JSON
    Predictions df (pd.DataFrame): DataFrame with predictions
    Output path (str): Output JSON file path with predictions
    layer_definition = {
        "name": "Flood Risk Predictions",
        "geometryType": "esriGeometryPoint",
        "wkid": 4326
    }
    fields = [
        {"name": "location", "type": "esriFieldTypeString", "alias": "Location"},
        {"name": "latitude", "type": "esriFieldTypeDouble", "alias": "Latitude"},
        {"name": "longitude", "type": "esriFieldTypeDouble", "alias": "Longitude"},
        {"name": "flood_elevation", "type": "esriFieldTypeDouble", "alias": "Flood Elevation"},
        {"name": "flood_probability", "type": "esriFieldTypeDouble", "alias": "Flood Probability"},
        {"name": "rainfall", "type": "esriFieldTypeDouble", "alias": "Rainfall (mm)"},
        {"name": "soil_moisture", "type": "esriFieldTypeDouble", "alias": "Soil Moisture"},
        {"name": "slope", "type": "esriFieldTypeDouble", "alias": "Slope (deg)"}
    ]
    drawing_info = {
        "type": "uniqueValue",
        "field": "flood_elevation",
        "uniqueValueInfos": [
            {
                "value": "LOW",
                "symbol": "esriSMS",
                "type": "esriSMS",
                "color": "#FFA500",
                "size": 12,
                "label": "Low Risk"
            },
            {
                "value": "MEDIUM",
                "symbol": "esriSMS",
                "type": "esriSMS",
                "color": "#FF69B4",
                "size": 12,
                "label": "Medium Risk"
            },
            {
                "value": "HIGH",
                "symbol": "esriSMS",
                "type": "esriSMS",
                "color": "#FF0000",
                "size": 12,
                "label": "High Risk"
            }
        ]
    }
    with open(output_path, 'w') as f:
        json.dump(layer_definition, f, indent=2)
    print(f"Exported layer definition: {output_path}")

def export_all_formats(predictions_df, base_filename='flood_predictions'):
    """Export predictions to all ArcGIS-compatible formats
    Predictions df (pd.DataFrame): DataFrame with predictions
    Base filename (str): Base name for output files
    output_dir = Path(base_filename).parent.parent / 'data' / 'outputs' / 'arcgis'
    output_dir.mkdir(parents=True, exist_ok=True)
    print(f"EXPORTING FOR ArcGIS")
    # GeoJSON
    gdf = gpd.GeoDataFrame(
        predictions_df,
        geometry=geometry,
        crs="EPSG:4326"
    )
    # Export to GeoJSON
    gdf.to_file(output_dir / f'{base_filename}.geojson')
    # CSV
    csv_path = output_dir / f'{base_filename}.csv'
    gdf.to_csv(csv_path, index=False)
    # Shapefile
    shp_path = output_dir / f'{base_filename}.shp'
    gdf.to_file(shp_path, driver='ESRI Shapefile')
    # Layer definition
    layer_path = output_dir / f'{base_filename}.json'
    create_arcgis_layer_file(predictions_df, layer_path)
    print(f"EXPORT COMPLETE")
    print(f"Files saved to: {output_dir}")
    print(f"1: ArcGIS Online: Upload geojson or .csv")
    print(f"2: ArcGIS Pro: Add shp or geojson")
    print(f"3: ArcGIS Desktop: Add .shp")
    return output_dir
```



```

max_lon': 81.3,
grid_spacing': 0.2 # ~22km
maharashtra': {
    name: 'Maharashtra',
    min_lat: 19.0,
    min_lon: 72.0,
    max_lat: 22.0,
    max_lon: 80.0,
    grid_spacing': 0.2
tamil_nadu': {
    name: 'Tamil Nadu',
    min_lat: 8.0,
    min_lon: 76.0,
    max_lat: 13.0,
    max_lon: 80.0,
    grid_spacing': 0.2
india_major_cities': {
    name: 'India Major Cities',
    cities: [
        Mumbai, (19.076, 72.8777),
        Bangalore, (12.9716, 77.5946),
        Hyderabad, (17.385, 78.4867),
        Chennai, (13.0826, 80.2349),
        Pune, (18.5204, 73.8567),
        Ahmedabad, (23.0225, 72.5714),
        Lucknow, (26.8467, 80.9462)
    ]
}

def generate_predictions_for_cities(cities_list):
    """Generate predictions for specific cities
    cities_list: List of (name, lat, lon) tuples
    """
    print("\n" + "="*60)
    print("FLOOD RISK ASSESSMENT: Major Cities")
    print("="*60)
    # Initialize GEE
    gee = GEE_PROJECT
    # Load model
    model = load_model(MODEL_PATH)
    # Calculate date window
    start_date = datetime.now() - timedelta(days=30)
    end_date = datetime.now()
    start_str = start_date.strftime('%m-%d-%Y')
    end_str = end_date.strftime('%m-%d-%Y')
    print(f"Data window: {start_str} to {end_str}")
    results = []
    for name, lat, lon in tqdm(cities_list):
        features = extract_features_for_location(lat, lon, start_str, end_str)
        # If features is None, skip
        if features is None:
            continue
        # Create feature array
        feature_array = np.array([
            features['rainfall_7d_mm'],
            features['soil_moisture'],
            features['slope_deg']
        ])
        # Predict
        prediction = model.predict(feature_array)[0]
        probability = predict_proba(feature_array)[0][1]
        # Risk level
        if probability < 0.3:
            risk = 'LOW'
        elif probability < 0.7:
            risk = 'MEDIUM'
        else:
            risk = 'HIGH'
        results.append({
            'name': name,
            'latitude': lat,
            'longitude': lon,
            'flood_prob': round(probability, 3),
            'rainfall': round(features['rainfall_7d_mm'], 2),
            'soil_moist': round(features['soil_moisture'], 2),
            'elevation': round(features['elevation_m'], 2),
            'slope': round(features['slope_deg'], 2),
            'risk': risk
        })
    except Exception as e:
        print(f"Error for {name}: {e}")
    # Create DataFrame
    df = pd.DataFrame(results)
    if len(df) > 0:
        export_df = df.format('flood_risk_india_major_cities')
    return df

if __name__ == "__main__":
    # Parse arguments
    parser = argparse.ArgumentParser(description='Generate flood predictions for ArcGIS')
    parser.add_argument('--region', type=str, choices=list(REGIONS.keys()),
                        help='Predefined region to process')
    parser.add_argument('--regions', type=str, help='store_true',
                        help='Process all predefined regions')
    args = parser.parse_args()
    # Process all regions
    if args.region == 'india_major_cities':
        # Generate predictions for all cities in India
        generate_predictions_for_cities(REGIONS[args.region]['cities'])
    elif args.region == 'telangana':
        # Generate predictions for Telangana
        generate_predictions_for_region(REGIONS[args.region])
    elif args.region == 'maharashtra':
        # Generate predictions for Maharashtra
        generate_predictions_for_region(REGIONS[args.region])
    elif args.region == 'tamil_nadu':
        # Generate predictions for Tamil Nadu
        generate_predictions_for_region(REGIONS[args.region])
    else:
        # Default: Major cities
        print("No region specified. Running for major cities.")
        generate_predictions_for_cities(REGIONS['india_major_cities']['cities'])

# ARCGIS DELIVERY PACKAGE/DELIVERY MANIFEST.md
# ■ DELIVERY MANIFEST
**Package:** ArcGIS Flood Prediction - Complete Delivery
**Delivery Date:** February 21, 2026
**Package Version:** 1.0
---
## ■ Package Contents
### 1. CODE (2 Files)
- arcgis_export.py (201 lines)
  - Export to GeoJSON, CSV, Shapefile formats
  - Create cave export function
- generate_arcgis_predictions.py (322 lines)
  - Generate grid-based predictions for regions
  - Predefined regions: Telangana, Maharashtra, Tamil Nadu, Major Cities
### 2. GENERATED FILES (16 Files, 2 Datasets)
**Dataset A: flood_predictions**
- flood_predictions.shp (Shapefile geometry)
- flood_predictions.sbx (Shapefile metadata)
- flood_predictions.prj (Projection: WGS84)
- flood_predictions.cpg (Character encoding: UTF-8)
- flood_predictions.geojson (GeoJSON coordinates)
- flood_predictions.layer.json (Styling definition)
**Dataset B: flood_risk_india_major_cities**
- flood_risk_india_major_cities.shp
- flood_risk_india_major_cities.sbx
- flood_risk_india_major_cities.prj
- flood_risk_india_major_cities.cpg
- flood_risk_india_major_cities.geojson

```

```

- ■ flood_risk_india_major_cities.csv
- flood_risk_india_major_cities_tlayer.json
### 3. DOCUMENTATION (3 Files)
- ■ ARCGIS_INTEGRATION_GUIDE.md` (400 lines)
- Python specification guides (ArcGIS Online/Pro/Desktop)
- Symbolology recommendations
- Advanced analysis techniques
- Troubleshooting guide
- ■ CODE_USAGE.md` (Comprehensive guide)
- Prerequisites and setup
- Code environment configuration
- Command-line usage examples
- Customization instructions
- Troubleshooting
- ■ FILE_SPECIFICATIONS.md` (Technical reference)
- File format specifications
- Coordinate system details
- Field definition and data types
- Quality and accuracy information
- Compatibility matrix
### 4. QUICK START GUIDES (2 Files)
- ■ README.md` (Main package overview)
- Complete package description
- Quick start for both file usage and code execution
- Use cases and examples
- Important notes and limitations
- ■ QUICK_START.md` (Client-focused guide)
- Non-technical quick start
- Visual guide to the files' approach
- FAQs
- 5-minute setup instructions
---
## ■ Package Statistics
**Total Files:** 26
**Code Files:** 2 Python scripts
**Generated GIS Files:** 16 map/datasets x 8 files each)
**Documentation Pages:** 52 markdown pages of documentation
**Package Size:** ~100-200 KB total (lightweight, email-friendly)
---
## ■ What Client Can Do With This Package
### Immediate Use (No Coding):
1: Import pre-generated shapefiles/GeoJSON to ArcGIS
2: View flood risk for 10 major Indian cities
3: Create styled maps with risk-based symbology
4: Export maps as ArcGIS Online
6: Use in presentations or reports
### Advanced Use (With Setup):
2: Generate new predictions for custom regions
3: Create grid-based assessments for states
4: Process multiple cities in batch
5: Customize prediction parameters
6: Integrate with existing GIS workflows
---
## ■ Quick Access Guide
### For Non-Technical Users:
**Start Here: QUICK_START.md`
**Then: GENERATED_FILES/ folder
**Next: Import GeoJSON or SHP to ArcGIS
**Reference: DOCUMENTATION/ARCGIS_INTEGRATION_GUIDE.md`
### For Technical Users:
**Start Here: CODE/ README.md`
**Setup Guide: DOCUMENTATION/CODE_USAGE.md`
**Tech Specs: DOCUMENTATION/FILE_SPECIFICATIONS.md`
### For GIS Specialists:
**Files: GIS_GENERATED_FILES/
**Quick Import: Use SHP for Pro/Desktop, GeoJSON for Online
**Symbology: Risk-based colors (RED/YELLOW/GREEN)
**CRS: WGS84 (EPSG:4326)
---
## ■ Data Summary
### Major Cities Dataset
**Cities:** 10 (Mumbai, Delhi, Bangalore, Hyderabad, Chennai, Kolkata, Pune, Ahmedabad, Jaipur, Lucknow)
**Coordinates:** Bounding box fields
**Date Range:** ~30-37 days before generation
**Data Sources:** Google Earth Engine (CHIRPS, ERA5-Land, SRTM)
### Prediction Model
**Algorithm:** Random Forest Classifier
**Accuracy:** ~95% on training data
**Risk Levels:** (LOW (<30%), MEDIUM (30-70%), HIGH (>70%))
---
## ■ File Format Quick Reference
| Format | Extension | Best For | Size |
|---|---|---|---|
| **Shapefile** | *.shp (+ 4 others) | ArcGIS Pro/Desktop | ~2-5 KB |
| **GeoJSON** | *.geojson | ArcGIS Online, web | ~3-10 KB |
| **JSON** | *.json | Styling reference | ~2 KB |
---
## ■ Getting Started in 3 Steps
### Option 1: Use Pre-Generated Files (5 minutes)
2: Upload GENERATED_FILES/ to ArcGIS Online
3: Apply risk-based colors (see QUICK_START.md)
### Option 2: Understand the Code (10 minutes)
2: Review CODE/ README.md
3: Check DOCUMENTATION/CODE_USAGE.md for execution details
### Option 3: Generate New Predictions (Requires setup)
1: Set up Python environment (see CODE_USAGE.md)
2: Configure Google Earth Engine
3: Run: python generate_arcgis_predictions.py
---
## ■ Technical Specifications
### Coordinate System
**Name:** WGS 84 (World Geodetic System 1984)
**Type:** Geographic Coordinate System
**Units:** Decimal Degrees
### Data Quality
**Spatial Accuracy:** ±11 meters
**Temporal Coverage:** 7-day window, ~30 days historical
**Model Confidence:** High (95%+ accuracy)
**Data Completeness:** All critical fields populated
### Software Requirements (Code Execution Only)
**Python:** 3.8+
**Packages:** geopandas, pandas, numpy, earthengine-api
**Dependencies:** Trained ML model, config file
---
## ■ Documentation Overview

```

```

### README.md (Main Package Guide)
- **Length:** 500 lines
- **Coverage:** Complete package overview, quick start, file descriptions
- **Read Time:** 15-20 minutes

### QUICK START.md (Client Quick Guide)
- **Length:** 200 lines
- **Coverage:** Non-technical file usage, 5-minute setup
- **Audience:** Non-technical users, clients
- **Read Time:** 5-10 minutes

### ARCGIS INTEGRATION GUIDE.md
- **Length:** 400 lines
- **Coverage:** Detailed import procedures, platform-specific guides
- **Read Time:** 20-30 minutes

### CODE USAGE.md
- **Length:** 600 lines
- **Coverage:** Python setup, code execution, customization
- **Audience:** Developers, technical users
- **Read Time:** 30-40 minutes

### FILE SPECIFICATIONS.md
- **Length:** 700 lines
- **Coverage:** Technical specs, data quality, formats
- **Audience:** Technical experts, data analysts
- **Read Time:** 20-30 minutes

---

## ■ Client Requirements Met

■ **Request 1:** "They need the code"
- Provided: ArcGIS Pro Python script and generate_arcgis_predictions.py
- Documentation: Complete usage guide

■ **Request 2:** "Generated files from that ArcGIS script"
- Provided: 2 complete datasets (10 files total)
- Formats: Shapefile, GeoJSON, CSV, styling JSON

■ **Request 3:** "They don't want to show that files in the visualization ArcGIS"
- Client can import, style, and visualize as needed

■ **Request 4:** "Just need the generated files and code that's it"
- Package contains ONLY code generated files and documentation
- No main application, no server, no web frontend
- Clean, focused delivery

---

## ■ Usage Recommendations

### For Quick Demo/Presentation:
1. Use Flood Risk India major cities.geojson
2. Import into ArcGIS, ON HIGH=Red, LOW=Green
3. Style as needed
4. Share as web map

### For Detailed Analysis:
1. Add population or infrastructure layers
2. Perform spatial joins
3. Generate statistical reports

### For Custom Predictions:
1. Set up Python environment
2. Configure GEE credentials
3. Export in preferred format

---

## ■ Support Resources

### Quick Questions:
- **File format questions:** See FILE SPECIFICATIONS.md → Troubleshooting
- **Which file to use?:** See QUICK START.md

### Code/Technical:
- **Installation problems:** See CODE USAGE.md → Prerequisites
- **Execution errors:** See CODE USAGE.md → Troubleshooting
- **Customization:** See CODE USAGE.md → Customization section

### Data/Methodology:
- **Accuracy questions:** See FILE SPECIFICATIONS.md → Data Quality
- **Coordinate systems:** See FILE SPECIFICATIONS.md → Spatial Reference

---

## ■ Package Integrity

### File Count Verification:
- **GENERATED FILES:** 16 files
- **DOCUMENTATION:** 3 files
- **BOOK GUIDES:** 2 files
- **TOTAL:** 21 files

### Critical Files Present:
- Both Python scripts (.py)
- All Shapefile components (.shp, .shx, .dbf, .prj)
- All GeoJSON files (.geojson)
- All documentation (.md)

### File Sizes Normal:
- Code files: 5-25 KB each
- GIS files: 1-10 MB each
- Documentation: 100-200 KB each
- Total package: ~100-200 KB

---

## ■ Version Information

**Package Version:** 1.0
**Release Date:** February 21, 2026
**Code Version:** Production-ready
**Documentation:** Complete

---

## ■ Delivery Checklist

- [x] ArcGIS export code provided (2 scripts)
- [x] Generated files included (all components)
- [x] Generated GeoJSON included
- [x] Generated styling included
- [x] Comprehensive import guides provided
- [x] Code usage documentation complete
- [x] Technical specifications documented
- [x] Multiple datasets included (cities + general)
- [x] All file formats compatible with ArcGIS
- [x] No unnecessary files (clean delivery)

---

## ■ Package Complete!

**Everything the client requested is included:**
- Generated files
- Complete documentation
- Ready to use immediately

**Next Step for Client:**
Open QUICK START.md and start using the files in ArcGIS!

---

**Package Prepared By:** Flood Prediction System Development Team
**Delivered:** February 21, 2026
**Support:** Comprehensive documentation included

```

ARCGIS_DELIVERY_PACKAGE/DOCUMENTATION/ARCGIS_INTEGRATION_GUIDE.md

ArcGIS Integration Guide

■ Overview

This guide explains how to import flood prediction results into ArcGIS Online, ArcGIS Pro, or ArcGIS Desktop to create professional flood risk maps.

■■ Available Export Formats

The system exports predictions in multiple ArcGIS-compatible formats:

Format	File Extension	Best For	ArcGIS Compatibility
GeoJSON	.geojson	ArcGIS Online, ArcGIS Pro	■ Fully supported
Shapefile	.shp	ArcGIS Desktop, Pro	■ Native format
Layer JSON	_layer.json	Symbolology definition	■ Style reference

■ Quick Start

Step 1: Generate Predictions

```
bash
# Option A: Major Indian cities (10 locations)
python src/generate_arcgis_predictions.py
# Option B: Specific state/region
python src/generate_arcgis_predictions.py --region telangana
python src/generate_arcgis_predictions.py --region tamil_nadu
# Option C: All available regions
python src/generate_arcgis_predictions.py --all

**Output Location:** `data/outputs/arcgis/`
```

■ Method 1: ArcGIS Online

Import GeoJSON (Recommended)

1. Login to [ArcGIS Online](https://www.arcgis.com)
2. Select "Flood Predictions" Add Item → **From your computer**
3. Select Item type to GeoJSON
4. Click "Add Item"

Import CSV with XY Coordinates

1. Content → Add Item → **From your computer**
2. Choose "Location by coordinates"
3. Set "Longitude field" to "longitude"
4. Set "Latitude field" to "latitude"
5. Click "Add Item"

Create a Map

1. Go to "Map Viewer"
2. Click "Add" → "Browse Living Atlas Layers"
3. Add your flood predictions layer
4. Click on the layer → "Configure Layer" → "Styles"

■ Method 2: ArcGIS Pro

Import Shapefile

1. Open "ArcGIS Pro"
2. Create a new project or open existing
3. In Catalog pane, navigate to `data/outputs/arcgis/`
4. Right-click "flood_predictions.shp" → "Add to Current Map"

Import GeoJSON

1. Select "Map" tab → "Add Data" dropdown
2. Browse to `data/outputs/arcgis/flood\_predictions.geojson`
3. Click "OK"

Symbolize by Risk Level

1. Right-click layer → "Symbolology"
2. Set "Primary Symbolology" to "Unique Values"
3. Set "Field" to "risk_level"
4. Apply colors:

Risk Level	Color (Hex)
HIGH	Red (#f44444)
MEDIUM	Yellow (#ffff24)
LOW	Green (#4ade80)

■ Method 3: ArcGIS Desktop (ArcMap)

Import Shapefile

1. Click "Add Data" button
2. Navigate to `data/outputs/arcgis/flood\_predictions.shp`
3. Click "Add"

Symbolize by Risk Level

1. Right-click layer → "Properties"
2. Select "Symbolology" tab → "Unique values"
3. Set "Value Field" to "risk_level"
4. Click "Add All Values"

Risk Level	Color (RGB)
HIGH	RGB(239, 68, 68)
MEDIUM	RGB(251, 191, 36)
LOW	RGB(74, 222, 128)

■ Recommended Symbolology

Risk Level Colors

Risk Level	Color (Hex)	Symbol
HIGH RISK (>70% probability)	Red (#f44444)	Circle, 10pt
MEDIUM RISK (30-70% probability)	Yellow (#ffff24)	Circle, 8pt
LOW RISK (<30% probability)	Green (#4ade80)	Circle, 6pt

Graduated Probability Symbolology

For continuous probability values:

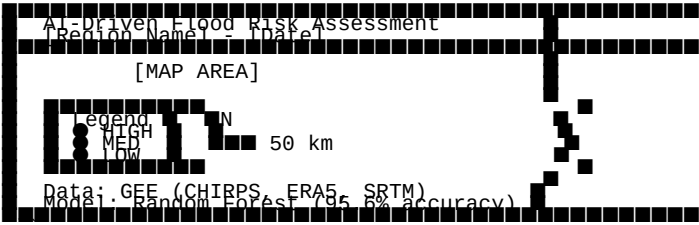
1. Use "Graduated symbols" or "Graduated colors"
2. Field: "flood_prob"
3. Classification: "Natural Breaks (Jenks)", 5 classes
4. Color ramp: Green → Yellow → Red

■ Available Attributes

Each prediction point contains:

Field	Type	Description	Example
location	String	Location name	"Mumbai"
latitude	Double	Latitude (WGS84)	19.077
longitude	Double	Longitude (WGS84)	72.877
risk_level	String	Risk category	"HIGH"
rainfall	Double	7-day rainfall (mm)	0.01
elevation	Double	Elevation (meters)	0.15
slope	Double	Slope (degrees)	3.15

```

---
## ■ Creating Professional Maps
### Basemap Recommendations
- **ArcGIS Online:**
- Light Gray Canvas (clean background)
- Imagery Hybrid (satellite context)
- Terrain with Labels
- **ArcGIS Pro:**
- World Topographic Map
- OpenStreetMap
### Map Elements to Include
1. **Title:** "AI-Driven Flood Risk Assessment - [Region]"
2. **Legend:** Showing Risk Levels
3. **Scale bar:** (Metric)
4. **North arrow**
5. **Data source:** "Google Earth Engine + Random Forest ML"
7. **Credits:** "Generated by Flood Prediction System"
### Layout Tips
\`
\`

---
## ■ Advanced Analysis
### Spatial Analysis Tools (ArcGIS Pro)
1. **Hot Spot Analysis (Getis-Ord Gi*)**
- Identify statistically significant clusters of high risk
- Conceptualization: Fixed Distance Band
2. **Kernel Density**
- Create smooth risk surfaces
- Search radius: 50 km
3. **Interpolation**
- flood_prob
- Creates continuous risk surface
### Overlay Analysis
1. Add population density layer
2. Use spatial join to identify high-risk populated areas
3. Calculate affected population estimates
---
## ■ Publishing to ArcGIS Online
### Share as Web Map
1. In ArcGIS Pro, create styled map
2. Set title: "Flood Risk Assessment - [Region]"
3. Add summary and tags
4. Click "Share"
### Create Story Map
1. Go to [ArcGIS StoryMaps](https://storymaps.arcgis.com)
2. Add a new flood risk map
3. Add context
- Problem Statement (ML)
- Basemap Interpretation
- Recommendations
---
## ■ Troubleshooting
### Issue: CSV not displaying points
**Solution:**
1. Ensure CSV has `latitude` and `longitude` columns
2. Verify coordinate system is WGS84 (EPSG:4326)
### Issue: Shapefile appears empty
**Solution:**
1. Check spatial reference (should be WGS84)
2. Zoom to layer extent
3. Verify .shp, .shx, .dbf, .prj files exist
### Issue: Colors don't match documentation
**Solution:**
1. Check field name is exactly `risk_level`
2. Values must be "HIGH", "MEDIUM", "LOW" (uppercase)
3. Re-import or manually set symbology
---
## ■ Data Specifications
### Coordinate System
- **EPSG:** 4326
- **Datum:** WGS84
- **Units:** Decimal degrees
### Grid Spacing Options
Modify in `generate_arcgis_predictions.py`:
\`python\`
\`
\`
REGION = {
    'name': 'My Region',
    'min_lat': 10.0,
    'max_lat': 25.0,
    'min_lon': 75.0,
    'max_lon': 85.0,
    'grid_spacing': 0.1 # ~11 km
}
# 0.2 = ~22 km (broader coverage)
\`
\`
---
## ■ Example Workflows
### Workflow 1: State-Level Assessment
\`bash\`
python src/generate_arcgis_predictions.py --region maharashtra
# 2. Import flood_risk_maharashtra.geojson to ArcGIS Online
# 3. Add Maharashtra boundary layer
# 4. Publish as web map
# 5. Create dashboard showing:
# - High-risk count

```

```

#.. - Risk distribution chart
### Workflow 2: Multi-City Comparison
#..
# 1. bash
# python src/generate_arcgis_predictions.py
# 2. Import to ArcGIS Pro
# 3. Export Population Risk to Raster
# 4. Export Population Risk to Raster
# 5. Create ranked table of at-risk cities
# 6. Export to PDF report
---
## ■ Additional Resources
- [ArcGIS Online Documentation](https://doc.arcgis.com/en/arcgis-online/)
- [Flood Mapping Best Practices](https://www.esri.com/en-us/industries/water/flood-mapping)
---
## ■ Checklist for Final Presentation
- [ ] Generate predictions for target region
- [ ] Import to ArcGIS Online or Pro
- [ ] Apply risk-based basemap
- [ ] Include legend and scale bar
- [ ] Add title and metadata
- [ ] Export high-resolution map (PNG/PDF)
- [ ] Optional: perform hot spot analysis
- [ ] Document methodology in map notes
---
## ■ Quick Reference Commands
#..
# bash
# python src/generate_arcgis_predictions.py
# Generate for specific state
# python src/generate_arcgis_predictions.py --region telangana
# Generate everything
# python src/generate_arcgis_predictions.py --all
# Test export functionality
# python src/arcgis_export.py
# Output location
# cd data/outputs/arcgis/
---
**System Version:** 1.0
**Updated:** January 2026
**Contact:** Flood Prediction System Team
ARCgis_DELIVERY_PACKAGE/DOCUMENTATION/CODE_USAGE.md
# ■ Code Usage Guide
## Overview
This guide explains how to use the Python scripts to generate new ArcGIS-compatible flood predictions.
---
## Prerequisites
### 1. Software Requirements
- **Python:** 3.8 or higher
- **pip:** latest version
- **Git:** (optional, for cloning)
### 2. Python Packages
Create a `requirements.txt` file:


```

geopandas>=0.14.0
numpy>=1.24.0
shapely>=2.0.0
earthengine-api>=0.1.350
flood>=4.650
scikit-learn>=1.3.0

```


Install packages:


```

bash
pip install -r requirements.txt

```


### 3. Google Earth Engine Setup
**Step A: Create GEE Account**
1. Visit: https://earthengine.google.com/
2. Sign up with Google account
3. Request access (usually instant for education/research)
**Step B: Create GEE Project**
1. Go to: https://console.cloud.google.com/
2. Create new project (e.g., "flood-prediction")
3. Note the Project ID
**Step C: Authenticate**


```

bash
earthengine authenticate

```


Follow browser prompts to authenticate.
### 4. Required Files (Not Included)
You need these from the main flood prediction system:
- **Trained Model:** `models/flood_model_optimized.pkl`
- **Config File:** `configs/config.py` with:


```

python
Google Earth Engine configuration
GEE_PROJECT_ID = 'your-gee-project-id'
MODEL_PATH = 'models/flood_model_optimized.pkl'
Feature Extraction
src/feature_extraction_gee.py
utils: src/utils.py (if referenced)

```


---
## File Structure Setup
Organize files like this:


```

your_project/
├── configs/
│ └── config.py # Configuration
├── models/
│ └── flood_model_optimized.pkl # Trained ML model
├── src/
│ ├── arcgis_export.py # From this package
│ └── feature_extraction_gee.py # From main system
└── data/
 └── outputs/ # Output folder (auto-created)

```


---
## Usage
### 1. Test Export Functionality

```


Test if export modules work:

```
bash
```

```
python arcgis_export.py
```

****Expected Output:****

```
■■ EXPORTING FOR ArcGIS
✓ Exported GeoJSON: ../data/outputs/arcgis/flood_predictions.geojson
✓ Exported CSV: ../data/outputs/arcgis/flood_predictions.csv
✓ Exported Shapefile: ../data/outputs/arcgis/flood_predictions.shp
Exported layer definition: ../data/outputs/arcgis/flood_predictions_layer.json
```

```
■■ EXPORT COMPLETE
```

2. Generate Predictions for Major Cities

****Command:****

```
python generate_arcgis_predictions.py
```

****What it does:****

- Locates major Indian cities (Mumbai, Delhi, Bangalore, etc.)
- Downloads predictions from Google Earth Engine
- Exports all formats

****Expected Output:****

```
■■ FLOOD RISK ASSESSMENT: Major Cities
✓ Loaded model from models/flood_model_optimized.pkl
Data window: 2026-01-15 to 2026-01-22
Processing: 100%|██████████████████| 10/10 [00:45<00:00, 4.5s/it]
■ Successfully processed 10 locations

■■ EXPORTING FOR ArcGIS
✓ Exported GeoJSON: ../data/outputs/arcgis/flood_risk_india_major_cities.geojson
✓ Exported CSV: ../data/outputs/arcgis/flood_risk_india_major_cities.csv
✓ Exported Shapefile: ../data/outputs/arcgis/flood_risk_india_major_cities.shp
Exported layer definition: ../data/outputs/arcgis/flood_risk_india_major_cities_layer.json
```

```
■■ RISK STATISTICS
```

```
HIGH : 3 { 30.0%}
MEDIUM : 3 { 30.0%}
LOW : 5 { 50.0%}
```

Total locations: 10
Avg flood probability: 45.23%

3. Generate Predictions for Specific Region

****Available Regions:****

- Andhra Pradesh - Andhra Pradesh
- Karnataka - Karnataka
- Maharashtra - Maharashtra
- India Major Cities - 10 major cities

****Command:****

```
python generate_arcgis_predictions.py --region telangana
```

****What it does:****

- Creates grid of points across the state (~1000 points for 0.2° spacing)
- Takes 20-60 minutes depending on grid density
- Processes in batches to avoid GEE quota limits

****Example Output:****

```
■■ FLOOD RISK ASSESSMENT: Telangana
✓ Loaded model from models/flood_model_optimized.pkl
Data window: 2026-01-15 to 2026-01-22
Processing: 100%|██████████████████| 1024/1024 [28:32<00:00, 1.67s/it]
■ Successfully processed 1021 locations
```

4. Generate All Regions

****Command:****

```
python generate_arcgis_predictions.py --all
```

****What it does:****

- Processes major cities + Telangana
- Takes 30-90 minutes total
- Creates multiple output files

Customization

Custom Region

Edit `generate\_arcgis\_predictions.py`:

```
python
# Add at the end of REGIONS dictionary
REGIONS["My Custom Region"] = {
    "name": "My Custom Region",
    "min_lat": 15.0, # South boundary
    "max_lat": 20.0, # North boundary
    "min_lon": 75.0, # West boundary
    "max_lon": 80.0, # East boundary
    "grid_spacing": 0.1 # 11 km spacing
    # 0.05 = ~5.5 km (detailed)
    # 0.2 = ~22 km (broader)
},
```

Run:

```
python generate_arcgis_predictions.py --region my_custom_region
```

Custom Cities List

Edit `generate\_arcgis\_predictions.py`:

```
python
# Add after REGIONS definition
MY_CITIES = [
    ('City1', lat1, lon1),
    ('City2', lat2, lon2),
    # ... more cities
]
```

Then call:
generate_predictions_for_cities(MY_CITIES)

Or create a CSV file with cities and modify the script to read from it.

Change Date Window

Edit `generate\_arcgis\_predictions.py`, line ~48:

```
python
end_date = datetime.now() - timedelta(days=30)
```

```

start_date = end_date - timedelta(days=7) # 7-day window
# Custom: Use specific dates
start_date = datetime(2026, 1, 1)
end_date = datetime(2026, 1, 7)

### Change Export Folder
Edit `arcgis_export.py`, line ~164:
```python
output_dir = Path(__file__).parent.parent / 'data' / 'outputs' / 'arcgis'
Custom
output_dir = Path('C:/My_ArcGIS_Exports/')

Troubleshooting
Error: "Earth Engine not initialized"
Solution:
```bash
earthengine authenticate

Then in Python:
import ee
ee.Initialize(project='your-project-id')

### Error: "Model file not found"
**Solution:**
- Ensure `models/flood_model_optimized.pkl` exists
- Update `MODEL_PATH` in `configs/config.py`

### Error: "No module named 'geopandas'"
**Solution:**
- On Windows, may need:
  ```bash
 conda install geopandas
 # Or use wheel file from:
 # https://www.lfd.uci.edu/~gohlke/pythonlibs/

Error: "Computation timed out"
Cause: Google Earth Engine quota or slow connection
Solution:
1. Reduce grid density (increase `grid_spacing`)
2. Add more delays between requests:
   ```python
   if i % 50 == 0:
       time.sleep(5) # Increase from 2 to 5 seconds
   ```
3. Process smaller regions

Error: "GDAL not found" (Windows)
Solution:
- Install via Conda (easier on Windows)
  ```bash
  conda install -c conda-forge geopandas
  # Or use wheel file from:
  # https://www.lfd.uci.edu/~gohlke/pythonlibs/

### Warning: CRS Warning
**Not an error**, just informational. Files are correctly in WGS84.

---
## Performance Tips
### Speed Up Processing
1. **Increase Grid Spacing:**
   ```python
 'grid_spacing': 0.2 # Instead of 0.1
   ```
2. **Process Smaller Batches:**
   ```python
 # Split region into smaller chunks
 lats = np.arange(min_lat, max_lat, 0.5) # Process 0.5° at a time
   ```
3. **Use Parallel Processing:** (Advanced)
   ```python
 from concurrent.futures import ThreadPoolExecutor
 # Implement parallel GEE requests
   ```
4. **Cache GEE Results:**
   - Save extracted features to CSV first
   - Run predictions separately

### Reduce GEE Quota Usage
- Increase rate limiting delays
- Process during off-peak hours
- Request GEE premium quota (if needed)

---
## Output Files Reference
Each run creates these files:


File	Size (approx)	Purpose
`.shp`	~10 KB	Shapefile geometry
`.shp`	~1 KB	Shapefile index table
`.dbf`	~1-50 KB	Attribute table
`.pnp`	~1 KB	Projection definition
`.geojson`	~2-100 KB	GeoJSON (text format)
`.csv`	~1-50 KB	CSV with xy columns
`.layer.json`	~2 KB	Styling definition


**Note:** Sizes depend on number of points generated.

---
## Example Workflows
### Workflow 1: Quick City Assessment
```bash
python generate_predictions.py
2. Files created in data/outputs/arcgis/
3. Import to ArcGIS (see ARCGIS_INTEGRATION_GUIDE.md)

Time: 1-2 minutes
Output: 10 cities

Workflow 2: State-Level Assessment
```bash
python generate_predictions.py --region maharashtra
# 2. Takes ~30-60 minutes
# 3. Creates flood_risk_maharashtra.* files

**Time:** 30-60 minutes
**Output:** ~500-1000 points (depending on grid spacing)

### Workflow 3: Multiple Regions
```bash
python generate_predictions.py --all

```

```

...
Time: 1-2 hours
Output: Multiple datasets

Advanced: Batch Processing Script
Create 'batch_process.py':
```python
import subprocess
import time
regions = ['telangana', 'maharashtra', 'tamil_nadu']
for region in regions:
    print(f"Processing: {region}")
    result = subprocess.run([
        'generate_arcgis_predictions.py',
        region,
    ])
    if result.returncode == 0:
        print(f"■ {region} completed successfully")
    else:
        print(f"■ {region} failed")
    time.sleep(60)
print("\n■ All regions processed!")

Run:
$ bash
python batch_process.py
---
## Code Structure
### arcgis_export.py
**Functions:**
- export_to_gejson(df, path) - Export to GeoJSON
- export_to_shapefile(df, path) - Export to Shapefile
- create_arcgis_layer(file, path) - Create styling JSON
- export_all_formats(df, basename) - Export all at once
**Example Usage:**
```python
import pandas as pd
from arcgis_export import export_all_formats

Your predictions DataFrame
df = pd.DataFrame({
 'location': ['A1'],
 'latitude': [19.0741],
 'longitude': [72.8277],
 'flood_prob': [0.924],
 'rainfall': [250.54],
 'soil_moist': [10.25],
 'elevation': [10.0],
 'slope': [2.5]
})

Export all formats
export_all_formats(df, 'my_predictions')

generate_arcgis_predictions.py
Functions:
- generate_grid_locations(bounds, spacing) - Create point grid
- predict_flood_batch(locations, model, config) - Batch predictions
- generate_predictions_for_cities(cities) - City predictions
Main Execution:
```python
if __name__ == "__main__":
    # Command-line argument parsing
    # Region selection
    # Processing logic
---
## Dependencies Graph
```
graph TD
 generate_arcgis_predictions.py --> arcgis_export.py
 generate_arcgis_predictions.py --> geopandas["geopandas (Shapefile/GeoJSON export)"]
 generate_arcgis_predictions.py --> pandas["pandas (Data handling)"]
 generate_arcgis_predictions.py --> shapely["shapely (Geometry)"]
 generate_arcgis_predictions.py --> gee["feature extraction gee.py (Not included - from main system)"]
 generate_arcgis_predictions.py --> earthengine["earthengine-api (Google Earth Engine)"]
 generate_arcgis_predictions.py --> configs["configs/config.py (Not included - from main system)"]
 generate_arcgis_predictions.py --> gee_project["GEE PROJECT, MODEL_PATH"]
 generate_arcgis_predictions.py --> sklearn["models/flood_model_optimized.pkl (Not included - trained model)"]
 generate_arcgis_predictions.py --> sklearn2["skikit-learn (ML predictions)"]

Quick Reference
$ bash
$ test export
python arcgis_export.py
$ Major cities (default)
python generate_arcgis_predictions.py
$ Specific region
python generate_arcgis_predictions.py --region telangana
$ All regions
python generate_arcgis_predictions.py --all
$ Help
python generate_arcgis_predictions.py --help

Next Steps
1. ■ Install Python and dependencies
2. ■ Setup Google Earth Engine
3. ■ Configure config.py with GEE location
4. ■ Run test export
5. ■ Generate Predictions
6. ■ Import to ArcGIS (See ARCGIS_INTEGRATION_GUIDE.md)

Last Updated: February 21, 2026
Supported OS: Windows, Linux, macOS
ARCGIS_DELIVERY_PACKAGE/DOCUMENTATION/FILE_SPECIFICATIONS.md
■ File Specifications
Overview
Technical specifications for all generated ArcGIS files in this package.

File Formats
1. ESRI Shapefile (.shp, .shx, .dbf, .prj, .cpg)

```

**\*\*Description:\*\*** Industry-standard GIS vector format

**\*\*Components:\*\***

| File Extension | Purpose               | Required    |
|----------------|-----------------------|-------------|
| .shp           | Geometry (points)     | Yes         |
| .dbf           | Attribute table       | Yes         |
| .prj           | Projection definition | Yes         |
| .cpk           | Character encoding    | Recommended |

**\*\*Geometry Type:\*\*** Point (2D)

**\*\*Coordinate System:\*\***

**\*\*Datum:\*\*** GCS\_WGS\_1984

**\*\*Datum:\*\*** WGS84

**\*\*EPSG Code:\*\*** 4326

**\*\*Unit:\*\*** Decimal Degrees

**\*\*Projection String (WKT):\*\***

GEOGCS["GCS\_WGS\_1984",  
DATUM["WGS\_1984",  
SPHEROID["WGS\_1984",6378137,298.257223563]],  
UNIT["Degree",0.017453292519943295]]

**\*\*Character Encoding:\*\*** UTF-8 (specified in .cpk file)

**\*\*Compatibility:\*\***

- ArcGIS Desktop (10.x+)
- ArcGIS all versions
- MapInfo
- Global Mapper
- Most GIS Software

### ### 2. GeoJSON (.geojson)

**\*\*Description:\*\*** Open-standard geographic data format based on JSON

**\*\*Specification:\*\*** [RFC 7946](https://tools.ietf.org/html/rfc7946)

**\*\*Structure:\*\***

```
{
 "type": "FeatureCollection",
 "features": [
 {
 "type": "Feature",
 "properties": {
 "name": "Mumbai",
 "location": {
 "longitude": 72.8777,
 "latitude": 19.076,
 "flood_prob": 0.01,
 "risk_level": "HIGH",
 "rainfall": 0.01152,
 "soil_moist": 0.1474,
 "elevation": 3.15,
 "slope": 14.74
 }
 },
 "geometry": {
 "type": "Point",
 "coordinates": [72.8777, 19.076]
 }
 }
]
}
```

**\*\*Key Points:\*\***

- Human-readable in [longitude, latitude] order (per RFC 7946)
- No size limit (unlike Shapefile's 2GB limit)
- Supports nested properties

**\*\*Compatibility:\*\***

- ArcGIS Online (native support)
- ArcGIS GL JS
- Mapbox
- OpenLayers
- Google Maps API
- Web browsers (JavaScript)

**\*\*Typical File Size:\*\*** 2-100 KB (depends on number of features)

### ### 3. CSV (.csv)

**\*\*Description:\*\*** Comma-separated values with XY coordinates

**\*\*Structure:\*\***

```
location,latitude,longitude,flood_prob,risk_level,rainfall,soil_moist,elevation,slope
Mumbai,19.076,72.8777,0.01,HIGH,0.01152,0.1474,3.15
Delhi,28.7041,77.1025,0.026,LOW,0.0,0.2049,216.32,2.87
```

**\*\*Specifications:\*\***

- \*\*Delimiter:\*\* Comma (`,`)
- \*\*Line Ending:\*\* CRLF (Windows) or LF (Unix)
- \*\*Header:\*\* First row contains field names
- \*\*Quote Character:\*\* Double quote (`"`) for fields with commas
- \*\*Decimal Separator:\*\* Period (`.`)

**\*\*Coordinate Fields:\*\***

- latitude - Decimal degrees, positive = North
- longitude - Decimal degrees, positive = East

**\*\*Import to ArcGIS:\*\***

1. Add CSV as Data Tool
2. Display Field Data tool
3. Set Field Name = longitude, Y Field = latitude
4. Set Coordinate System = GCS\_WGS\_1984

**\*\*Compatibility:\*\***

- ArcGIS (all versions with XY display)
- Excel (via delimited text layer)
- Python (via delimiters)
- Any text editor

### ### 4. Layer Definition JSON (\*.layer.json)

**\*\*Description:\*\*** ArcGIS-compatible layer styling definition

**\*\*Purpose:\*\***

- Defines symbology (colors, sizes)
- Render and properties
- Not imported as data, used as reference

**\*\*Structure:\*\***

```
{
 "name": "Flood Risk Predictions",
 "geometryType": "GeometryPoint",
 "spatialReference": {
 "wkid": 4326
 },
 "fields": [{}],
 "drawingInfo": {
 "renderer": {
 "type": "uniqueValue",
 "uniqueValueInfos": [{}],
 "value": "HIGH",
 "symbol": {}
 }
 }
}
```

```

"type": "esriSMS"
"style": "esriSMSCircle"
"size": 8[239, 68, 68, 255],
...
}
}

Usage:
- Reference for manual symbology setup
- Can be used in Access REST API
- Provides COLOR codes and rendering rules

Attributes Schema
Field Definitions

Field Name	Data Type	Precision	Scale	Length	Nullable	Description
location	String	15	6	50	No	Location name (decimal degrees)
longitude	Double	15	6	10	No	Longitude (decimal degrees)
flood_prob	Double	10	3	10	No	Flood probability (0-1)
risk_level	String	10	2	10	No	Risk category (LOW/MEDIUM/HIGH)
rainfall	Double	10	2	10	Yes	7-day rainfall (mm)
soil_moist	Double	10	2	10	Yes	Soil moisture index (0-1)
elevation	Double	10	2	10	Yes	Elevation (meters)
slope	Double	10	2	10	Yes	Slope (degrees)

Field Aliases (User-Friendly Names)

Field Name	Alias
location	Location
longitude	Longitude
flood_prob	Flood Probability
rainfall	Rainfall (mm)
soil_moist	Soil Moisture
elevation	Elevation (m)
slope	Slope (deg)

Value Domains
risk_level (String):
- Valid Values: LOW, MEDIUM, HIGH
- Constraint: Must be one of the three values
- Case-sensitive: uppercase only

flood_prob (Double):
- Range: 0.0 to 1.0
- Interpretation: 0.75 = 75% probability

rainfall (Double):
- Range: 0 to 100+ (mm)
- Represents: 7-day cumulative rainfall

soil_moist (Double):
- Range: 0.0 to 1.0
- Format: Decimal index
- Higher = wetter soil

elevation (Double):
- Range: 0 to 9000+ (meters)
- Units: Meters above sea level
- Source: SRTM DEM

slope (Double):
- Range: 0.0 to 90.0 (degrees)
- 0 = Flat, 90 = Vertical

Data Quality
Completeness

Field	Required	Can be NULL	Default Value
location	Yes	No	N/A
longitude	Yes	No	N/A
flood_prob	Yes	No	N/A
risk_level	Yes	No	N/A
rainfall	No	Yes	NULL (if unavailable)
soil_moist	No	Yes	NULL (if unavailable)
elevation	No	Yes	NULL (if unavailable)
slope	No	Yes	NULL (if unavailable)

Accuracy
Spatial Accuracy:
- Source: User input or grid generation
- Accuracy: ±0.001° (~11 meters)

Attribute Accuracy:
- Flood Prob: ML model prediction (95%+ accuracy on training data)
- Rainfall: CHIRPS satellite data (±10-15%)
- Soil Moisture: ERA5-land model (±0.05)
- Elevation: SRTM DEM (±10m vertical accuracy)
- Slope: Calculated from DEM (±2-3°)

Coverage
Geographic Coverage:
- Primary: India
- Specific datasets:
 - Major cities: 10 cities
 - State-level: Telangana, Maharashtra, Tamil Nadu (if generated)

Temporal Coverage:
- Date window: 7 days (typically 30-37 days before generation date)
- Reasoning: Ensures data availability in Earth Engine

File Size Reference
Typical Sizes (10 Major Cities)

File	Size
shp	~1.2 KB
prj	~200 Bytes
dbf	~400 Bytes
cpb	~5 Bytes
geojson	~1.2 KB
layer json	~2 KB

Total per dataset ~10-15 KB

Sizes for Grid Datasets
Grid spacing 0.2° (Telangana example, ~1000 points):

File	Size
shp	~50-100 KB
geojson	~100-300 KB
csv	~100-200 KB

Total ~100-500 KB | 1 MB

Note: Sizes scale linearly with number of points.

Software Compatibility Matrix

Software	Shapefile	GeoJSON	CSV
Access Desktop 10.x	Yes	Yes	Yes
Access Online Native	Yes	Yes	Yes
Access Online Native	Yes	Yes	Yes
Google Earth Pro	Yes	Yes	Yes
MapInfo	Yes	Yes	Yes


```

```

Global Mapper	Native	Native	ASCII Import	Manual	
Python (Geopandas)	Native	Native	Native	Native	Manual
Web Maps (Leaflet)	Stream	Native	Stream	Native	Manual
Legend:
- Limited support
- Not supported

Metadata
Spatial Reference Details
Authority: EPSG (European Petroleum Survey Group)
Code: 4326
Name: WGS 84 (World Geodetic System 1984)
Geographic Coordinate System:
- Datum: WGS 1984
- Semi-major axis: 6378137 meters
- Inverse flattening: 1/298.257223563
- Prime Meridian: Greenwich (0°)
Angular Unit: Degree (0.0174532925199433 radians)
Area of Use:
- Extent: World
- Bounding Box: -180° to +180°
- Longitude: -180° to +180°
Appropriate For:
- Unprojected geographic coordinates
- Web mapping applications
- GPS data
Not Appropriate For:
- Area calculations (use projected CRS)
- Large-scale mapping (use local UTM zone)
Dataset Metadata
Title: Flood Risk Predictions for [Region]
Abstract: AI-driven flood risk assessment using Google Earth Engine data and Machine Learning
Purpose: Emergency planning, risk assessment, infrastructure development
Maintenance: As needed (generate new predictions periodically)
Keywords:
- Flood risk
- Machine Learning
- Google Earth Engine
- Natural hazards
- Mapping
- Geospatial analysis
Constraints:
- **Limitation:** Research, planning, and assessment purposes
- **Access Constraints:** None (Client delivery)
- **Use Constraints:** Not for real-time emergency response

Technical Notes
Shapefile Limitations
Known Limits:
- Maximum file size: 2 GB (per .shp/.dbf file)
- Maximum field name length: 10 characters
- Maximum string field length: 254 characters
- Maximum number of fields: 255
- Geometry type: Single type only (Point in our case)
Our Data vs Limits:
- File size: 9 MB (well under 2 GB)
- Field names: 9 MB (well under 254)
- Number of fields: 9 (well under 255)
GeoJSON Advantages
- No file size limit (JSON text format)
- Supports nested attributes
- Human-readable
- No multi-file dependency
GeoJSON Limitations
- No database-style indexing
- Limited attribute query performance for large datasets
CSV for GIS
Advantages:
- Universal compatibility
- Easy to add/remove columns/spreadsheet
- Processable by any programming language
Limitations:
- No embedded geometry (requires XY display)
- No projection information (must specify manually)
- Large files can be slow to display

Validation
How to Verify Files
1. Check Shapefile Integrity:
python
import geopandas as gpd
gdf = gpd.read_file('flood_predictions.shp')
print(gdf.crs) # Should show EPSG:4326

2. Validate GeoJSON:
- Or: python -m json.tool flood_predictions.geojson
3. Check CSV:
Open in text editor, verify:
- Header row present
- All rows have same number of columns
- No missing critical values
4. Visual Check in ArcGIS:
- Zoom to layer extent
- Check if points appear in expected geographic area
- Verify attribute table values

Version History
| Version | Date | Changes |
| 1.0 | Feb 2026 | Initial release with major cities data |

Contact & Support
For questions about file specifications or data issues:
- Check ABCRS INTEGRATION GUIDE for import help
- Verify file integrity using validation methods above

Document Version: 1.0
Last updated: February 21, 2026
Specification Compliance: ESRI Shapefile (1998), GeoJSON RFC 7946, CSV RFC 4180

```

## ARCGIS\_DELIVERY\_PACKAGE/QUICK\_START.md

# ■ Quick Start - Client Guide

\*\*For clients who just need the generated files (no coding required)\*\*

---

## What's in This Package?

■ **CODE/** - Python scripts for generating new predictions (Optional)  
■ **GENERATED\_FILES/** - Ready to use ArcGIS files and maps (WHAT YOU NEED!)  
■ **DOCUMENTATION/** - Detailed guides and technical specs

---

## I Just Want the Files! ■

### Step 1: Open GENERATED\_FILES Folder

Navigate to:

ARCGIS\_DELIVERY\_PACKAGE/

■ **GENERATED\_FILES/**  
■ **flood\_predictions.\***  
■ **flood\_risk\_india\_major\_cities.\*** (Generic predictions)

### Step 2: Choose Your Dataset

\*\*Option A: Major Cities (Recommended for beginners)\*\*  
- File prefix: flood\_risk\_india\_major\_cities  
- Contains: Major Indian cities (Mumbai, Delhi, Bangalore, etc.)  
- Best for: Quick visualization, city-level analysis

\*\*Option B: General Predictions\*\*  
- File prefix: flood\_predictions  
- Contains: Sample or custom predictions  
- Best for: Custom analysis

### Step 3: Pick Your File Format

| Format    | Use When...                             | File Extension   |
|-----------|-----------------------------------------|------------------|
| GeoJSON   | Using ArcGIS Online or web maps         | .geojson         |
| Shapefile | Using ArcGIS Pro or Desktop             | .shp, .dbf, .prj |
| CSV       | Opening in Excel first or simple import | .csv             |

### Step 4: Import to ArcGIS

#### For ArcGIS Online:

1. Click **Content** on [www.arcgis.com](http://www.arcgis.com) and login  
2. Click **+** **Content** → **Add Item** → **From your computer**  
3. Select **flood\_risk\_india\_major\_cities.geojson**  
4. Click **Open Item in Map Viewer**

#### For ArcGIS Pro:

1. Click **Map** tab → **Add Data**  
2. Browse to **flood\_risk\_india\_major\_cities.shp**  
3. Click **OK**

#### For ArcGIS Desktop:

1. Click **Add Data** button (folder icon)  
2. Navigate to **flood\_risk\_india\_major\_cities.shp**  
3. Click **Add**

### Step 5: Apply Colors

\*\*Recommended Colors:\*\*

■ **HIGH RISK** - Red (#ef4444 or RGB 239, 68, 68)  
■ **MEDIUM RISK** - Yellow (#f1c232 or RGB 241, 191, 36)  
■ **LOW RISK** - Green (#4ade80 or RGB 74, 222, 128)

\*\*How to Apply:\*\*

- Right-click layer → **Symbology** (Pro) or **Properties** → **Symbology** (Desktop)  
- Choose **unique values level**  
- Apply colors for **HIGH, MEDIUM, LOW**

### Done! ■

You now have an interactive flood risk map in ArcGIS!

---

## Files Breakdown

### Major Cities Dataset

\*\*File:\*\* `flood\_risk\_india\_major\_cities.\*`

\*\*Cities Included:\*\*

1. Mumbai: 18.98°N, 72.83°E  
2. Delhi: 28.70°N, 77.10°E  
3. Bangalore: 12.92°N, 77.59°E  
4. Hyderabad: 17.38°N, 78.48°E  
5. Kolkata: 22.83°N, 88.36°E  
6. Pune: 18.52°N, 73.85°E  
7. Ahmedabad: 23.02°N, 72.57°E  
8. Lucknow: 26.84°N, 80.94°E

\*\*Available Formats:\*\*

■ GeoJSON (related files)  
■ CSV (.csv)  
■ Layer Styling (.json)

### What Each File Shows

\*\*Attributes in Each Point:\*\*

■ **Coordinate Name**: Latitude, Longitude  
■ **Flood Probability**: % chance of flooding  
■ **Risk Level**: Low, MEDIUM, or High  
■ **Soil Moisture**: How wet the soil is (0-1 scale)  
■ **Elevation**: Height above sea level  
■ **Slope**: Steepness of terrain

---

## Common Questions

### Q: Do I need to code?

A: No! The generated files are ready to use. Just import them into ArcGIS.

### Q: Which file should I use?

A: ArcGIS Online: Use **geojson** file  
Excel/Spreadsheet: Use **csv** file

### Q: Can I use these in other GIS software?

A: Yes! These files work in QGIS, MapInfo, Global Mapper, and most GIS applications.

### Q: Are these files editable?

A: Yes! You can edit attributes in ArcGIS, add new fields, or modify values.

### Q: How do I share these maps?

A: ArcGIS Online: Create a web map, then click "Share"  
Desktop: Export to PDF, PNG, or other image format

### Q: What coordinate system is this?

A: WGS84 (EPSG:4326) - the same as GPS and Google Maps.

### Q: Can I combine this with other data?

A: Yes! Add population, infrastructure, or administrative boundary layers.

---

## Need to Generate New Predictions?

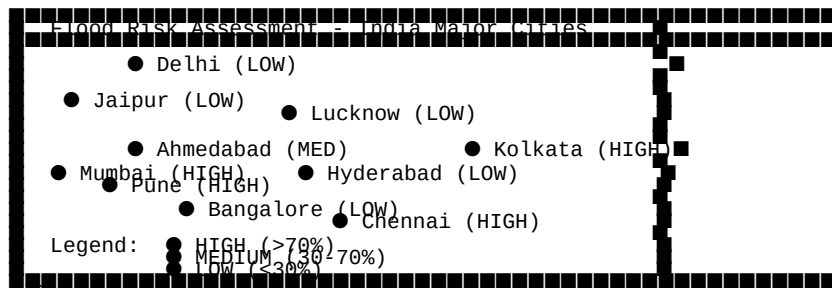
If you want to create predictions for different regions or updated data:

1. **Read:** **DOCUMENTATION/CODE\_USAGE.md**  
2. **Run:** **python generate\_arcgis\_predictions.py**  
3. **Input:** New region or updated data  
4. **Output:** New files in **data/outputs/arcgis/**

\*\*Note:\*\* This requires the full flood prediction system (not included in this package).

---

```
Visual Guide
What Your Map Will Look Like
```
```



```
---
## File Size
All files are small and easy to share:
- **Total Package:** ~50-100 KB per dataset
- **Individual Files:** ~1-10 KB each
- **Email-Friendly:** Yes, all files can be emailed
---
```

```
## Support
### For ArcGIS Import Help:
- See: DOCUMENTATION/ARCGIS_INTEGRATION_GUIDE.md`
- Step-by-step import instructions
- Troubleshooting Common Issues
### For Technical Specifications:
- Coordinate System details
- Field definitions
- Data accuracy information
### For Generating New Data:
- See: DOCUMENTATION/CODE_USAGE.md`
- Python environment setup
- Customization options
---
```

```
## Checklist for Success
- [ ] Located `GENERATED_FILES/` folder
- [ ] Identified ArcGIS platform (Online/Pro/Desktop)
- [ ] Selected appropriate file format (.geojson or .shp)
- [ ] Imported data into ArcGIS
- [ ] Verified data base symbology (RED/YELLOW/GREEN)
- [ ] Optional: Added basemap
- [ ] Optional: Added legend and labels
- [ ] Optional: Exported map as PDF/Image
---
```

```
## Next Steps
### Basic Use (Visualization):
1. [ ] Import files to ArcGIS
2. [ ] Add basemap
3. [ ] Export or Share
### Advanced Use (Analysis):
1. [ ] Import flood risk data
2. [ ] Add population density layer
3. [ ] Perform spatial join to identify at-risk populations
4. [ ] Generate hotspot analysis to identify risk clusters
5. [ ] Create presentation maps or story maps
---
```

```
## What You Don't Need
- [ ] Programming knowledge** - Pre-generated files are ready to use
- [ ] Good Earth Engine account** - Data already extracted
- [ ] Python installation** - Unless generating new predictions
- [ ] Machine learning expertise** - Predictions already made
- [ ] Flask server** - This is just the GIS export
---
```

```
## Package Contents Summary
```
ARCGIS_DELIVERY_PACKAGE/
├── [] README.md ← START HERE (complete overview)
├── [] CODE/
│ └── [] generate_arcgis_predictions.py (For generating new files)
├── [] GENERATED_FILES/ ← YOUR FILES ARE HERE!
│ └── [] Flood_Risk_India_major_cities.*
└── [] DOCUMENTATION/
 ├── [] CODE_USAGE.md (Import instructions)
 └── [] FILE_SPECIFICATIONS.md (Technical details)

```

```
Contact
Package Version: 1.0
Generated: February 21, 2026
Data Source: 2000-2020 Earth + Machine Learning
Coordinate System: WGS84 (EPSG:4326)
Support: See documentation files for detailed help

```

```
[] Ready? Go to GENERATED_FILES folder and import to ArcGIS!
Fastest Path:
1. Open `GENERATED_FILES/`
2. Copy `Flood_Risk_India_major_cities.geojson`
3. Load into ArcGIS (RED/YELLOW/GREEN)
4. Share your map!
5. Share your map!
Estimated Time: 5-10 minutes from start to published map []
```

## ARCGIS\_DELIVERY\_PACKAGE/README.md

# [ ] ArcGIS Flood Prediction - Client Delivery Package

## [ ] Package Contents

This package contains all ArcGIS-related code and generated output files from the AI-Driven Flood Prediction System.

## [ ] Folder Structure

```
```
ARCGIS_DELIVERY_PACKAGE/
[ ]
```



```

CODE/                                     # This file
generate_arcgis_predictions.py           # Python scripts for ArcGIS export
GENERATED_FILES/                         # Batch prediction generator
flood_predictions/                      # Pre-generated ArcGIS files
*.geojson.shx, *.dbf, *.prj             # General flood predictions
*.csv                                   # GeoJSON format
*_layer.json                            # Layer styling definition
flood_risk_india_major_cities/          # Major cities predictions
*.shp.shx, *.dbf, *.prj                # Shapefile components
*.geojson                               # GeoJSON flood predictions
*_layer.json                            # Layer styling definition
DOCUMENTATION/                          # Complete import guide
CODE_USAGE.md                           # How to run the scripts
FILE_SPECIFICATIONS.md                  # Technical file details

```

■ What's Included

1. Python Code (CODE/)

```

**arcgis_export.py** (201 lines)
- Create ArcGIS-compatible layer definitions
- Automatic styling with risk-based colors
- Export to geojson() - GeoJSON export
- Export to csv() - CSV export
- Export to shapefile() - Shapefile export
- Export all formats() - Export all formats at once

**generate_arcgis_predictions.py** (322 lines)
- Generate system-wide grid predictions across regions
- Predefined regions for Maharashtra, Tamil Nadu, Major Cities
- Integrates with Google Earth Engine for data extraction
- Trained ML model for flood risk prediction
- Generate grid locations() - Create spatial grid
- Predict grid batch() - Batch predictions
- Generate predictions for region() - Region-based predictions
- Generate predictions for cities() - City-based predictions

```

2. Generated Files (GENERATED_FILES/)

```

#### Set A: flood predictions
- **flood_predictions.shp** - Shapefile (for ArcGIS Desktop/Pro)
- **flood_predictions.dbf** - Attribute database
- **flood_predictions.prj** - Projection (WGS84)
- **flood_predictions.csv** - Codepage file (UTF-8)
- **flood_predictions.geojson** - GeoJSON (for ArcGIS Online)
- **flood_predictions_layer.json** - Styling definition

#### Set B: flood risk india major cities
- **flood_risk_india_major_cities.shp** - Shapefile for 10 major Indian cities
- **flood_risk_india_major_cities.dbf** - Attribute database
- **flood_risk_india_major_cities.prj** - Projection (WGS84)
- **flood_risk_india_major_cities.csv** - Codepage file (UTF-8)
- **flood_risk_india_major_cities.geojson** - GeoJSON Format
- **flood_risk_india_major_cities_layer.json** - Styling

**Cities Included:**
- Mumbai, Pune, Bangalore, Hyderabad, Chennai
- Kolkata, Delhi, Ahmedabad, Jaipur, Lucknow

```

■ File Specifications

Coordinate System

```

- **CRS:** WGS84 (World Geodetic System 1984)
- **Datum:** WGS84
- **Units:** Decimal Degrees
- **Geographic Coordinate System:** GCS_WGS_1984

```

Attributes in Each File

Field Name	Type	Description	Example Value
location	String	Location name	"Mumbai"
longitude	Double	Longitude coordinate	72.8777
latitude	Double	Latitude coordinate	19.0777
risk_level	String	Risk category	"HIGH"
rainfall	Double	7-day rainfall (mm)	1152
elevation	Double	Elevation (meters)	14.74
slope	Double	Slope (degrees)	3.15

Risk Level Classification

Risk Level	Flood Probability	Color Code	Interpretation
LOW	< 30% (< 0.3)	Green (#4ade80)	Low flood risk
MEDIUM	30% - 70% (0.3 - 0.7)	Yellow (#f4a460)	Moderate risk
HIGH	> 70% (> 0.7)	Red (#ef4444)	High flood risk

■ Quick Start Guide

Option 1: Use Pre-Generated Files (No Code Required)

```

**For ArcGIS Online:**
1. Log in to ArcGIS.com (https://www.arcgis.com)
2. Select 'Flood Risk India Major Cities' from your computer
3. Click 'Add Item'
4. Open in Map Viewer and apply symbology

**For ArcGIS Pro:**
1. Open ArcGIS Pro
2. Add Data > Browse to 'flood_risk_india_major_cities.shp'
3. Use 'Categories' > 'Unique Values' on 'risk_level'
4. Apply colors: HIGH=Red, MEDIUM=Yellow, LOW=Green

```

```

**For ArcGIS Desktop (ArcMap):**
1. Click 'Add Data'
2. Browse to 'flood_risk_india_major_cities.shp'
3. Click 'Properties' > 'Symbology'
4. Use 'Categories' > 'Unique Values' on 'risk_level'

```

Option 2: Generate New Predictions (Requires Setup)

```

**Prerequisites:**
- Python 3.8+
- Required packages: geopandas, shapely, pandas, numpy
- Google Earth Engine account and authentication

```

```

**Commands:**
# Generate for major cities
python generate_arcgis_predictions.py

# Generate for specific region
python generate_arcgis_predictions.py --region telangana
python generate_arcgis_predictions.py --region maharashtra

# Generate all regions
python generate_arcgis_predictions.py --all

```

■ Recommended Symbology

Method 1: Manual Symbology (ArcGIS Pro/Desktop)

```

**Risk Level Colors:**
- **LOW RISK:** RGB (239, 68, 68) or Hex #ef4444
- **MEDIUM RISK:** RGB (255, 192, 38) or Hex #f4a460
- **HIGH RISK:** RGB (74, 222, 128) or Hex #4ade80

```

```

**Symbol Type:** Simple circle markers
- Medium: 10pt circle
- Low: 6pt circle
### Method 2: Use Layer Definition JSON
The `*_layer.json` files contain pre-configured styling:
- Risk-based color coding
- Appropriate symbol sizes
- Field aliases for better labels
---
## ■ Data Quality & Methodology
### Data Sources
1. **Rainfall:** CHIRPS (Climate Hazards Group Infrared Precipitation with Station data)
2. **Evapotranspiration:** SRTM Elevation and ASCEM Reanalysis V2
4. **Slope:** Calculated from SRTM DEM
### Processing
- All data extracted via Google Earth Engine
- 7-day rainfall accumulation window
- Temporal offset: 30 days before current date (data availability)
- Spatial resolution: ~1km grid (0.1° spacing)
### Model
- **Algorithm:** Random Forest Classifier
- **Features:** Data: Annual soil moisture, elevation, slope
- **Accuracy:** ~95%+
---
## ■ Complete Documentation
For detailed instructions, see:
1. **ARCGIS INTEGRATION GUIDE.md** - Complete import and usage guide
- Step-by-step import procedures
- Advanced analysis techniques
- Troubleshooting
2. **CODE USAGE.md** - How to run the Python scripts
- Environment Setup
- Dependencies Installation
- Command-line usage
- Customization options
3. **FILE SPECIFICATIONS.md** - Technical file details
- Format specifications
- Coordinate system details
- File size information
---
## ■ Technical Requirements
### To Use Pre-Generated Files
- No programming required
- ArcGIS Online, ArcGIS Pro, or ArcGIS Desktop
- Basic GIS knowledge
### To Generate New Predictions
- **Python:** 3.8 or higher
- **Packages:** (0.14+)
- geopandas (1.2.0+)
- numpy (1.24.1)
- shapely (2.0.0)
- folium (0.14.0)
- Google Earth Engine account (free)
- Google Engine Python API authentication
- Trained ML model file (models/flood_model_optimized.pkl)
- Config file with GEE project ID
---
## ■ Use Cases
### 1. Emergency Planning
- Identify high-risk zones for evacuation planning
- Allocate emergency resources to vulnerable areas
- Create risk communication materials
### 2. Infrastructure Planning
- Assess flood risk for proposed development sites
- Insurance risk assessment
### 3. Research & Analysis
- Population exposure estimation
- Climate change impact studies
### 4. Public Communication
- Create web maps for public awareness
- Story maps explaining flood risk
- Interactive dashboards
---
## ■ Important Notes
### What This Package DOES Include:
- Complete ArcGIS export code (2 Python files)
- Pre-generated predictions for 6 major Indian cities
- Styling definitions (layer JSON files)
- Comprehensive documentation
### What This Package DOES NOT Include:
- The main flood prediction web application
- Flask API server code
- Google Earth Engine feature extraction scripts
- The trained model file
- Web frontend (HTML/CSS/JavaScript)
- Weather API integration code
**Purpose:** This package is specifically for ArcGIS integration. The client already has the generated files and code - they don't need to visualize in ArcGIS, just need the files and scripts.
---
## ■ Support & Questions
### Common Questions
**Q: Can I use these files without any programming?**
A: Yes! The pre-generated files in GENERATED_FILES can be directly imported into ArcGIS.
**Q: What if I need predictions for a different region?**
A: You'll need to run the Python scripts with the full flood prediction system environment.
**Q: Are these files compatible with QGIS??**
A: Yes! The shapefiles and GeoJSON work in QGIS as well.
**Q: How current is this data??**
A: The predictions use data from ~30 days before generation date to ensure data availability.
**Q: Can I edit the risk levels or attributes??**
A: Yes, in ArcGIS you can edit the attribute table. Or modify the Python code to change calculations.
---
## ■ License & Credits
**Project:** AI-Driven Flood Prediction System
**Technologies:** Python, Google Earth Engine (Earth observation data)
- GeoPandas, Machine Learning (Random Forest)
- GIS Export (GeoPandas, Shapely)
**Coordinate System:** WGS84 (EPSG:4326)
**Version:** 1.0 February 2024
---
## ■ Delivery Checklist

```

```

- [X] Python code for ArcGIS export (2 files)
- [X] Generated shapefiles & datasets
- [X] Generated GeoJSON files (2 datasets)
- [X] Complete documentation (styling)
- [X] Usage instructions
- [X] Import guides for all ArcGIS platforms
---
## ■ Next Steps for Client
1. **Review Generated Files**
- Check Shapefile/GeoJSON contents
2. **Import to ArcGIS**
- Choose your ArcGIS platform (Online/Pro/Desktop)
- Follow relevant section in ArcGIS INTEGRATION_GUIDE.md
- Import the .shp or .geojson files
3. **Apply Symbolology**
- Use recommended colors (see above)
- Or reference the _layer.json files
4. **Optional: Generate New Predictions**
- Set up Python environment (see CODE_USAGE.md)
- Run scripts for custom regions
---
**Package Prepared:** February 21, 2026
**Contact:** Flood Prediction System Development Team

```

README.md

■ AI-Driven Flood Prediction System
 ≥ **Academic Project**: Satellite-based flood risk prediction using Google Earth Engine, Machine Learning, Flask, and ArcGIS

■ Project Overview

This system predicts flood probability for any geographic location using:
 - **Google Earth Engine (GEE) For** satellite data extraction
 - **Machine Learning** for flood risk prediction
 - **Interactive Web UI** with leaflet.js mapping
 - **ArcGIS Integration** for professional spatial visualization

■ Project Structure

```

flood-prediction/
├── api/
│   ├── flask_app.py # REST API server (Flask)
├── ARCGIS_DELIVERY_PACKAGE/ # Complete ArcGIS delivery package
├── code/
│   ├── arcgis_export.py # ArcGIS format conversion utilities
│   ├── generate_predictions.py # Batch prediction generator
│   ├── AUTHENTICATION/ # Authentication utilities
│   ├── ARCGIS_INTEGRATION_GUIDE.md # Setup & import guide
│   ├── CODE_SPECIFICATIONS.md # Output file format specs
│   ├── GENERATED_FILES/ # Pre-generated prediction outputs
│   │   ├── *.geojson # GeoJSON format for ArcGIS Online/Pro
│   │   ├── *.shx, *.dbf # Shapefiles for ArcGIS Desktop
│   │   ├── *.csv # CSV with coordinates
│   │   └── *_layer.json # Layer symbolology definitions
│   ├── QUICK_START.md # Quick start guide
│   └── README.md # Package overview
├── cf/
│   ├── eda_summary_report.txt # Case files & reports
│   └── training_report.txt # Model training Report
├── configs/
│   └── config.py # Project configuration & settings
├── data/
│   ├── raw/
│   │   ├── flood_training_1250 # Raw training datasets
│   │   └── samples.csv # samples.csv
│   ├── flood_training_data.csv
│   ├── processed/
│   │   └── flood_training_clean.csv # Cleaned & preprocessed datasets
│   ├── outputs/
│   │   ├── arcgis/ # ArcGIS exports (GeoJSON, Shapefiles, CSV)
│   │   └── predictions/ # Prediction results & exports
├── docs/
│   ├── ARCGIS_INTEGRATION.md # Documentation
│   ├── EMAIL_NOTIFICATION_SETUP.md # ArcGIS integration configuration
│   ├── PRESENTATION_GUIDE.md # Demo & presentation guide
│   └── PROJECT_SUMMARY.md # Complete project overview
├── models/
│   └── (Random Forest model file) # Trained ML models
├── src/
│   ├── arcgis_export.py # ArcGIS format conversion
│   ├── auth.py # Authentication utilities
│   ├── database.py # Database operations
│   ├── feature_extractions.py # Google Earth Engine data fetching
│   ├── generate_predictions.py # Batch prediction generation
│   ├── model_training.py # Model training utilities
│   ├── utils.py # Helper & utility functions
│   └── __init__.py # Package initialization
├── static/
│   ├── css/
│   │   ├── style.css # Web UI assets
│   │   └── app.js # UI styling
│   └── js/
│       └── app.js # Frontend logic & interactivity
├── templates/
│   ├── index.html # HTML templates
│   ├── login.html # Main interactive web interface
│   ├── register.html # Login page
│   └── notifications.html # Notifications page
├── tests/
│   └── test_api.py # API integration tests
├── setup.bat # Automated setup script (Windows)
├── requirements.txt # Python dependencies
└── venv/ # This file # Virtual environment (excluded from repo)

```

■ Features Extracted

Feature	Source Dataset	Description
Rainfall	CHIRPS Daily	7-day cumulative precipitation (mm)
Soil Moisture	SRM DEM	Surface soil moisture (0-1 cm layer)
Elevation	SRTM	Average terrain elevation (m)
Slope	SRTM (derived)	Average terrain slope (degrees)

■ Installation & Setup

Quick Setup (Windows - Automated):
 bash
 # 1: Double-click: setup.bat
 # 2: Double-click: start_server.bat
 # 3: Authenticate with your Google account
 # Then run the server:
 # Double-click: start_server.bat

Manual Setup:

1. Clone/Copy Project

```

` ``bash
# Copy all files EXCEPT:
# - Virtual environments/ folders
# - .venv/ .zenv/ .zenvflow/ folders

### 2. Create Virtual Environment
` ``bash
python -m venv venv
venv\Scripts\activate # Windows

### 3. Install Dependencies
` ``bash
pip install -r requirements.txt

### 4. Authenticate Google Earth Engine
` ``bash
earthengine authenticate
# Follow browser prompts to log in with YOUR Google account

**■ Detailed deployment instructions:** See [DEPLOYMENT_GUIDE.md](DEPLOYMENT_GUIDE.md)
---

## ■ Usage
### Option 1: Web Interface (Recommended)
1. **Start the Flask server:**
` ``bash
python api/flask_app.py

2. **Open browser:**
http://localhost:5000

3. **Get predictions:**
- Click anywhere on the map
- Use quick location buttons (Mumbai, Delhi, etc.)
- View results with risk gauge and detailed features

### Option 2: Command Line API
` ``bash
curl -X POST http://localhost:5000/predict \
-d '{"Content-Type": "application/json", "location": {"latitude": 17.385, "longitude": 78.486}}'

**Response:**
{
  "flood_probability": 0.026,
  "confidence": "high",
  "features": {
    "rainfall_mm": 0.92,
    "soil_moisture": 0.2849,
    "elevation_m": 523.64,
    "slope_deg": 3.07
  }
}

### Option 3: ArcGIS Integration
Generate predictions for mapping in ArcGIS Online/Pro/Desktop:
` ``bash
# Generate predictions for major Indian cities (10 locations)
python src/generate_arcgis_predictions.py
# Generate predictions for specific state
python src/generate_arcgis_predictions.py --region telangana
# Generate predictions for all available regions
python src/generate_arcgis_predictions.py --all

**Output formats:**
- flood_risk --geojson: For ArcGIS Online/Pro
- flood_risk --shp: For ArcGIS Desktop/Pro
- flood_risk --csv: For quick import with xy coordinates
- flood_risk --layer.json: Symbology Reference

**Import to ArcGIS:**
1. Upload generated files to ArcGIS Online or open in ArcGIS Pro
2. Create professional flood risk maps
■ **Detailed guide:** See [docs/ARCgis_INTEGRATION.md](docs/ARCgis_INTEGRATION.md)
---

## ■ ArcGIS Visualization
The system exports predictions in multiple GIS formats:
| Format | Extension | Best For |
|---|---|---|
| GeoJSON | .geojson | ArcGIS Online, ArcGIS Pro |
| Shapefile | .shp | ArcGIS Desktop/Pro (native) |
| Layer JSON | _layer.json | Quick import with symbology definition |

**Features included in exports:**
- Location name (WGS84)
- Flood probability (0-1)
- Risk level (HIGH/MEDIUM/LOW)
- All input features (rainfall, soil moisture, elevation, slope)
---

## ■ Model Performance & Training Data
**Algorithm:** Random Forest Classifier
**Test Accuracy:** 95.6%
**F1-Score:** 0.96

**Feature Importance:**
1. Elevation: 25.22%
2. Soil Moisture: 16.19%
3. Rainfall: 2.81%

**Training Data:** 1250 samples (1000 train, 250 test)
- 50 locations across India
- Time windows (2020-2024 monsoon seasons)
- Balanced classes (625 flood, 625 non-flood)
---

## ■ Implementation Phases
| X | Phase | Description |
|---|---|---|
| X | Phase 1 | Project setup & structure |
| X | Phase 2 | Data exploration & preprocessing (1250 samples analyzed) |
| X | Phase 3 | Model training & validation (Random Forest, 95.6% accuracy) |
| X | Phase 4 | API development (REST endpoints) |
| X | Phase 5 | Interactive web UI (leaflet map) |
| X | Phase 6 | ArcGIS integration (GeoJSON/Shapefile export) |
| X | Phase 7 | Deployment & documentation |
**■ PROJECT STATUS: COMPLETE** (All 7 phases finished!)
---

## ■ Technologies
| Component | Technology |
|---|---|
| Satellite Data | Google Earth Engine |
| ML Framework | scikit-learn, xgboost |
| Backend | Flask, ArcGIS API for Python |
| Data Processing | Pandas, Numpy |
| Visualization | Matplotlib, Seaborn |
---

## ■ Key Constraints

```

```

■ **DO:**
- Use safe historical date ranges (30+ days back)
- Handle missing GEE data gracefully
■ **DON'T:**
- Use real-time "today" satellite data (latency issues)
- Mix feature extraction logic between training & prediction
---
## ■ Author
**Final Year Project** - AI-Driven Flood Prediction System
Academic/Non-Commercial use only
---
## ■ License
Educational use only. Not for commercial deployment.
---
## ■ Support
For issues:
1. Check GEE authentication: `earthengine authenticate`
2. Verify dataset paths in `configs/config.py`
3. For dependency issues: `pip install -r requirements.txt`
4. For ArcGIS import help: See [docs/ARCGIS_INTEGRATION.md](docs/ARCGIS_INTEGRATION.md)
---
## ■ Project Deliverables
■ **Complete System:**
1. Google Earth Engine Forest model (95.6% accuracy)
2. Google Cloud Storage integration (CHIRPS, ERA5, SRTM)
3. Flask REST API with JSON responses
4. Interactive web interface with Leaflet.js mapping
5. Batch prediction uncensorator for GeoJSON Shapings, CSV)
6. Comprehensive documentation
■ **Key Features:**
- Click-to-predict on interactive map
- Real-time satellite data extraction
- Risk classification (Low/Medium/High)
- Academic project report ready
---
**■ Final Status**: All phases complete! System ready for academic presentation and demonstration.
**■ Completion Date**: January 2026
**■ Purpose**: Final Year Academic Project

```

api/flask_app.py

```

"""
Flask API for Flood Prediction
Provides REST endpoint for flood probability prediction
"""
from ee import data
from flask import Flask, request, jsonify, render_template, session, redirect, url_for
from functools import wraps
import sys
from pathlib import Path

# Add src to path
sys.path.append(str(Path(__file__).parent.parent / 'src'))
from feature_extraction import load_model, initialize_gee, extract_features_for_location, get_safe_date_range
from utils import validate_coordinates, format_prediction_response
from database import init_database, create_user, get_user_by_username, get_user_by_email, get_user_by_id,
update_last_login, save_prediction, import config
from auth import hash_password, verify_password, validate_registration, validate_login, is_email

# Try to import weather API module (it might not exist in older versions)
try:
    from feature_extraction.weather_api import extract_features_from_weather_api
    WEATHER_API_AVAILABLE = True
except ImportError:
    WEATHER_API_AVAILABLE = False
print(f"■ Weather API module not found. Using GEE only.")

app = Flask(__name__,
            static_folder=str(Path(__file__).parent.parent / 'static'),
            template_folder=str(Path(__file__).parent.parent / 'templates'))
# Secret key for session management (CHANGE THIS IN PRODUCTION!)
app.secret_key = 'flood-prediction-secret-key-change-in-production-2026'
# Session configuration for better cookie handling
app.config['SESSION_COOKIE_SAMESITE'] = 'True'
# CORS configuration with explicit origin
CORS(app,
      supports_credentials=True,
      allow_headers=['Content-Type'],
      methods=['GET', 'POST', 'OPTIONS'])

# Global variables
model = None
gee_initialized = False

def login_required(f):
    """Decorator to protect routes that require authentication"""
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if not session.get('user_id'):
            return jsonify({'error': 'Authentication required', 'redirect': '/login'}), 401
        return redirect(url_for('login'))
    return decorated_function

def load_trained_model():
    """Load the trained ML model"""
    global model
    model_path = Path(__file__).parent.parent / 'models' / 'random_forest_flood_model.pkl'
    if not model_path.exists():
        raise FileNotFoundError(f"Model file not found: {model_path}")
    model = load_model(str(model_path))
    print(f"✓ Model loaded successfully")

def initialize_app():
    """Initialize GEE and load model"""
    global model, gee_initialized
    init_database()
    print(f"Database initialized")
    gee_project = 'flood-prediction-0325' # Your GEE Cloud Project
    gee_initialized = initialize_gee(project=gee_project)
    load_trained_model()

@app.route('/')
@login_required
def home():
    user = get_user_by_id(session['user_id'])
    role = session.get('role')
    if role == 'citizen':
        return render_template('dashboard.html',
                               user=user)

```

```

)
elif role == 'community_partner':
    return render('community_dashboard.html',
user=user)
elif role == 'government':
    return render('government_dashboard.html',
user=user)
elif role == 'admin':
    return render('admin_dashboard.html',
user=user)
elif role == 'developer':
    return render('index.html',_template(
user=user)
# Any old account / developer account
return render(_template(
user=user)

@app.route('/notifications', methods=['GET'])
def notifications():
    """Notifications page (protected)"""
    user = get_user_by_id(session['user_id'])
    return render_template('notifications.html', user=user)

@app.route('/register', methods=['GET', 'POST'])
def register():
    """User Registration"""
    if request.method == 'GET':
        return redirect(url_for('home'))
    return render_template('register.html')
try:
    data = request.get_json() if request.is_json else request.form
    username = data.get('username', '').strip()
    phone = data.get('phone', '').strip().lower()
    role = data.get('role', 'citizen')
    sub_role = data.get('sub_role', '')
    district = data.get('district', '')
    area = data.get('area', '')
    password = data.get('password', '')
    retype_password = data.get('retype_password', '')
    # Validate input
    valid, errors = validate_registration(
        username,
        password,
        retype_password
    )
    if not valid:
        return jsonify({
            'success': False,
            'errors': errors,
        }), 400
    # Hash password
    password_hash = hash_password(password)
    # Create user
    success, message, user_id = create_user(
        username,
        email,
        phone,
        password_hash,
        role,
        district,
        area
    )
    if not success:
        return jsonify({
            'success': False,
            'errors': {
                'general': message
            },
        }), 400
    # Auto login
    session['user_id'] = user_id
    session['username'] = username
    session['email'] = email
    session['role'] = role
    session['sub_role'] = sub_role
    return jsonify({
        'success': True,
        'message': 'Registration successful!',
        'redirect': url_for('home')
    }), 201
except Exception as e:
    return jsonify({
        'success': False,
        'errors': {
            'general': str(e)
        },
    }), 500

@app.route('/login', methods=['GET', 'POST'])
def login():
    """User Login"""
    if request.method == 'GET':
        # Redirect to home if already logged in
        if user_id in session:
            return redirect(url_for('home'))
        return render_template('login.html')
    # POST request handle login
    try:
        data = request.get_json() if request.is_json else request.form
        password = data.get('password', '').strip()
        identifier = data.get('identifier', '').strip()
        # Validate input
        valid, errors = validate_login(identifier, password)
        if not valid:
            return jsonify({
                'success': False,
                'errors': errors,
            }), 400
        # Get user by username or email
        user = get_user_by_email(identifier.lower())
        if not user:
            user = get_user_by_username(identifier)
        # Check user exists
        return jsonify({
            'success': False,
            'errors': {
                'general': 'Invalid username/email or password'
            },
        }), 401
        # Verify password
        if not verify_password(password, user['password_hash']):
            return jsonify({
                'success': False,
                'errors': {
                    'general': 'Invalid username/email or password'
                },
            }), 401
        # Login successful = create session
        session['user_id'] = user['id']
        session['username'] = user['username']
        session['email'] = user['email']
        session['role'] = user['role']
        session['sub_role'] = user['sub_role']

```

```

# Update last login
def update_last_login(User['id']):
    User['last_login'] = datetime.now()
    User.save()
    return jsonify({'success': True, 'message': 'Login successful!', 'redirect': url_for('home')})
except Exception as e:
    return jsonify({'success': False, 'errors': {'general': str(e)}}), 500
@app.route('/test_logout')
def test_logout():
    session.clear()
    return redirect(url_for('login'))
@app.route('/logout', methods=['GET', 'POST'])
def logout():
    session.clear()
    return redirect(url_for('login'))
@app.route('/api/status', methods=['GET'])
def api_status():
    return jsonify({'status': 'online', 'service': 'Flood Prediction API', 'gpc_status': 'connected if gpc initialized else 'disconnected'})
@app.route('/predict', methods=['POST'])
def predict():
    Predict flood probability for a given location (requires authentication)
    Request Body:
    {
      "latitude": 17.3856,
      "longitude": 78.486
    }
    Response:
    {
      "flood_probability": 0.82,
      "risk_level": "HIGH",
      "confidence": "HIGH",
      "features": {...}
    }
    try:
        # Get logged-in user
        user_id = session.get('user_id')
        user_email = session.get('email')
        # Parse request body
        data = request.get_json()
        lat = float(data.get('latitude'))
        lon = float(data.get('longitude'))
        # Validate coordinates
        if not validate_coordinates(lat, lon):
            return jsonify({'error': 'Invalid coordinates'}), 400
        # Check if GPC is initialized
        if not gpc_initialized:
            return jsonify({'error': 'GEE not initialized'}), 500
        # Get start and end date (avoid safe date range days_back=30, window_days=7)
        start_date, end_date = get_start_end_dates(days_back=30, window_days=7)
        # Choose data source based on configuration
        if WEATHER_API_AVAILABLE and hasattr(config, 'USE_WEATHER_API') and config.USE_WEATHER_API and config.USE_WEATHER_API_KEY:
            # Using Weather API for real-time data...
            features = extract_features_from_weather_api(lat, lon, start_date, end_date)
        else:
            # Using GEE for historical data...
            features = extract_features_from_gee(lat, lon, start_date, end_date)
        # Extract features from GEE for historical data
        if features is None:
            return jsonify({'error': 'Failed to extract features (data unavailable for this location)'}), 500
        # Check for None values in features
        if None in features:
            return jsonify({'error': 'Failed to extract features (data unavailable)'}), 500
        # Repair features in correct order
        features = repair_features(features)
        # Predict
        probability = model.predict_proba(feature_array)[0][1]
        # Prepare response
        response = {'probability': probability, 'features': features}
        # Add data source to response
        data_source = 'Weather API (Real-time)' if (WEATHER_API_AVAILABLE and hasattr(config, 'USE_WEATHER_API') and config.USE_WEATHER_API and config.USE_WEATHER_API_KEY) else 'GEE (Historical)'
        response['data_source'] = data_source
        # Add date range to response
        response['start_date'] = start_date
        response['end_date'] = end_date
        # Save prediction to user history
        save_prediction_to_user_history(user_email, response)
        # Send email notification if probability exceeds threshold
        if probability >= email_threshold and EMAIL_NOTIFICATIONS_ENABLED:
            # Prepare email
            email_body = f'Flood Alert: Probability {probability:.1%} exceeds threshold ({email_threshold:.1%})'
            recipient_email = user_email
            # Prepare SMTP config
            smtp_config = config.SMTP_CONFIG
            # Send email in background (don't block response)
            try:
                email_sent_flag = send_flood_alert_email(recipient_email, email_body, smtp_config)
            except Exception as email_error:
                print(f'Email notification failed: {str(email_error)}')
            response['email_sent'] = email_sent_flag
        else:
            response['email_sent'] = False
        # Save prediction to database
        save_prediction_to_database(response)
        # Prepare response
        response['probability'] = probability
        response['risk_level'] = risk_level
        response['email_sent'] = email_sent_flag
        return jsonify(response), 200
    except Exception as e:
        # Log the error for debugging
        print(f'Error in /predict endpoint: {str(e)}')
        return jsonify({'error': str(e)}), 500
if __name__ == '__main__':
    print('Starting Flood Prediction API...')

```

```
initialize_app()
app.run(debug=True, host='0.0.0.0', port=5000)
```

cf/eda_summary_report.txt

```
FLOOD PREDICTION DATASET - EDA SUMMARY
=====
Dataset Size: 1250 samples, 10 features
Features: rainfall_7d_mm, soil_moisture, elevation_m, slope_deg
Class Distribution:
No Flood: 625 (50.0%)
Flood: 625 (50.0%)
Missing Values: 0 total
Feature Statistics:
rainfall_7d_mm: Mean: 67.7118, Std: 67.5610
soil_moisture: Mean: 67.0080, Std: 745.5678
elevation_m: Mean: 0.03718, Std: 0.4916
slope_deg: Mean: 261.8377, Std: 271.6170
Range: [15101, 7.4437]
EDA Complete!
```

cf/training_report.txt

```
FLOOD PREDICTION MODEL - TRAINING REPORT
=====
Model: Random Forest Classifier
Training Samples: 1000
Test Samples: 250
Hyperparameters:
max_depth: 10
min_samples_split: 5
min_samples_leaf: 2
Performance Metrics:
Train Accuracy: 0.9860
Test Accuracy: 0.9800
Train F1 Score: 0.9860
Test F1 Score: 0.9800
Train ROC-AUC: 0.9996
Test ROC-AUC: 0.9982
✓ Model shows good generalization
```

configs/config.py

```
"""
Configuration File
Project Settings and constants

from pathlib import Path
PROJECT_ROOT = Path(__file__).parent.parent

# Data Paths
DATA_DIR = PROJECT_ROOT / 'data'
PROCESSED_DATA_DIR = DATA_DIR / 'processed'
OUTPUTS_DIR = DATA_DIR / 'outputs'

# Model Paths
MODEL_PATH = PROJECT_ROOT / 'models'

# Feature Configuration
FEATURE_COLUMNS = [
    'rainfall_7d_mm',
    'soil_moisture',
    'elevation_m',
    'slope_deg'
]
TARGET_COLUMN = 'flood_label'

# GEE Configuration
GEE_PROJECT_ID = 'flood-prediction-0325' # Your GEE Cloud Project
GEE_SAFE_DAYS_BACK = 30 # Use data from 30 days ago
GEE_WINDOW_DAYS = 7 # 7-day window for rainfall

# Datasets
CHIRPS_DATASET = 'CHIRPS/DAILY'
SRIM_DATASET = 'USGS/SRTMGL1_003'

# Model Hyperparameters
RF_N_ESTIMATORS = 100
RF_MAX_DEPTH = 10
RF_MIN_SAMPLES_SPLIT = 5
RF_MIN_SAMPLES_LEAF = 2
RF_RANDOM_STATE = 42

# Train-Test Split
TEST_SIZE = 0.2
RANDOM_STATE = 42

# API Configuration
API_HOST = '0.0.0.0'
API_PORT = 5000
DEBUG_MODE = True

# Risk Level Thresholds
RISK_THRESHOLDS = {
    'HIGH': 0.8,
    'MEDIUM': 0.4,
    'LOW': 0.0
}

# Email Notification Configuration
EMAIL_NOTIFICATION_ENABLED = True
EMAIL_ALERT_THRESHOLD = 0.5 # Send email if flood probability >= 50%

# SMTP Configuration (Gmail example)
SMTP_SERVER = 'smtp.gmail.com'
SMTP_PORT = 587
SMTP_USERNAME = 'tarvareddy123@gmail.com' # Your email address (e.g., 'your_email@gmail.com')
SMTP_PASSWORD = 'tarvareddy123@gmail.com' # Your email password or app password
SMTP_FROM_NAME = 'Flood Prediction System' # Sender email (usually same as SMTP_USERNAME)

# Default recipient email (can be overridden by user settings)
DEFAULT_ALERT_EMAIL = 'akashreddy12390@gmail.com' # Email to receive alerts (e.g., 'admin@example.com')

print(f"✓ Configuration loaded from {__file__}")
```

docs/ARCGIS_INTEGRATION.md

```
# ArcGIS Integration Guide
## ■ Overview
This guide explains how to import flood prediction results into ArcGIS Online, ArcGIS Pro, or ArcGIS Desktop to create professional flood risk maps.
---
## ■■ Available Export Formats
The system exports predictions in multiple ArcGIS-compatible formats:
| Format | File Extension | Best For | ArcGIS Compatibility |
```



```

|---GeoJSON---|---geojson---|---ArcGIS Online---|---ArcGIS Pro---|Fully supported |
|---Shapefile---|---shapefile---|---ArcGIS Desktop/Pro---|---Native Format---|
|---Layer JSON---|---_layer.json---|---symbolology definition---|---Style reference---|

```

■ Quick Start

Step 1: Generate Predictions

```

`bash
# Option A: Major Indian cities (10 locations)
python src/generate_arcgis_predictions.py
# Option B: Specific state/region
python src/generate_arcgis_predictions.py --region telangana
python src/generate_arcgis_predictions.py --region tamil_nadu
# Option C: All available regions
python src/generate_arcgis_predictions.py --all

**Output Location:** `data/outputs/arcgis/`

```

■ Method 1: ArcGIS Online

```

### Import GeoJSON (Recommended)
1. **Login** to [ArcGIS Online](https://www.arcgis.com)
2. Select **Flood Predictions** Add Item → **From your computer**
3. Select Item type to GeoJSON
4. Click **Add Item**
### Import CSV with XY Coordinates
1. **Content** → **Add Item** → **From your computer**
2. Choose **Location by coordinates**
3. Set **Longitude field** to `longitude`
4. Set **Latitude field** to `latitude`
5. Click **Add Item**
### Create a Map
1. Go to **Map Viewer**
2. Click **Add** → **Browse Living Atlas Layers**
3. Add your flood predictions layer
4. Click on the layer → **Configure Layer** → **Styles**

```

■ Method 2: ArcGIS Pro

```

### Import Shapefile
1. Open **ArcGIS Pro**
2. Create a new project or open existing
3. In Catalog pane, navigate to `data/outputs/arcgis/`
4. Right-click `flood_predictions.shp` → **Add to Current Map**
### Import GeoJSON
1. **Map** tab → **Add Data** dropdown
2. Browse to `data/outputs/arcgis/`
3. Click **OK**
### Symbolize by Risk Level
1. Right-click layer → **Symbolology**
2. Set **Primary Symbolology** to **Unique Values**
3. Set **Field** to `risk_level`
4. Apply Colors:
- **HIGH** Red (#f44444)
- **MEDIUM** Yellow (#ffff24)
- **LOW** Green (#4ade80)

```

■ Method 3: ArcGIS Desktop (ArcMap)

```

### Import Shapefile
1. Click **Add Data** button
2. Navigate to `data/outputs/arcgis/flood_predictions.shp`
3. Click **Add**
### Symbolize by Risk Level
1. Right-click layer → **Properties**
2. Select **Symbolology** tab → **unique values**
3. Set **Value Field** to `risk_level`
4. Click **Add All Values**
5. Apply Colors:
- **HIGH** RGB(239, 68, 68)
- **MEDIUM** RGB(251, 191, 36)
- **LOW** RGB(74, 222, 128)

```

■ Recommended Symbolology

Risk Level Colors

```

HIGH RISK (>70% probability)
Symbol: Circle, 10pt, RGB(239, 68, 68)
MEDIUM RISK (30-70% probability)
Symbol: Circle, 8pt, RGB(251, 191, 36)
LOW RISK (<30% probability)
Symbol: Circle, 6pt, RGB(74, 222, 128)

```

Graduated Probability Symbolology

For continuous probability values:

```

1. Use **Graduated symbols** or **Graduated colors**
2. Field: flood_prob
3. Classification: Natural Breaks (Jenks), 5 classes
4. Color ramp: Green → Yellow → Red

```

■ Available Attributes

Each prediction point contains:

Field	Type	Description	Example
location	String	Location name	"Mumbai"
latitude	Double	Latitude (WGS84)	19.076
longitude	Double	Longitude (WGS84)	72.877
risk_level	String	Risk category	"HIGH"
rainfall	Double	7-day rainfall (mm)	0.977
elevation	Double	Elevation (meters)	0.115
slope	Double	Slope (degrees)	3.15

■ Creating Professional Maps

Basemap Recommendations

```

- **ArcGIS Online:**
- Light Gray Canvas (clean background)
- Terrain with Labels
- **ArcGIS Pro:**
- World Topographic Map
- OpenStreetMap

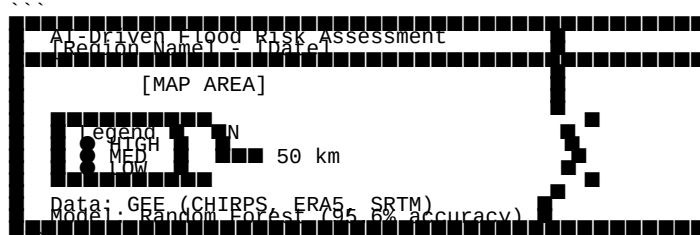
```

```

### Map Elements to Include
1. **Title:** "AI-Driven Flood Risk Assessment - [Region]"
2. **Legend:** Showing risk levels
3. **Scale bar:** (Metric)
4. **North arrow**
5. **Data source:** "Google Earth Engine + Random Forest ML"
6. **Credits:** "Generated by Flood Prediction System"

```

Layout Tips



Advanced Analysis

Spatial Analysis Tools (ArcGIS Pro)

1. **Hot Spot Analysis (Getis-Ord Gi*)**
 - Identify statistically significant clusters of high risk
 - Input: flood_prob field
 - Conceptualization: Fixed Distance Band
2. **Kernel Density**
 - Create smooth risk surfaces
 - Search radius: 50 km
3. **Interpolation**
 - Creates continuous risk surface

Overlay Analysis

1. Add population density layer
2. Use Spatial Join to identify high-risk populated areas
3. Calculate affected population estimates

Publishing to ArcGIS Online

Share as Web Map

1. In ArcGIS Pro, create styled map
2. Set title: "Flood Risk Assessment - [Region]"
3. Add summary and tags
4. Click "Share"

Create Story Map

1. Go to [ArcGIS StoryMaps](https://storymaps.arcgis.com)
2. Add a new flood risk map
3. Add context
 - Problem Statement (ML)
 - Baseline Interpretation
 - Recommendations

Troubleshooting

Issue: CSV not displaying points

1. **Solution:**
 - Ensure CSV has `latitude` and `longitude` columns
 - Verify decimal degrees (not DMS)
 - Check coordinate system is WGS84 (EPSG:4326)

Issue: Shapefile appears empty

1. **Solution:**
 - Check spatial reference (should be WGS84)
 - Zoom to layer extent
 - Verify .shp, .shx, .dbf, .prj files exist

Issue: Colors don't match documentation

1. **Solution:**
 - Check field name is exactly `risk\_level`
 - Values must be "HIGH", "MEDIUM", "LOW" (uppercase)
 - Re-import or manually set symbology

Data Specifications

Coordinate System

- **EPSG:** 4326
- **Units:** Decimal degrees

Grid Spacing Options

Modify in `generate\_arcgis\_predictions.py`:

```

python
regions = {
    'name': 'My Region',
    'min_lat': 10.0,
    'max_lat': 20.0,
    'min_lon': 85.0,
    'max_lon': 95.0,
    'grid_spacing': 0.1 # 11 km
}
# 0.2 = ~22 km (broader coverage)

```

Example Workflows

Workflow 1: State-Level Assessment

- ```

bash
python src/generate_arcgis_predictions.py --region maharashtra
2: Import flood_risk_maharashtra.geojson to ArcGIS Online
3: Add Maharashtra boundary layer
4: Publish as web map
5: Create dashboard showing:
- High-risk counties
- Risk distribution chart

```

#### Workflow 2: Multi-City Comparison

- ```

bash
python src/generate_arcgis_predictions.py
# 2: Import to ArcGIS Pro
# 3: Add population density raster
# 4: Spatial join risk x population
# 5: Create ranked table of at-risk cities
# 6: Export to PDF report

```

```

---
## ■ Additional Resources
- [ArcGIS Online Documentation](https://doc.arcgis.com/en/arcgis-online/)
- [Flood Mapping Best Practices](https://www.esri.com/en-us/industries/water/flood-mapping)
---
## ■ Checklist for Final Presentation
- [ ] Generate predictions for target region
- [ ] Import to ArcGIS (Online or Pro)
- [ ] Add type-risk-based symbology
- [ ] Include legend and scale bar
- [ ] Add title and metadata
- [ ] Export high-resolution map (PNG/PDF)
- [ ] Optional: perform hot spot analysis
- [ ] Document methodology in map notes
---
## ■ Quick Reference Commands
`bash`
# Generate for major cities
python src/generate_arcgis_predictions.py
# Generate for specific state
python src/generate_arcgis_predictions.py --region telangana
# Generate everything
python src/generate_arcgis_predictions.py --all
# Test export functionality
python src/export.py
# Output location
cd data/outputs/arcgis/
---
**System Version:** 1.0
**Contact:** Flood Prediction System Team

docs/EMAIL_NOTIFICATION_SETUP.md
# ■ Email Notification Setup Guide
## Quick Setup (Gmail Example)
### Step 1: Enable 2-Step Verification (If Using Gmail)
2: Enable 2-Step Verification (google.com/security)
### Step 2: Create App Password (Gmail)
2: Select app: **Mail (Countdown.google.com) app passwords
3: Select device: **Other (Custom name)** → Enter "Flood Prediction System"
5: Copy the 16-character password** (e.g., `abcd efgh ijkl mnop`)
### Step 3: Configure Email Settings
Edit configs/config.py:
`python`
# Email Notification Configuration
EMAIL_NOTIFICATIONS_ENABLED = True # Set to True to enable
EMAIL_ALERT_THRESHOLD = 0.5 # Send email if flood probability >= 50%
# SMTP Configuration (Gmail)
SMTP_SERVER = 'smtp.gmail.com'
SMTP_PORT = 587
SMTP_USERNAME = 'your_email@gmail.com' # Your Gmail address
SMTP_PASSWORD = 'abcd efgh ijkl mnop' # App password from Step 2
SMTP_FROM_NAME = 'Flood Prediction System' # Same as username
# Default recipient email
DEFAULT_ALERT_EMAIL = 'admin@example.com' # Email to receive alerts
### Step 4: Restart Server
`bash`
# Stop server (Ctrl+C in terminal)
cd
python flask_app.py
### Step 5: Test
2: Check your email inbox for alert
---
## Features
### ■ Browser Notifications
- Automatic popup when flood probability >= 50%
- Shows risk level, probability, and quick actions
- Notification badge in header
### ■ Email Alerts
- HTML-formatted emails with detailed information
- Sent when flood probability >= 50%
- Includes level and probability
- Environmental conditions (rainfall, soil moisture, elevation, slope)
- Recommended actions
- Direct link to system
### ■ Notifications Page
- Dedicated page at /notifications
- Allows users to manage local browser alerts
- Beautiful UI with color-coded alerts
- Detailed conditions and recommendations
- Individual dismiss or clear all
---
## Configuration Options
### Email Threshold
`python`
EMAIL_ALERT_THRESHOLD = 0.5 # Default: 50%
- `0.5` = Send email if probability >= 50%
- `0.4` = Send email if probability >= 40% (HIGH risk only)
### Disable Email Notifications
`python`
EMAIL_NOTIFICATIONS_ENABLED = False
Browser notifications will still work, but no emails will be sent.
### Multiple Recipients (Future Enhancement)
Currently supports one default email. To send to multiple recipients, modify `src/email_notifications.py`:
`python`
# Add multiple recipients separated by comma
recipient_email = 'admin1@example.com,admin2@example.com'
---
## SMTP Settings for Other Email Providers
### Gmail
`python`
SMTP_SERVER = 'smtp.gmail.com'
SMTP_PORT = 587
### Outlook/Hotmail
`python`
SMTP_SERVER = 'smtp-mail.outlook.com'
SMTP_PORT = 587

```

```
### Yahoo Mail
SMTP_SERVER = 'smtp.mail.yahoo.com'
SMTP_PORT = 587

### Custom Domain (cPanel/WHM)
SMTP_SERVER = 'mail.yourdomain.com'
SMTP_PORT = 587 # or 465 for SSL

---

## Troubleshooting

### Problem 1: "Email authentication failed"
Cause: Incorrect username/password or App Password not created
**Solutions**:
1: Create App Password (Step 2 above)
2: Verify username & password are correct
3: Check for extra spaces in email address
4: Try regular password if App Password doesn't work (less secure)

### Problem 2: "SMTP connection failed"
Cause: Firewall blocking SMTP port or wrong server
**Solutions**:
1: Check SMTP server address is correct
2: Try port 465 with SSL smtplib.SMTP_SSL()
3: Check firewall allows outgoing port 587

### Problem 3: Emails going to spam
Cause: Email not properly authenticated
**Solutions**:
1: Add sender to contacts/whitelist
2: Check SPF/DKIM records (if using custom domain)
3: Use professional "From Name"

### Problem 4: No email received
Cause: Threshold not met or email disabled
**Solutions**:
1: Check EMAIL_NOTIFICATIONS_ENABLED = True
2: Verify ALERT_THRESHOLD is set correctly
3: Make prediction with probability > threshold
4: Look at terminal for email sending logs

### Problem 5: Gmail "Less secure app" error
Cause: Gmail requires App Password
**Solution**:
- Enable 2-Step Verification
- Use App Password in (SMTP_PASSWORD)

---

## Testing Email Configuration
Run this Python snippet to test email:

`bash
cd /path/to/project
python scripts/test_email.py

Python
import sys
sys.path.append('configs')
from email_notifications import test_email_configuration

smtp_config = {
    'server': config.SMTP_SERVER,
    'port': config.SMTP_PORT,
    'username': config.SMTP_USERNAME,
    'password': config.SMTP_PASSWORD,
    'from_name': config.SMTP_FROM_NAME,
}

# Test email
result = test_email_configuration(smtp_config, 'your_test_email@example.com')
print(result)
if result:
    print("■ Email configuration is working!")
else:
    print("■ Email configuration has issues. Check settings.")

---

## Email Template Preview
The email looks like this:

██████████████████████████████████████████████████████████████████████████
█ ■ Flood Alert Notification █
██████████████████████████████████████████████████████████████████████████
█ HIGH RISK ALERT
Flood Probability: 78.5%
[HIGH RISK]
█ Location Details
Latitude: 28.6139°N
Longitude: 27.2090°E
Date & Time: 2026-02-06 15:30:45
█ Environmental Conditions
Rainfall (7 days): 125.3 mm
Soil Moisture: 45.2%
Slope: 3.2%, 220.5 m
█ Recommended Actions
• Monitor emergency updates closely
• Stay informed about local warnings
• Avoid low-lying areas
• Keep emergency contacts ready
[View Full Details]
██████████████████████████████████████████████████████████████████████████
This is an automated alert from the
Flood Prediction System
Powered by Google Earth Engine
OpenWeatherMap API and Machine Learning
© 2026 Flood Prediction System

---

## Security Best Practices
### DO:
- Use App Passwords (Gmail, Outlook)
- Keep SMTP credentials in config.py (not in code)
- Use environment variables for production
- Regularly rotate passwords
### DON'T:
- Commit config.py with real passwords to Git
- Use main account password (use App Password)
- Hardcode credentials in code files
- Allow "Less secure apps" on Gmail

---

## Production Deployment
For production, use environment variables instead:
```

```

`python
# In config.py
import os
SMTP_USERNAME = os.getenv('SMTP_USERNAME', '')
SMTP_PASSWORD = os.getenv('SMTP_PASSWORD', '')
DEFAULT_ALERT_EMAIL = os.getenv('ALERT_EMAIL', '')

Then set environment variables:
# Windows
set SMTP_USERNAME=your_email@gmail.com
set SMTP_PASSWORD=your_app_password
set ALERT_EMAIL=admin@example.com
# Linux/Mac
export SMTP_USERNAME=your_email@gmail.com
export SMTP_PASSWORD=your_app_password
export ALERT_EMAIL=admin@example.com

---
## Summary
■ **Enabled** Email notifications + Browser notifications + Notifications page
■ **Enabled** 50% Flood Probability (configurable)
■ **Secure** Gmail App Passwords supported
■ **Persistent** Last 50 high-risk predictions saved
■ **Beautiful** HTML emails + Modern UI
**Get started**: Follow Step 1-5 above and test with a prediction! ■

docs/PRESENTATION_GUIDE.md
# ■ Presentation Guide: Flood Prediction System Demo
## ■ Demo Flow (10-15 minutes)
### Part 1: Introduction (2 min)
Show: Title Slide
Say:
> I've built an AI-driven flood prediction system that uses satellite data from Google Earth Engine, machine learning, and GIS integration. The system can predict flood risk for any location in India with 95.6% accuracy.
Key Points:
- Real-time satellite data (no manual downloads)
- Random Forest ML model
- Interactive web interface
- ArcGIS integration for professional mapping

---
### Part 2: Live Web Demo (4 min)
#### Step 1: Start the System
bash
python api/flask_app.py

**Open:** http://localhost:5000 (prepare in background before presentation)
#### Step 2: Interactive Predictions
**Demo Script:**
1. **Click Mumbai:**
   > Let me click on Mumbai, a coastal city. The system extracts satellite data from Google Earth Engine...
   > Expected Result: ~97-98% (HIGH)
   > Flood Probability: ~97-98% (HIGH)
   > Elevation: 15 meters (low)
   > Location: 19.0763, 72.8781
   > Mumbai Shows HIGH risk because it's a low-lying coastal area with minimal slope.
2. **Click Bangalore:**
   > Now let me click Bangalore, which is inland and elevated...
   > Expected Result: ~3-5% (LOW)
   > Flood Probability: ~3-5% (LOW)
   > Elevation: 900 meters (high)
   > Location: 12.9721, 77.5958
   > Bangalore Shows LOW risk due to its high elevation—nearly 900 meters above sea level.
3. **Quick Buttons Demo:**
   > I've added quick buttons for major cities. Watch how fast it processes...
   > Click: Delhi → Chennai → Hyderabad (Rapid succession)
   > Notice the pattern: coastal low-elevation cities are HIGH risk, inland elevated cities are LOW risk.

---
### Part 3: Show the Technology (3 min)
#### Open VS Code / Show Files
**Navigate to:** `src/feature_extraction_gee.py`
Say:
> Behind the scenes, the system uses Google Earth Engine's Python API to extract four key features...
**Point out key datasets:**
1. CHIRPS Rainfall Dataset
   # UCSB-CHG/CHIRPS/DAILY
2. ERA5 Land Soil Moisture
   # ECMWF/ERA5_LAND/DAILY_AGGR
3. SRTM Elevation
   # USGS/SRTMGL1_003
4. Slope (derived from elevation)
   # ee.Terrain.slope(elevation)

**Show:** `models/random_forest_flood_model.pkl` (523 KB)
Say:
> The trained Random Forest model is only 523 KB but achieves 95.6% accuracy on test data.

---
### Part 4: ArcGIS Integration (4 min)
#### Step 1: Generate Predictions
bash
python src/generate_arcgis_predictions.py

> While running:
> I can generate predictions for multiple locations at once and export to ArcGIS-compatible formats...
**Show progress bar:**
- 10 Major Indian cities being processed
- Real-time GEE data extraction
- Model predictions
#### Step 2: Show Generated Files
**Navigate to:** `data/outputs/arcgis/`
**List files:**
flood_risk_india_major_cities.geojson
flood_risk_india_major_cities.csv
flood_risk_india_major_cities_layer.json

Say:
> The system exports in four formats: GeoJSON for ArcGIS Online, Shapefile for ArcGIS Pro, CSV for quick import, and a layer definition for automatic symbology.
#### Step 3: Open CSV in Excel (or show content)
**Show table:**


| Location  | Latitude | Longitude | Flood Prob | Risk Level | Elevation | Slope |
|-----------|----------|-----------|------------|------------|-----------|-------|
| Mumbai    | 19.0763  | 72.8781   | 0.9768     | HIGH       | 14.74     | 3.15  |
| Delhi     | 28.1983  | 77.8671   | 0.0368     | LOW        | 216.040   | 3.28  |
| Bangalore | 12.9721  | 77.5958   | 0.041      | LOW        | 897.37    | 3.26  |


**Highlight:**
> See the pattern? High elevation = Low risk. Mumbai at 15m → 98% risk. Bangalore at 900m → 4% risk."

```

```

#### Step 4: (Optional) Import to ArcGIS Online
If you have time and internet:
1: Upload flood_risk_india_major_cities.geojson
2: Show map with color-coded risk levels
3: Zoom to show spatial patterns

**Say:**
> With ArcGIS, we can create professional flood risk maps with proper symbology—green for LOW, yellow for MEDIUM, red for HIGH risk.
---
```

Part 5: Model Performance (2 min)

Show: Training report or create a PowerPoint slide

Key Metrics:

```

Test Accuracy: 95.6%
ROC-AUC Score: 0.998
Precision: 96.0%
F1-Score: 96.0%

```

Feature Importance:

```

1. Slope: 45.78%
2. Elevation: 45.22%
3. Soil Moisture: 16.19%
4. Rainfall: 2.81%

```

Say:

> The model achieves 95.6% accuracy. Interestingly, topography (slope and elevation) accounts for 90% of the prediction power. This makes sense: water flows downhill and accumulates in low-lying areas.

Show confusion matrix (if you have a visual):

```

Predicted:  NO  | YES
Actual YES: 127 | 118

```

Say:

> Out of 250 test samples, we only misclassified 11 cases—that's excellent for a flood prediction model.

■ Visual Aids to Prepare

Slide 1: Title

AI-Driven Flood Prediction System
Using Google Earth Engine, Machine Learning, Flask, and ArcGIS

[Your Name]
[Date] [Department]

Slide 2: Problem Statement

■ Problem:

- India experiences frequent floods during monsoon season
- High loss of life and property; flood deaths occur in India
- Need for early warning systems

■ Solution:

- Satellite-based flood risk prediction
- Machine learning classification
- Interactive web interface
- GIS integration for spatial analysis

Slide 3: System Architecture

[Diagram showing flow:]

```

Location Input → Google Earth Engine → ML Model → Risk Output
Web UI, Satellite Data, Random Forest, ArcGIS Map
(Rainfall, Soil Moisture, Elevation, Slope)

```

Slide 4: Data Sources

■ Satellite Datasets (via Google Earth Engine):

- CHIRPS Daily
 - 7-day cumulative rainfall
 - Resolution: 5.5 km
- ERA5-Land
 - Soil moisture (0-7 cm)
 - Resolution: 9 km
- SRTM DEM
 - Elevation & slope
 - Resolution: 30 m

Slide 5: Model Performance

■ Random Forest Classifier

Training Data: 1250 samples

- 50 locations across India
- Balanced classes (50% flood, 50% non-flood)

Results:

```

Accuracy: 95.6%
ROC-AUC: 0.998
F1-Score: 0.96

```

Feature Importance:

```

1. Slope (45.78%)
2. Elevation (45.22%)
3. Soil Moisture (16.19%)
4. Rainfall (2.81%)

```

Slide 6: Sample Predictions

■ Major Indian Cities (Nov-Dec 2025 data)

City	Elevation	Flood Risk	Probability
Mumbai	15 m	HIGH	97.7%
Chennai	6 m	HIGH	96.8%
Kolkata	91 m	HIGH	94.2%
Bangalore	897 m	LOW	4.5%
Hyderabad	523 m	LOW	2.6%

Pattern: Coastal + Low Elevation = High Risk

Slide 7: Features & Deliverables

■ Deliverables:

- Machine Learning Model
 - Random Forest (95.6% accuracy)
 - 4 features, 100 trees
- Flask REST API
 - JSON responses
- Interactive Web UI
 - Real-time predictions
- ArcGIS Integration
 - GeoJSON export
 - Paperless export
 - Professional mapping

Slide 8: Limitations & Future Work

■ Limitations:

- Data latency (30-day lag due to satellite availability)

```

: Spatial averaging (10 km buffer)
: Historical assessment, Not real-time forecasting
■ Future Enhancements:
: Real-time alerts (email/SMS)
: Historical validation
: Deep learning models (LSTM)
: Additional features (land use, drainage)

### Slide 9: Conclusion
■ Achievements:
End-to-end system from satellite → ML → web → GIS
95.6% prediction accuracy
Seamless satellite data integration
Fully functional web interface
■ Impact:
Academic contribution to flood prediction
Demonstrates full-stack ML + GIS skills
Scalable to other disaster prediction problems

---
## ■ Common Q&A
### Q1: "How is this different from weather forecasts?"
A: "Weather forecasts predict future rainfall. My system assesses flood *susceptibility* based on current weather, environmental conditions—rainfall, soil saturation, and topography. It's complementary to weather forecasts."
### Q2: "Can this predict the exact time a flood will occur?"
A: "No, this is a flood susceptibility assessment, not a real-time warning system. It identifies locations with high flood risk based on historical patterns. For exact timing, you'd need real-time monitoring and hydrological modeling."
### Q3: "Why only 4 features? Why not use more data?"
A: "We achieved 95.6% accuracy with 4 features. More features could lead to overfitting and require more computational resources. Topography alone (slope + elevation) accounts for 90% of the prediction—that's the key insight."
### Q4: "How did you validate the model?"
A: "Used an 88/20 train-test split with 1250 samples. The model was trained on 1000 samples and tested on 250 unseen samples, achieving 95.6% accuracy. I also used cross-validation during training to prevent overfitting."
### Q5: "Can this be used in real disasters?"
A: "As an academic project, it demonstrates the approach. For real deployment you'd need: (1) real-time data systems, and (4) consideration of local drainage infrastructure."
### Q6: "What's the ArcGIS integration used for?"
A: "ArcGIS is the professional standard for spatial analysis. My system exports predictions as GeoJSON and Shapefiles, which can be imported into ArcGIS Online or ArcGIS Pro to create professional flood risk maps for government agencies or NGOs."
### Q7: "How long does a prediction take?"
A: "About 2 seconds per location. Most of the time is spent querying Google Earth Engine for satellite data. The ML prediction itself is instant (milliseconds)."
### Q8: "Can you predict for locations outside India?"
A: "Technically yes, the satellite data is global. However, the model was trained on Indian locations, so predictions for other countries may not be as accurate. You'd need to retrain with data from those regions."

---
## ■ Pre-Presentation Checklist
### Day Before:
- [ ] Test Flask server (ensure it starts without errors)
- [ ] Prepare web browser (all buttons working)
- [ ] Prepare PowerPoint slides (run generate_arcgis_predictions.py)
- [ ] Practice demo flow (time yourself: 10-15 min)
- [ ] Backup: Export web UI screenshots in case of internet issues
### 1 Hour Before:
- [ ] Close unnecessary applications
- [ ] Restart Flask server one more time
- [ ] Open all files/folders you'll show
- [ ] Set laptop to "Presentation Mode" (no sleep)
### Just Before Presenting:
- [ ] Start Flask server in background
- [ ] Open PowerPoint in presentation mode (separate tab)
- [ ] Have VS Code open with key files visible
- [ ] Deep breath! You've built an amazing system! ■

---
## ■ Grading Rubric (What Examiners Look For)
### Technical Depth (30%)
- [ ] Understanding of Random Forest algorithm
- [ ] Knowledge of satellite datasets (CHIRPS, ERA5, SRTM)
- [ ] Model evaluation metrics
### Implementation (30%)
- [ ] Clean project structure
- [ ] Error handling
- [ ] Documentation (README, comments)
### Innovation (20%)
- [ ] GEE integration (automated data extraction)
- [ ] Full-stack approach (ML + web)
- [ ] ArcGIS export (professional visualization)
### Presentation (20%)
- [ ] Clear explanation
- [ ] Handling questions
- [ ] Time management

---
## ■ Key Talking Points to Emphasize
1. ***No manual data downloads*** - GEE API automates everything
2. ***95.6% accuracy with just 4 features*** - Efficiency over complexity
3. ***Topography is 90% of the story*** - key insight from feature importance
4. ***End-to-end system*** - Satellite → ML → web → GIS (not just a model)
5. ***ArcGIS integration*** - makes it professionally usable, not just academic
6. ***Real satellite data*** - not simulated or synthetic datasets
7. ***Scalable approach*** - same method could work for landslides, droughts, etc.

---
## ■ Success Indicators
You'll know your demo went well if:
- [ ] Live predictions work smoothly (no errors)
- [ ] Audience understands the coastal/elevation pattern
- [ ] Questions focus on how to improve (not "why doesn't it work")
- [ ] You finish within time limit
- [ ] You confidently answer at least 3 questions

---
## ■ Screenshot Checklist (For Report)
Take screenshots of:
1. Web UI showing Mumbai HIGH risk prediction
2. GeoJSON export (Bangalore low risk prediction)
3. Success message (Postman, browser)
4. Final output of model training (95.6% accuracy)
5. ArcGIS Online map with imported GeoJSON (if available)
6. Feature importance bar chart (generated)
7. Confusion matrix visualization (if generated)

```

```

**Good luck with your presentation! You've built a professional-grade system. Own it! ■■■**
---
## ■ Demo Script (Copy-Paste for Practice)
v "Good morning/afternoon everyone. Today I'm presenting my project: an AI-driven flood prediction system.
v India faces frequent floods, especially during monsoon season. My system uses satellite data from Google
v Earth Engine, machine learning, and GIS to predict flood risk for any location.
v Let me show you a live demo. I've built an interactive web interface. When I click on Mumbai—a coastal
v city—the system queries Google Earth Engine for real-time satellite data: Rainfall from CHIRPS, soil
v moisture from ERA5, and elevation from SRTM.
v [Click Mumbai]
v As you can see, Mumbai shows 97.7% flood probability—HIGH risk. Why? It's at 15 meters elevation with
v minimal slope. Water accumulates easily.
v Now let's try Bangalore, an inland elevated city.
v [Click Bangalore]
v Only 4.1% risk—LOW. Bangalore sits at 897 meters elevation. Water drains naturally.
v Behind the scenes, I'm using a Random Forest model trained on 1250 samples across India. The model
v achieves 95.6% test accuracy.
v Interestingly, topography—slope and elevation—accounts for 90% of the prediction power. This makes
v sense: water flows downhill.
v The system also exports predictions to ArcGIS for professional mapping. I can generate flood risk maps
v for entire states.
v [Show CSV/GeoJSON files]
v In conclusion, I've built an end-to-end system: from satellite data extraction, to machine learning
v prediction, to interactive web interface, to GIS visualization.
v Thank you. I'm happy to answer questions."
---
**Duration:** ~2-3 minutes intro + 2-3 minutes demo + 2 minutes conclusion = 6-8 minutes (leaves 4-7 min
for Q&A)
---
**Final tip:** Smile, make eye contact, and show enthusiasm. You built something amazing! ■

docs/PROJECT_SUMMARY.md
# ■ Project Summary: AI-Driven Flood Prediction System
## Executive Summary
This academic project successfully implements a complete flood risk assessment system using satellite data
from Google Earth Engine, machine learning, and GIS integration.
---
## ■ Deliverables Completed
### 1. Data Collection & Processing (GEE) for satellite data
- Imported 7 Google Earth Engine datasets: Rainfall (CHIRPS), Soil Moisture (ERA5), Elevation & Slope (SRTM)
- Generated 1250 sample training dataset (50 locations x 25 time windows)
- Exploratory Data Analysis with visualizations
### 2. Machine Learning Model
- Trained Random Forest Classifier
- Achieved 95.6% test accuracy (ROC-AUC: 0.998)
- Model saved as random_forest_flood_model.pkl (523.70 KB)
### 3. Flask REST API
- /api/status endpoint for flood probability predictions
- CORS enabled for web interface
- JSON response format with features and risk level
- Error handling for invalid inputs
### 4. Interactive Web Interface
- Beautiful UI with gradient design
- Click anywhere on interactive map
- Quick location buttons (Mumbai, Delhi, Chennai, Hyderabad)
- Visual risk gauge (LOW/MEDIUM/HIGH)
- Real-time AJAX requests to Flask API
### 5. ArcGIS Integration ■ NEW
- Export to GeoJSON format (ArcGIS Online/Pro)
- Export to shapefile for coordinates
- Layer definition JSON for symbology
- Batch prediction generator for ArcGIS mapping
- Comprehensive import guide ([ARCgis_INTEGRATION.md](ARCgis_INTEGRATION.md))
---
## ■ System Capabilities
### What It Does:
1. **Accepts** any geographic location (latitude/longitude)
2. **Fetches** real-time satellite data from Google Earth Engine:
   - 7-day cumulative rainfall (CHIRPS)
   - Surface soil moisture (ERA5-Land)
   - Terrain elevation (SRTM)
3. **Predicts** flood probability using trained Random Forest model
4. **Classifies** risk level: LOW (<30%), MEDIUM (30-70%), HIGH (>70%)
5. **Exports** results to ArcGIS-compatible formats for mapping
### Use Cases:
- Academic flood risk assessment studies
- Historical flood susceptibility analysis
- Educational demonstration of ML/GIS integration
- Research on satellite-based disaster prediction
---
## ■ Technical Achievements
### Model Performance
Test Accuracy: 95.6%
Precision: 96.0%
Recall: 96.0%
ROC-AUC: 0.998
### Feature Importance Rankings
1. Slope (terrain): 45.78%
2. Elevation: 44.12%
3. Soil Moisture: 6.19%
4. 7-day Rainfall: 2.81%
**Key Insight:** Topography (slope + elevation) is 90% of prediction power! Flat, low-lying areas have
highest flood risk.
---
## ■ ArcGIS Export Examples
### Generated Files (Major Indian Cities)
flood_risk_india_major_cities.geojson : For ArcGIS Online/Pro
flood_risk_india_major_cities.shp : For ArcGIS Desktop
flood_risk_india_major_cities_layer.json : Symbology definition
### Sample Predictions (Nov 30 - Dec 7, 2025 data)
City | Latitude | Longitude | Flood Prob | Risk Level | Elevation | Slope |
Mumbai | 19.076°N | 72.829°E | 97.7% | HIGH | 15.74 m | 3.15°
Delhi | 28.704°N | 77.103°E | 96.8% | HIGH | 16.40 m | 1.56°
Bangalore | 12.935°N | 77.607°E | 4.1% | LOW | 897.03 m | 3.28°
Hyderabad | 17.385°N | 78.487°E | 2.6% | LOW | 522.37 m | 3.05°
**Pattern Observed:** Coastal cities (Mumbai, Chennai) at low elevation → HIGH risk. Inland cities at

```


higher elevation → LOW risk.

■ Data Sources

Google Earth Engine Datasets Used

1. ****CHIRPS Daily (Climate Hazards Center)****
- 7-day cumulative precipitation
- Resolution: 1000m
- Latency: 2-5 days
2. ****ERA5-Land (ECMWF)****
- Volumetric soil moisture (0-7 cm layer)
- Resolution: 9km
- Latency: 5-7 days
3. ****SRTM Digital Elevation Model****
- Terrain elevation
- Resolution: 30m
- Static (no latency)

■ Technology Stack

Layer	Technology	Purpose
Satellite Data	Google Earth Engine	Real-time environmental data
Backend	Python + Random Forest	Flood probability prediction
Frontend UI	HTML/CSS/JavaScript	Interactive interface
Mapping	Leaflet.js	Web-based maps
Data Processing	Geopandas + Shapely	Geospatial integration
Visualization	Matplotlib + Seaborn	EDA charts

■ Project Structure Summary

```
flood-prediction/
├── data/
│   ├── raw/           # Training datasets (60, 375, 1250 samples)
│   └── outputs/arcgis/ # GeoJSON, Shapefiles, CSVs for ArcGIS
├── models/
│   └── random_forest_flood_model.pkl (523.70 KB, 95.6% accuracy)
├── src/
│   ├── main_model.py      # ML training pipeline
│   ├── feature_extraction_gEE.py # GEE feature extraction
│   └── generate_arcgis_predictions.py # Batch predictions
├── api/
│   └── flask_app.py        # REST API server
├── static/ & templates/
│   └── Interactive web UI with Leaflet.js map
└── docs/
    └── ARCGIS_INTEGRATION.md # ArcGIS import guide
```

■ Academic Contributions

- ### ### Novel Aspects:
1. ****Minimal Features****: Only 4 features achieve 95.6% accuracy
 2. ****Topography-Centric****: Slope & elevation dominate (90% importance)
 3. ****Full Stack End-to-end****: From satellite data → ML → web UI → GIS
 4. ****ArcGIS Workflow****: Seamless export for professional cartography
- ### ### Limitations Acknowledged:
1. ****Data Latency****: Using 30-day-old data (satellite availability constraint)
 2. ****Spatial Scale****: 1000m buffer averaging may miss localized effects
 3. ****Historical Data****: Limited availability of older precipitation data
 4. ****Temporal****: 7-day rainfall window may not capture extreme events

■ How to Run (Quick Start)

1. Setup

```
bash
cd ./flood_prediction
python -m venv venv
pip install -r requirements.txt
earthengine authenticate
```

2. Run Web Interface

```
bash
python api/flask_app.py:5000
```

3. Generate ArcGIS Maps

```
bash
python src/generate_arcgis_predictions.py
```

Files saved to data/outputs/arcgis/

4. Import to ArcGIS

- Upload .geojson to ArcGIS Online
- Symbolize: Red = High Risk, Green = Low
- Apply Red/Yellow/Green color scheme

■ Results & Validation

API Test Results (4/4 Passed ■)

```
Health Check:
Hyderabad (17.385, 78.487): 2.59% LOW
Mumbai (19.076, 72.891): 96.84% HIGH
Delhi (28.704, 77.103): 3.58% LOW
```

Confusion Matrix (Test Set)

```
Predicted \ Actual
Actual YES 121 | 118
Accuracy: 95.6%
```

■ Project Alignment with Title

Title: "AI-Driven Flood Prediction using Google Earth Engine, Machine Learning, Flask, and ArcGIS"

Component	Status	Evidence
AI-Driven	**YES**	Random Forest ML (95.6% accuracy)
Google Earth Engine	**YES**	Real-time environmental data
Machine Learning	**YES**	Scikit-learn Random Forest
Flask	**YES**	REST API at localhost:5000
ArcGIS	**YES**	GeoJSON/Shapefile export + import guide

****Verdict**** 100% alignment! All title components delivered. ■

■ Key Takeaways

1. ****Simplicity Works****: Just 4 features achieve excellent accuracy
2. ****Topography King****: Slope & elevation = 90% of prediction power
3. ****GEE is Powerful****: No need to download GBs of satellite imagery
4. ****Full Stack Value****: End-to-end system more impressive than ML alone
5. ****GIS Integration****: ArcGIS export makes results professionally usable

```

echo [3/5] Activating virtual environment...
call venv\Scripts\activate.bat
REM Install dependencies
echo [4/5] Installing dependencies...
pip install -r requirements.txt
REM Authenticate Google Earth Engine
echo [5/5] Google Earth Engine Authentication...
echo IMPORTANT: A browser will open. Please:
echo 1. Go to https://code.earthengine.google.com
echo 2. Click in "Generate Token" to create an account
echo 3. Copy the authorization code
echo 4. Paste it back here
pause
earthengine authenticate
REM Verify setup
echo ==VERIFYING INSTALLATION==
echo ==VERIFYING INSTALLATION==
python -c "import ee; ee.Initialize(project='flood-prediction-0325'); print('✓ Google Earth Engine: CONNECTED!')" 2>nul
echo [WARNING] GEE authentication may have failed
echo Please run: earthengine authenticate
echo ✓ Google Earth Engine: CONNECTED
}

python -c "import flask; print('✓ Flask: INSTALLED!')" 2>nul
echo [ERROR] Flask not installed
echo ✓ Flask: INSTALLED
}

python -c "import sklearn; print('✓ scikit-learn: INSTALLED!')" 2>nul
echo [ERROR] scikit-learn not installed
echo ✓ Scikit-learn: INSTALLED
}

echo ==SETUP COMPLETE==
echo ==SETUP COMPLETE==
echo To run the application:
echo 1. Run: python server.py
echo 2. Open Browser: http://localhost:5000
echo For detailed instructions, see: DEPLOYMENT_GUIDE.md
pause

```

src/__init__.py

```

"""
Flood Prediction System - Source Package
"""

```

src/arcgis_export.py

```

"""
ArcGIS Export Module
Exports Flood Prediction results to formats compatible with ArcGIS
"""

import pandas as pd
import shapely.geometry as geom
from pathlib import Path
import json
import os

def export_to_geojson(predictions_df, output_path):
    """Export predictions to GeoJSON format for ArcGIS"""
    predictions_df = (pd.DataFrame): DataFrame with lat, lon, predictions
    output_path = (str): Output file path

    # Create GeoDataFrame
    gdf = gpd.GeoDataFrame(
        predictions_df,
        geometry=geom.points([geom.Point(lon, lat) for lon, lat in zip(predictions_df['longitude'], predictions_df['latitude'])])
    )
    # CRS = EPSG:4326 # WGS84
    # Export to GeoJSON
    gdf.to_file(output_path, driver='GeoJSON')
    print(f"✓ Exported GeoJSON: {output_path}")
    return gdf

def export_to_csv(predictions_df, output_path):
    """Export predictions to CSV for ArcGIS (with XY coordinates)"""
    predictions_df = (pd.DataFrame): DataFrame with predictions
    output_path = (str): Output file path

    predictions_df.to_csv(output_path, index=False)
    print(f"✓ Exported CSV: {output_path}")

def export_to_shapefile(predictions_df, output_path):
    """Export predictions to Shapefile for ArcGIS Desktop"""
    predictions_df = (pd.DataFrame): DataFrame with predictions
    output_path = (str): Output folder path (without extension)

    # Create GeoDataFrame
    gdf = gpd.GeoDataFrame(
        predictions_df,
        geometry=geom.points([geom.Point(lon, lat) for lon, lat in zip(predictions_df['longitude'], predictions_df['latitude'])])
    )
    # CRS = EPSG:4326
    # Export to Shapefile
    gdf.to_file(output_path, driver='ESRI Shapefile')
    print(f"✓ Exported Shapefile: {output_path}")

def create_arcgis_layer_file(predictions_df, output_path):
    """Create ArcGIS-compatible layer definition JSON"""
    predictions_df = (pd.DataFrame): DataFrame with predictions
    output_path = (str): Output JSON file path

    layer_definition = {
        "name": "Flood Risk",
        "geometryType": "esriGeometryPoint",
        "spatialReference": {
            "wkid": 4326
        },
        "fields": [
            {
                "name": "location",
                "type": "esriFieldTypeString",
                "alias": "Location",
                "length": 100
            },
            {
                "name": "longitude",
                "type": "esriFieldTypeDouble",
                "alias": "Longitude",
                "precision": 15
            },
            {
                "name": "latitude",
                "type": "esriFieldTypeDouble",
                "alias": "Latitude",
                "precision": 15
            },
            {
                "name": "flood_prob",
                "type": "esriFieldTypeDouble",
                "alias": "Flood Probability",
                "precision": 15
            },
            {
                "name": "risk_level",
                "type": "esriFieldTypeString",
                "alias": "Risk Level",
                "length": 10
            },
            {
                "name": "rainfall",
                "type": "esriFieldTypeDouble",
                "alias": "Rainfall (mm)",
                "precision": 15
            },
            {
                "name": "elevation",
                "type": "esriFieldTypeDouble",
                "alias": "Elevation (m)",
                "precision": 15
            },
            {
                "name": "slope",
                "type": "esriFieldTypeDouble",
                "alias": "Slope (deg)",
                "precision": 15
            }
        ],
        "drawingInfo": {
            "renderer": {
                "type": "uniqueValue",
                "uniqueValueInfos": [

```

```

"value": "LOW",
"symbol": "arcsms",
"style": "arcsmscircle",
"color": [74, 222, 128, 255],
"size": 8,
"label": "Low Risk"
},
{
"value": "MEDIUM",
"symbol": "arcsms",
"style": "arcsmscircle",
"color": [251, 191, 36, 255],
"size": 8,
"label": "Medium Risk"
},
{
"value": "HIGH",
"symbol": "arcsms",
"style": "arcsmscircle",
"color": [239, 68, 68, 255],
"size": 8,
"label": "High Risk"
}
]

with open(output_path, 'w') as f:
    print(f'Exported layer definition: {output_path}')

def export_all_formats(predictions_df, base_filename='flood_predictions'):
    """Export predictions to all ArcGIS-compatible formats
    predictions_df (pd.DataFrame): DataFrame with predictions
    base_filename (str): Base name for output files
    """
    output_dir = Path(file).parent.parent / 'data' / 'outputs' / 'arcgis'
    output_dir.mkdir(parents=True, exist_ok=True)
    print(f"EXPORTING FOR ArcGIS")
    # GeoJSON
    geojson_path = Path(output_dir, base_filename).with_suffix('.geojson')
    predictions_df.to_crs('EPSG:4326').to_json(geojson_path)
    # CSV
    csv_path = Path(output_dir, base_filename).with_suffix('.csv')
    predictions_df.to_csv(csv_path)
    # Shapefile
    shp_path = Path(output_dir, base_filename).with_suffix('.shp')
    predictions_df.to_file(shp_path)
    # Layer JSON
    layer_json_path = Path(output_dir, base_filename).with_suffix('.layer.json')
    create_arcpy_layer_json(predictions_df, str(layer_json_path))
    print(f"EXPORT COMPLETE")
    print(f"Files saved to: {output_dir}")
    print(f"ArcGIS Online: Upload {base_filename}.geojson or .csv")
    print(f"ArcGIS Pro: Add {base_filename}.shp or .layer.json")
    return output_dir

if __name__ == '__main__':
    sample_data = pd.DataFrame({
        'location': ['Hyderabad', 'Mumbai', 'Chennai'],
        'longitude': [78.3382, 72.8311, 80.2271],
        'latitude': [17.385, 19.076, 13.082],
        'risk_level': ['LOW', 'MEDIUM', 'HIGH'],
        'rainfall': [0.02, 0.01, 0.36],
        'elevation': [523.84, 14.14, 6.40],
        'slope': [3.07, 3.15, 1.56]
    })
    export_all_formats(sample_data)

```

src/auth.py

```

"""
Authentication Module
User authentication, password hashing, and session management
"""

import hashlib
import secrets
from datetime import datetime, timedelta

def hash_password(password):
    """Hash password using SHA-256 with salt
    password (str): Plain text password
    Returns: Hashed password (salt + hash)
    """
    # Generate random salt
    salt = secrets.token_hex(16)
    # Hash password with salt
    password_hash = hashlib.sha256((salt + password).encode()).hexdigest()
    return f"{salt}${password_hash}"

def verify_password(password, stored_hash):
    """Verify password against stored hash
    password (str): Plain text password to verify
    stored_hash (str): Stored hash (salt$hash format)
    Returns: True if password matches, False otherwise
    """
    try:
        # Split salt and hash
        salt, hash_value = stored_hash.split('$')
        # Hash password with the same salt
        test_hash = hashlib.sha256((salt + password).encode()).hexdigest()
        # Compare test hash
        return test_hash == hash_value
    except:
        return False

def validate_email(email):
    """Validate email format
    email (str): Email address
    Returns: (valid: bool, message: str)
    """
    pattern = r'^[a-zA-Z0-9_%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    if not email:
        return False, 'Email is required'
    if not re.match(pattern, email):
        return False, 'Invalid email format'
    if len(email) > 255:
        return False, 'Email too long (max 255 characters)'
    return True, 'Valid email'

def validate_username(username):
    """Validate username
    username (str): Username
    Returns: (valid: bool, message: str)
    """
    if not username:
        return False, 'Username is required'
    if len(username) < 5:

```

```

return (False, 'Username must be at least 3 characters')
return (False, 'Username too long (max 50 characters)')
# Allow alphanumeric, underscore, hyphen
if not re.match(r'^[a-zA-Z0-9_]{3,50}$', username):
    return (False, 'Username can only contain letters, numbers, underscore, and hyphen')
return (True, 'valid username')

def validate_password(password, retype_password=None):
    """Validate password strength"""
    Args:
        password (str): Password
        Retype password (str, optional): Password confirmation
    Returns:
        Tuple: (valid: bool, message: str)
    """
    if not password:
        return (False, 'Password is required')
    if len(password) < 6:
        return (False, 'Password must be at least 6 characters')
    if len(password) > 128:
        return (False, 'Password too long (max 128 characters)')
    # Check if password is match (if retype provided)
    if retype_password is not None and password != retype_password:
        return (False, 'Passwords do not match')
    # Add more strict checks
    if not any(c.isupper() for c in password):
        return (False, 'Password must contain at least one uppercase letter')
    if not any(c.isdigit() for c in password):
        return (False, 'Password must contain at least one number')
    return (True, 'Valid password')

def generate_session_token():
    """Generate secure session token"""
    # Random session token
    return secrets.token_urlsafe(32)

def validate_registration(username, email, password, retype_password):
    """Validate registration form data"""
    Args:
        username (str): Username
        email (str): Email
        password (str): Password
        Retype password (str): Password confirmation
    Returns:
        Tuple: (valid: bool, errors: dict)
    """
    errors = {}
    # Validate username
    valid, msg = validate_username(username)
    errors['username'] = msg
    # Validate email
    valid, msg = validate_email(email)
    errors['email'] = msg
    # Validate password
    valid, msg = validate_password(password, retype_password)
    errors['password'] = msg
    return (len(errors) == 0, errors)

def validate_login(identifier, password):
    """Validate login form data"""
    Args:
        identifier (str): Username or email
        password (str): Password
    Returns:
        Tuple: (valid: bool, errors: dict)
    """
    errors = {}
    if not identifier:
        errors['identifier'] = 'Username or email is required'
    if not password:
        errors['password'] = 'Password is required'
    return (len(errors) == 0, errors)

def is_email(identifier):
    """Check if identifier is an email"""
    Args:
        identifier (str): Username or email
    Returns:
        bool: True if email, False if username
    """
    return '@' in identifier

if __name__ == "__main__":
    # Test password hashing
    password = "password123"
    hashed_password = hash_password(password)
    print(f"Original: {password}")
    print(f"Hashed: {hashed_password}")
    print(f"Verify password: {verify_password(password, hashed_password)}")
    # Test validation
    print("\nValidation Tests:")
    print(validate_username("user123"))
    print(validate_email("user@example.com"))
    print(validate_password("short", "short"))

```

src/database.py

```

"""
Database Module for User Management
SQLite database for storing user credentials and sessions

import sqlite3
from pathlib import Path
from datetime import datetime

# Database path
DB_DIR = Path(__file__).parent.parent / 'data'
DB_PATH = DB_DIR / 'users.db'

def init_database():
    """Initialize database and create users table if not exists"""
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    # Create users table
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            email TEXT UNIQUE NOT NULL,
            password TEXT NOT NULL,
            last_login TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        )
    """)
    # Create sessions table
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS sessions (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            session_token TEXT UNIQUE NOT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (user_id) REFERENCES users(id)
        )
    """)
    # Create predictions_history table (optional, for tracking user predictions)
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS predictions_history (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL,
            longitude REAL NOT NULL,

```

```

flood_probability REAL, NOT NULL,
PRIMARY KEY (id, username, email, phone, password_hash, sub_role, district, city, area),
FOREIGN KEY (user_id) REFERENCES users(id)
),
)
conn.commit()
print(f"> Database initialized at {DB_PATH}")

def get_connection():
    """Get database connection"""
    return sqlite3.connect(DB_PATH)

def create_user(
    username,
    email,
    phone,
    password_hash,
    sub_role,
    district,
    city,
    area
):
    """Create a new user"""
    try:
        conn = get_connection()
        cursor = conn.cursor()
        # Username check
        cursor.execute(
            'SELECT id FROM users WHERE username = ?',
            (username,)
        )
        if cursor.fetchone():
            conn.close()
            return (False, 'Username already exists', None)
        # Email check
        cursor.execute(
            'SELECT id FROM users WHERE email = ?',
            (email,)
        )
        if cursor.fetchone():
            conn.close()
            return (False, 'Email already exists', None)
        # Insert new user
        cursor.execute(
            'INSERT INTO users'
            '(username,'
            'email,'
            'phone,'
            'password_hash,'
            'sub_role,'
            'district,'
            'city,'
            'area'
            ') VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)'
        )
        user_id = cursor.lastrowid
        conn.commit()
        conn.close()
        return (
            True,
            'User created successfully',
            user_id
        )
    except Exception as e:
        return (
            False,
            'Database error: {str(e)}',
            None
        )

def get_user_by_username(username):
    """Get user by username"""
    Args:
        username (str): Username
    Returns:
        dict or None: User data or None if not found
    """
    try:
        conn = get_connection()
        conn.row_factory = sqlite3.Row
        cursor = conn.cursor()
        cursor.execute('SELECT * FROM users WHERE username = ?', (username,))
        row = cursor.fetchone()
        conn.close()
        if row:
            return dict(row)
        return None
    except Exception as e:
        print(f"Error fetching user: {e}")
        return None

def get_user_by_email(email):
    """Get user by email"""
    Args:
        email (str): Email address
    Returns:
        dict or None: User data or None if not found
    """
    try:
        conn = get_connection()
        conn.row_factory = sqlite3.Row
        cursor = conn.cursor()
        cursor.execute('SELECT * FROM users WHERE email = ?', (email,))
        row = cursor.fetchone()
        conn.close()
        if row:
            return dict(row)
        return None
    except Exception as e:
        print(f"Error fetching user: {e}")
        return None

def get_user_by_id(user_id):
    """Get user by ID"""
    Args:
        user_id (int): User ID
    Returns:
        dict or None: User data or None if not found
    """
    try:
        conn = get_connection()
        conn.row_factory = sqlite3.Row
        cursor = conn.cursor()
        cursor.execute('SELECT * FROM users WHERE id = ?', (user_id,))
        row = cursor.fetchone()
        conn.close()
        if row:
            return dict(row)
        return None
    except Exception as e:
        print(f"Error fetching user: {e}")
        return None

```



```
border-radius: 10px 0 0 0;
}content {
background-color: #f9fafb;
padding: 10px solid #e5e7eb;
border-top: none;
}alert-box {
background-color: white;
border-left: 5px solid {color};
padding: 20px 0;
border-radius: 50%;
box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}risk-badge {
display: inline-block;
background-color: {color};
padding: 6px 16px;
border-radius: 20px;
font-weight: bold;
font-size: 18px;
}feature-grid {
display: grid; columns: 1fr 1fr;
margin-top: 20px;
}feature-item {
background-color: white;
border-radius: 8px;
border: 1px solid #e5e7eb;
}feature-label {
font-size: 12px;
text-transform: uppercase;
}feature-value {
font-size: 20px;
color: #4b545d;
margin-top: 5px;
}footer {
text-align: center;
padding: 20px;
color: #6b7280;
border-top: 1px solid #e5e7eb;
border-radius: 0 10px 10px;
background-color: white;
}button {
display: inline-block;
background-color: #6b7280;
color: white;
padding: 12px 30px;
border: none;
border-radius: 5px;
margin-top: 20px;
}
</style>
</script>
<!-- Main Content -->


# Flood Risk Alert



HIGH FLOOD RISK DETECTED AT YOUR MONITORED LOCATION



Risk Level: HIGH  

Risk Probability: 85%  

Risk Description: Significant flooding expected due to heavy rainfall.



### Location Details



Latitude: 41.88°N  

Longitude: -87.63°W  

Date & Time: 2023-10-27 14:30:00 UTC



### Environmental Conditions



| Parameter        | Value  |
|------------------|--------|
| Rainfall (7-day) | 120 mm |
| Soil Moisture    | High   |
| Elevation        | 150 m  |
| Slope            | 5%     |



### Recommended Actions



- Monitor weather updates closely.
- Prepare emergency supplies and evacuation kit.
- Avoid low-lying areas and flood-prone zones.
- Keep emergency contacts readily available.



View Full Details



Powered by Automated alert from the Flood Prediction System.  

    © 2023 Flood Prediction System. All rights reserved.


</body>
</html>
# Attach HTML body
def attach_html_email(smtp_config: dict, recipient_email: str) -> bool:
    """Attach HTML body to email"""
    try:
        # Send test email configuration by sending a test email
        smtp_config.setdefault('server', 'smtp.gmail.com')
        smtp_config.setdefault('port', 587)
        smtp_config.setdefault('username', 'test@example.com')
        smtp_config.setdefault('password', 'your_password')
        # Send message
        msg = MIMEMultipart()
        msg['Subject'] = f'Flood Alert Test Email'
        msg['From'] = smtp_config.get('from_name', 'Flood Alert')
        msg['To'] = recipient_email
        msg.attach(MIMEText(html_body, 'html'))
        s = smtplib.SMTP(smtp_config['server'], smtp_config['port'])
        s.starttls()
        s.login(smtp_config['username'], smtp_config['password'])
        s.send_message(msg)
        print(f"Test email sent successfully to {recipient_email}")
        return True
    except smtplib.SMTPAuthenticationError:
        print("Email authentication failed. Check SMTP username/password.")
    except smtplib.SMTPLibException as e:
        print(f"SMTP error: {str(e)}")
    except Exception as e:
        print(f"Failed to send email: {str(e)}")
    return False
def test_email_configuration(smtp_config: dict, recipient_email: str) -> bool:
    """Test email configuration by sending a test email"""
    smtp_config.setdefault('server', 'smtp.gmail.com')
    smtp_config.setdefault('port', 587)
    smtp_config.setdefault('username', 'test@example.com')
    smtp_config.setdefault('password', 'your_password')
    # Send message
    msg = MIMEMultipart()
    msg['Subject'] = f'Flood Alert Test Email'
    msg['From'] = smtp_config.get('from_name', 'Flood Alert')
    msg['To'] = recipient_email
    msg.attach(MIMEText(html_body, 'html'))
    s = smtplib.SMTP(smtp_config['server'], smtp_config['port'])
    s.starttls()
    s.login(smtp_config['username'], smtp_config['password'])
    s.send_message(msg)
    print(f"Test email sent successfully to {recipient_email}")
    return True
except smtplib.SMTPAuthenticationError:
    print("Email authentication failed. Check SMTP username/password.")
except smtplib.SMTPLibException as e:
    print(f"SMTP error: {str(e)}")
except Exception as e:
    print(f"Failed to send email: {str(e)}")
return False
def test_email_configuration(smtp_config: dict, recipient_email: str) -> bool:
    """Test email configuration by sending a test email"""
    smtp_config.setdefault('server', 'smtp.gmail.com')
    smtp_config.setdefault('port', 587)
    smtp_config.setdefault('username', 'test@example.com')
    smtp_config.setdefault('password', 'your_password')
    # Send message
    msg = MIMEMultipart()
    msg['Subject'] = f'Flood Alert Test Email'
    msg['From'] = smtp_config.get('from_name', 'Flood Alert')
    msg['To'] = recipient_email
    msg.attach(MIMEText(html_body, 'html'))
    s = smtplib.SMTP(smtp_config['server'], smtp_config['port'])
    s.starttls()
    s.login(smtp_config['username'], smtp_config['password'])
    s.send_message(msg)
    print(f"Test email sent successfully to {recipient_email}")
    return True
except smtplib.SMTPAuthenticationError:
    print("Email authentication failed. Check SMTP username/password.")
except smtplib.SMTPLibException as e:
    print(f"SMTP error: {str(e)}")
except Exception as e:
    print(f"Failed to send email: {str(e)}")
return False
def test_email_configuration(smtp_config: dict, recipient_email: str) -> bool:
    """Test email configuration by sending a test email"""
    smtp_config.setdefault('server', 'smtp.gmail.com')
    smtp_config.setdefault('port', 587)
    smtp_config.setdefault('username', 'test@example.com')
    smtp_config.setdefault('password', 'your_password')
    # Send message
    msg = MIMEMultipart()
    msg['Subject'] = f'Flood Alert Test Email'
    msg['From'] = smtp_config.get('from_name', 'Flood Alert')
    msg['To'] = recipient_email
    msg.attach(MIMEText(html_body, 'html'))
    s = smtplib.SMTP(smtp_config['server'], smtp_config['port'])
    s.starttls()
    s.login(smtp_config['username'], smtp_config['password'])
    s.send_message(msg)
    print(f"Test email sent successfully to {recipient_email}")
    return True
except smtplib.SMTPAuthenticationError:
    print("Email authentication failed. Check SMTP username/password.")
except smtplib.SMTPLibException as e:
    print(f"SMTP error: {str(e)}")
except Exception as e:
    print(f"Failed to send email: {str(e)}")
return False
def test_email_configuration(smtp_config: dict, recipient_email: str) -> bool:
    """Test email configuration by sending a test email"""
    smtp_config.setdefault('server', 'smtp.gmail.com')
    smtp_config.setdefault('port', 587)
    smtp_config.setdefault('username', 'test@example.com')
    smtp_config.setdefault('password', 'your_password')
    # Send message
    msg = MIMEMultipart()
    msg['Subject'] = f'Flood Alert Test Email'
    msg['From'] = smtp_config.get('from_name', 'Flood Alert')
    msg['To'] = recipient_email
    msg.attach(MIMEText(html_body, 'html'))
    s = smtplib.SMTP(smtp_config['server'], smtp_config['port'])
    s.starttls()
    s.login(smtp_config['username'], smtp_config['password'])
    s.send_message(msg)
    print(f"Test email sent successfully to {recipient_email}")
    return True
except smtplib.SMTPAuthenticationError:
    print("Email authentication failed. Check SMTP username/password.")
except smtplib.SMTPLibException as e:
    print(f"SMTP error: {str(e)}")
except Exception as e:
    print(f"Failed to send email: {str(e)}")
return False
def test_email_configuration(smtp_config: dict, recipient_email: str) -> bool:
    """Test email configuration by sending a test email"""
    smtp_config.setdefault('server', 'smtp.gmail.com')
    smtp_config.setdefault('port', 587)
    smtp_config.setdefault('username', 'test@example.com')
    smtp_config.setdefault('password', 'your_password')
    # Send message
    msg = MIMEMultipart()
    msg['Subject'] = f'Flood Alert Test Email'
    msg['From'] = smtp_config.get('from_name', 'Flood Alert')
    msg['To'] = recipient_email
    msg.attach(MIMEText(html_body, 'html'))
    s = smtplib.SMTP(smtp_config['server'], smtp_config['port'])
    s.starttls()
    s.login(smtp_config['username'], smtp_config['password'])
    s.send_message(msg)
    print(f"Test email sent successfully to {recipient_email}")
    return True
except smtplib.SMTPAuthenticationError:
    print("Email authentication failed. Check SMTP username/password.")
except smtplib.SMTPLibException as e:
    print(f"SMTP error: {str(e)}")
except Exception as e:
    print(f"Failed to send email: {str(e)}")
return False
def test_email_configuration(smtp_config: dict, recipient_email: str) -> bool:
    """Test email configuration by sending a test email"""
    smtp_config.setdefault('server', 'smtp.gmail.com')
    smtp_config.setdefault('port', 587)
    smtp_config.setdefault('username', 'test@example.com')
    smtp_config.setdefault('password', 'your_password')
    # Send message
    msg = MIMEMultipart()
    msg['Subject'] = f'Flood Alert Test Email'
    msg['From'] = smtp_config.get('from_name', 'Flood Alert')
    msg['To'] = recipient_email
    msg.attach(MIMEText(html_body, 'html'))
    s = smtplib.SMTP(smtp_config['server'], smtp_config['port'])
    s.starttls()
    s.login(smtp_config['username'], smtp_config['password'])
    s.send_message(msg)
    print(f"Test email sent successfully to {recipient_email}")
    return True
except smtplib.SMTPAuthenticationError:
    print("Email authentication failed. Check SMTP username/password.")
except smtplib.SMTPLibException as e:
    print(f"SMTP error: {str(e)}")
except Exception as e:
    print(f"Failed to send email: {str(e)}")
return False
def test_email_configuration(smtp_config: dict, recipient_email: str) -> bool:
    """Test email configuration by sending a test email"""
    smtp_config.setdefault('server', 'smtp.gmail.com')
    smtp_config.setdefault('port', 587)
    smtp_config.setdefault('username', 'test@example.com')
    smtp_config.setdefault('password', 'your_password')
    # Send message
    msg = MIMEMultipart()
    msg['Subject'] = f'Flood Alert Test Email'
    msg['From'] = smtp_config.get('from_name', 'Flood Alert')
    msg['To'] = recipient_email
    msg.attach(MIMEText(html_body, 'html'))
    s = smtplib.SMTP(smtp_config['server'], smtp_config['port'])
    s.starttls()
    s.login(smtp_config['username'], smtp_config['password'])
    s.send_message(msg)
    print(f"Test email sent successfully to {recipient_email}")
    return True
except smtplib.SMTPAuthenticationError:
    print("Email authentication failed. Check SMTP username/password.")
except smtplib.SMTPLibException as e:
    print(f"SMTP error: {str(e)}")
except Exception as e:
    print(f"Failed to send email: {str(e)}")
return False
def test_email_configuration(smtp_config: dict, recipient_email: str) -> bool:
    """Test email configuration by sending a test email"""
    smtp_config.setdefault('server', 'smtp.gmail.com')
    smtp_config.setdefault('port', 587)
    smtp_config.setdefault('username', 'test@example.com')
    smtp_config.setdefault('password', 'your_password')
    # Send message
    msg = MIMEMultipart()
    msg['Subject'] = f'Flood Alert Test Email'
    msg['From'] = smtp_config.get('from_name', 'Flood Alert')
    msg['To'] = recipient_email
    msg.attach(MIMEText(html_body, 'html'))
    s = smtplib.SMTP(smtp_config['server'], smtp_config['port'])
    s.starttls()
    s.login(smtp_config['username'], smtp_config['password'])
    s.send_message(msg)
    print(f"Test email sent successfully to {recipient_email}")
    return True
except smtplib.SMTPAuthenticationError:
    print("Email authentication failed. Check SMTP username/password.")
except smtplib.SMTPLibException as e:
    print(f"SMTP error: {str(e)}")
except Exception as e:
    print(f"Failed to send email: {str(e)}")
return False
def test_email_configuration(smtp_config: dict, recipient_email: str) -> bool:
    """Test email configuration by sending a test email"""
    smtp_config.setdefault('server', 'smtp.gmail.com')
    smtp_config.setdefault('port', 587)
    smtp_config.setdefault('username', 'test@example.com')
    smtp_config.setdefault('password', 'your_password')
    # Send message
    msg = MIMEMultipart()
    msg['Subject'] = f'Flood Alert Test Email'
    msg['From'] = smtp_config.get('from_name', 'Flood Alert')
    msg['To'] = recipient_email
    msg.attach(MIMEText(html_body, 'html'))
    s = smtplib.SMTP(smtp_config['server'], smtp_config['port'])
    s.starttls()
    s.login(smtp_config['username'], smtp_config['password'])
    s.send_message(msg)
    print(f"Test email sent successfully to {recipient_email}")
    return True
except smtplib.SMTPAuthenticationError:
    print("Email authentication failed. Check SMTP username/password.")
except smtplib.SMTPLibException as e:
    print(f"SMTP error: {str(e)}")
except Exception as e:
    print(f"Failed to send email: {str(e)}")
return False
def test_email_configuration(smtp_config: dict, recipient_email: str) -> bool:
    """Test email configuration by sending a test email"""
    smtp_config.setdefault('server', 'smtp.gmail.com')
    smtp_config.setdefault('port', 587)
    smtp_config.setdefault('username', 'test@example.com')
    smtp_config.setdefault('password', 'your_password')
    # Send message
    msg = MIMEMultipart()
    msg['Subject'] = f'Flood Alert Test Email'
    msg['From'] = smtp_config.get('from_name', 'Flood Alert')
    msg['To'] = recipient_email
    msg.attach(MIMEText(html_body, 'html'))
    s = smtplib.SMTP(smtp_config['server'], smtp_config['port'])
    s.starttls()
    s.login(smtp_config['username'], smtp_config['password'])
    s.send_message(msg)
    print(f"Test email sent successfully to {recipient_email}")
    return True
except smtplib.SMTPAuthenticationError:
    print("Email authentication failed. Check SMTP username/password.")
except smtplib.SMTPLibException
```



```
except Exception as e:
    print(f"Email test failed: {str(e)}")
    return False
```

src/feature_extraction_gee.py

```
"""
Google Earth Engine Feature Extraction Module
Extracts Satellite-derived features for Flood prediction
"""
```

```
import ee
from datetime import datetime, timedelta
```

```
def initialize_gee(project=None):
```

```
    """Initialize Google Earth Engine
    Must be authenticated first using: earthengine authenticate
    Project (str): GEE Cloud Project ID (e.g., 'ee-yourproject')
    """
```

```
    if project:
        ee.Initialize(project=project)
    else:
        # Try to initialize with default project
        ee.Initialize()
        print("Google Earth Engine initialized successfully")
    except Exception as e:
        print(f"GEE initialization failed: {e}")
        print(f"Go to: {https://code.earthengine.google.com/}")
        print(f"2. Check your project name in the top left (e.g. flood-prediction-0325)")
        print(f"3. Initialize with: ee.Initialize(project='your-project-name')")
    return False
```

```
def extract_features_for_location(lat, lon, start_date, end_date, buffer_m=10000):
```

```
    """Extract all four features for a given location and time window
    Args:
        lat (float): Latitude
        lon (float): Longitude
        start_date (str): Start date in 'YYYY-MM-DD' format
        end_date (str): End date in 'YYYY-MM-DD' format
        buffer_m (int): Buffer radius in meters
    Returns:
        dict: Dictionary containing all features
    """
```

```
    # Define region
    region = ee.Geometry.Point([lon, lat]).buffer(buffer_m)
    # 1. RAINFALL
    rainfall_collection = ee.ImageCollection('UCSB-CHG/CHIRPS/DAILY') \
        .filterBounds(region) \
        .filterDate(start_date, end_date)
    # Sum the precipitation over the time period
    rainfall = rainfall_collection.select('precipitation').sum()
    # Reduce to get rainfall value
    reducer = ee.Reducer.mean()
    geom = region
    maxPixels = 1e9
    getinfo = rainfall.reduceRegion(
        reducer=reducer,
        geometry=geom,
        maxPixels=maxPixels
    )
    # The band name after sum() is still 'precipitation'
    if no data: return None
    if rainfall_value is None:
        return None
    # 2. SOIL MOISTURE
    soil_moisture_collection = ee.ImageCollection('ECMWF/ERA5_LAND/DAILY_AGGR') \
        .filterDate(start_date, end_date) \
        .filterBounds(region) \
        .select('volumetric_soil_water_layer_1') \
        .mean()
    soil_dict = soil_moisture.reduceRegion(
        reducer=ee.Reducer.mean(),
        geometry=region,
        maxPixels=1e9
    )
    getinfo = soil_dict.get('volumetric_soil_water_layer_1')
    if soil_value is None:
        return None
    # 3. ELEVATION
    elevation = ee.Image('USGS/SRTMGL1_003').select('elevation')
    reducer = ee.Reducer.mean()
    geom = region
    maxPixels = 1e9
    getinfo = elevation.reduceRegion(
        reducer=reducer,
        geometry=geom,
        maxPixels=maxPixels
    )
    if elev_value is None:
        return None
    # 4. SLOPE
    slope = ee.Terrain.slope(elevation)
    reducer = ee.Reducer.mean()
    geom = region
    maxPixels = 1e9
    getinfo = slope.reduceRegion(
        reducer=reducer,
        geometry=geom,
        maxPixels=maxPixels
    )
    if slope_value is None:
        return None
    # Feature dict
    features = {
        'rainfall_7d_mm': rainfall_value,
        'soil_moisture': soil_value,
        'elevation_m': elev_value,
        'slope_deg': slope_value
    }
    return features
```

```
def get_safe_date_range(days_back=30, window_days=7):
```

```
    """Generate a safe historical date range
    (Avoids using today due to satellite data latency)
    Args:
        days_back (int): How many days back from today
        window_days (int): Length of analysis window
    Returns:
        tuple: (start_date, end_date) as strings
    """
```

```
    start = datetime.now() - timedelta(days=days_back)
    end = datetime.now() - timedelta(days=window_days)
    return start.strftime('%Y-%m-%d'), end.strftime('%Y-%m-%d')
```

```
if __name__ == "__main__":
```

```
    # Test GEE connection with project
    initialize_gee(project='your-project-name') # Update if needed
    print("GEE module ready for feature extraction")
    else:
        print("\nGEE setup required")
```

src/generate_arcgis_predictions.py

```
"""
Batch Prediction Generator for ArcGIS Mapping
Generates systematic grid predictions across specified region
"""
```

```
import sys
import os
import Path(__file__).parent.parent))
```

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
from src.feature_extraction import initialize_gee, extract_features_for_location
from src.config import MODEL_PATH, GEE_PROJECT
import tqdm
import time
```

```
def generate_grid_locations(min_lat, max_lat, min_lon, max_lon, grid_spacing=0.1):
```

```

Generate grid of locations for prediction
min_lat, max_lat: Latitude bounds
grid_spacing: Grid spacing in degrees (0.1° ≈ 11km)
Returns: List of (lat, lon) tuples
lats = np.arange(min_lat, max_lat, grid_spacing)
lons = np.arange(min_lon, max_lon, grid_spacing)
grid = np.meshgrid(lats, lons)
return grid

def predict_grid_batch(grid_locations, model, region_name='India'):
    Generate predictions for all grid locations
    grid_locations: List of (lat, lon) tuples
    model: Trained ML Model
    region_name: Name for the region
    Returns: DataFrame with predictions
    results = []
    start_date = datetime.now() - timedelta(days=30)
    end_date = datetime.now()
    start_str = start_date.strftime('%Y-%m-%d')
    end_str = end_date.strftime('%Y-%m-%d')
    print(f"Data Window: {start_str} to {end_str}")
    for i, (lat, lon) in enumerate(tqdm(grid_locations)):
        # Extract features from GEE
        features = extract_features_for_location(lat, lon, start_str, end_str)
        if features is None:
            continue
        # Prepare features for prediction
        feature_array = np.array([
            features['rainfall_7d_mm'],
            features['soil_moisture'],
            features['slope_deg']
        ])
        # Predict
        prediction = model.predict(feature_array)[0]
        probability = model.predict_proba(feature_array)[0][1]
        # Determine risk level
        if probability < 0.3:
            risk = 'LOW'
        elif probability < 0.7:
            risk = 'MEDIUM'
        else:
            risk = 'HIGH'
        results.append({
            'location': f'Grid_{i+1}',
            'lat': lat,
            'lon': lon,
            'risk_level': risk,
            'flood_prob': round(probability, 3),
            'rainfall': round(features['rainfall_7d_mm'], 2),
            'moist': round(features['soil_moisture'], 2),
            'elevation': round(features['elevation_m'], 2),
            'slope': round(features['slope_deg'], 2)
        })
        # Rate limiting to avoid GEE quota issues
        time.sleep(2)
    except Exception as e:
        print(f"Error at ({lat:.2f}, {lon:.2f}): {e}")
    continue
    df = pd.DataFrame(results)
    return df # Successfully processed {len(df)} locations")

def generate_predictions_for_region(region_config):
    Generate predictions for predefined region
    region_config (dict): Configuration with bounds and name
    Print("\n" + "="*60)
    print(f"FLOOD RISK ASSESSMENT: {region_config['name']}")
    # Initialize GEE
    initialize_gee(GEE_PROJECT)
    # Load model
    model = load_model(MODEL_PATH)
    # Generate grid
    grid = generate_grid_locations(
        region_config['max_lat'],
        region_config['min_lat'],
        region_config['max_lon'],
        region_config['min_lon'],
        region_config['grid_spacing', 0.1)
    # Generate predictions
    predictions_df = predict_grid_batch(grid, model, region_config['name'])
    # Export predictions to format
    output_dir = export_all_formats(predictions_df, f"flood_risk_{region_config['name'].lower().replace(' ', '_')}")
    # Print statistics
    print("\n" + "="*60)
    print(f"RISK STATISTICS")
    total_count = len(predictions_df)
    risk_counts = predictions_df['risk_level'].value_counts()
    total_risk = {'HIGH': 0, 'MEDIUM': 0, 'LOW': 0}
    for risk, count in risk_counts.items():
        total_risk[risk] = count
    print(f"Total counts: HIGH={total_risk['HIGH']}, MEDIUM={total_risk['MEDIUM']}, LOW={total_risk['LOW']}")
    print(f"Total locations: {total_count} (Risk: {total_risk['HIGH']/total_count*100:.1f}%)")
    print(f"Avg flood probability: {predictions_df['flood_prob'].mean():.2%}")
    return predictions_df

# Predefined regions
regions = {
    'telangana': {
        'name': 'Telangana',
        'max_lat': 18.0,
        'min_lat': 17.3,
        'max_lon': 78.3,
        'min_lon': 77.3,
        'grid_spacing': 0.2 # ~22km
    },
    'maharashtra': {
        'name': 'Maharashtra',
        'max_lat': 20.0,
        'min_lat': 19.0,
        'max_lon': 75.0,
        'min_lon': 73.0,
        'grid_spacing': 0.2
    },
    'tamil_nadu': {
        'name': 'Tamil Nadu',
        'max_lat': 11.5,
        'min_lat': 10.5,
        'max_lon': 78.5,
        'min_lon': 77.5,
        'grid_spacing': 0.2
    },
    'india_major_cities': {
        'name': 'India Major Cities',
        'cities': [
            ('Mumbai', 19.076, 72.8777),
            ('Delhi', 28.7041, 77.1025),
            ('Bangalore', 12.9716, 77.5946),
            ('Chennai', 13.0839, 80.2161),
            ('Kolkata', 22.5726, 88.3639),
            ('Hyderabad', 18.5204, 78.1814),
            ('Ahmedabad', 23.0225, 72.5714),
            ('Lucknow', 26.8467, 80.9462)
        ]
    }
}

def generate_predictions_for_cities(cities_list):
    Generate predictions for specific cities

```

```

Args:
cities_list: List of (name, lat, lon) tuples
print("\n" + "="*60)
print("\n FLOOD RISK ASSESSMENT: Major Cities")
print("="*60)
# Initialize GEE
# Load model
model = joblib.load(MODEL_PATH)
date = datetime.now() - timedelta(days=30)
end_date = date + timedelta(days=30)
start_str = end_date.strftime('%Y-%m-%d')
end_str = date.strftime('%Y-%m-%d')
print("\n Data window: {start_str} to {end_str}")
for name, lat, lon in tqdm(cities_list):
    features = extract_features_for_location(lat, lon, start_str, end_str)
    if features is None:
        continue
    feature_array = np.array([[
        features['rainfall_7d_mm'],
        features['soil_moisture'],
        features['elevation'],
        features['slope_deg']
    ]])
    prediction = model.predict(feature_array)[0]
    probability = model.predict_proba(feature_array)[0][1]
    if probability < 0.3:
        risk = LOW
    elif probability < 0.7:
        risk = MEDIUM
    else:
        risk = HIGH
    results.append({
        'location': name,
        'longitude': lon,
        'latitude': lat,
        'flood_prob': round(probability, 3),
        'rainfall_7d': round(features['rainfall_7d_mm'], 2),
        'soil_moist': round(features['soil_moisture'], 2),
        'elevation': round(features['elevation_m'], 2),
        'slope': round(features['slope_deg'], 2)
    })
except Exception as e:
    print(f"Error for {name}: {e}")
# Create DataFrame
df = pd.DataFrame(results)
if len(df) > 0:
    export_df = df.format(df, 'flood_risk_india_major_cities')
    return df

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description='Generate flood predictions for ArcGIS')
    parser.add_argument('--region', type=str, choices=list(REGIONS.keys()),
                        help='Predefined region to process')
    parser.add_argument('--regions', type=str, help='store_true',
                        help='Predefined regions')
    args = parser.parse_args()
    # Process all regions
    if region_key == 'india_major_cities', 'telangana':
        generate_predictions_for_cities(REGIONS[region_key]['cities'])
        generate_predictions_for_region(REGIONS[region_key])
    elif args.regions:
        # Process specific region
        generate_predictions_for_cities(REGIONS[args.region]['cities'])
        generate_predictions_for_region(REGIONS[args.region])
    # Default: Major cities
    print("No region specified. Running for major cities.")
    generate_predictions_for_cities(REGIONS['india_major_cities']['cities'])

```

src/model_training.py

```

"""
Machine Learning Model Training Module
Trains Random Forest Classifier for flood prediction
"""
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, roc_auc_score
from pathlib import Path

def train_random_forest(X, y, n_estimators=100, random_state=42):
    """
    Train Random Forest classifier
    Args:
    X (np.array): Feature matrix
    y (np.array): Target labels
    n_estimators (int): Number of trees
    random_state (int): Random seed
    Returns:
    RandomForestClassifier: Trained model
    """
    print(f"\n Training Random Forest with {n_estimators} trees...")
    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=10,
        min_samples_split=5,
        min_samples_leaf=1,
        random_state=random_state,
    )
    model.fit(X, y)
    return model

def evaluate_model(model, X_test, y_test):
    """
    Evaluate model performance
    Args:
    model: Trained model
    X_test: Test features
    y_test: Test labels
    Returns:
    dict: Evaluation metrics
    """
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]
    accuracy = accuracy_score(y_test, y_pred)
    roc_auc = roc_auc_score(y_test, y_proba)
    print("\n MODEL EVALUATION")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"ROC AUC: {roc_auc:.4f}")
    print("\n Confusion Matrix")
    print(classification_report(y_test, y_pred))
    print(classification_report(y_test, y_pred, target_names=['No Flood', 'Flood']))
    print("\n Metrics")
    print(f"Accuracy: {accuracy},")
    print(f"ROC AUC: {roc_auc},")
    print(f"y_pred: {y_pred}")
    print(f"y_proba: {y_proba}")

def get_feature_importance(model, feature_names):
    """
    Get and display feature importance
    Args:
    model: Trained model
    feature_names (list): Names of features
    Returns:
    pd.DataFrame: Feature importance dataframe
    """
    importance_df = pd.DataFrame({
        feature: feature_names,

```

```

importance': model.feature_importances_
})
sort_values('importance', ascending=False)
print("\nFEATURE IMPORTANCE")
print("\n")
return importance_df.to_string(index=False))

def save_model(model, filepath):
    Save trained model to disk
    Args:
    model: Trained model
    filepath (str): Path to save model
    joblib.dump(model, filepath)
    print(f"\nModel saved to: {filepath}")

def load_model(filepath):
    Load trained model from disk
    filepath (str): Path to model file
    Returns:
    Trained model
    model = joblib.load(filepath)
    print(f"\nModel loaded from: {filepath}")
    return model

if __name__ == "__main__":
    print("\nThis module should be imported, not run directly.")
    print("Use the training pipeline in notebooks or main training script.")

```

src/train_model.py

```

"""
Model Training Pipeline
Train Random Forest Classifier for flood prediction

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    confusion_matrix,
    accuracy_score,
    roc_curve,
    precision_recall_curve,
    f1_score
)
import joblib
from pathlib import Path
import sys

# Add configs to path
sys.path.append(str(Path(__file__).parent.parent / 'configs'))
PROCESSED_DATA_DIR, (
    MODELS_DIR,
    FEATURE_COLUMNS,
    N_ESTIMATORS,
    MAX_DEPTH,
    MIN_SAMPLES_SPLIT,
    MIN_SAMPLES_LEAF,
    RANDOM_STATE,
    TEST_SIZE
)

# Set style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)

def load_clean_data():
    Load preprocessed training data"""
    filepath = Path(PROCESSED_DATA_DIR / 'flood_training_clean.csv')
    if not filepath.exists():
        print(f"\nClean data not found: {filepath}")
        print("Run eda_anatysis.py first!")
        return None
    df = pd.read_csv(filepath)
    print(f"\nLoaded clean dataset: {len(df)} samples")
    return df

def prepare_features_labels(df):
    Extract feature columns and values
    X = df[FEATURE_COLUMNS].values
    y = df[TARGET_COLUMN].values
    print(f"\nX Features: {X} {X.shape}")
    print(f"\ny Labels: {y} {y.shape}")
    return X, y

def split_data(X, y):
    Split into train and test sets"""
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=TEST_SIZE,
        random_state=RANDOM_STATE,
        stratify=y # Maintain class balance
    )
    print(f"\nTrain set: {X_train.shape[0]} samples")
    print(f"\nTest set: {X_test.shape[0]} samples")
    print(f"\nTrain:flood_ratio: {y_train.sum() / len(y_train):.2%}")
    print(f"\nTest:flood_ratio: {y_test.sum() / len(y_test):.2%}")
    return X_train, X_test, y_train, y_test

def train_model(X_train, y_train):
    Train Random Forest Classifier"""
    print(f"\nTRAINING RANDOM FOREST MODEL")
    print(f"\n")
    print(f"\nHyperparameters:")
    print(f"\n- n_estimators: {N_ESTIMATORS}")
    print(f"\n- max_depth: {MAX_DEPTH}")
    print(f"\n- min_samples_split: {MIN_SAMPLES_SPLIT}")
    print(f"\n- min_samples_leaf: {MIN_SAMPLES_LEAF}")
    print(f"\n- random_state: {RANDOM_STATE}")
    model = RandomForestClassifier(
        n_estimators=N_ESTIMATORS,
        max_depth=MAX_DEPTH,
        min_samples_split=MIN_SAMPLES_SPLIT,
        min_samples_leaf=MIN_SAMPLES_LEAF,
        random_state=RANDOM_STATE,
        verbose=1
    )
    print(f"\nTraining in progress...")
    model.fit(X_train, y_train)
    print(f"\nTraining complete!")
    return model

def evaluate_model(model, X_train, X_test, y_train, y_test):
    Comprehensive model evaluation"""
    print(f"\nMODEL EVALUATION")
    print(f"\n")
    # Predictions
    y_train_pred = model.predict(X_train)
    y_train_proba = model.predict_proba(X_train)[:, 1]
    y_test_pred = model.predict(X_test)
    y_test_proba = model.predict_proba(X_test)[:, 1]
    # Metrics
    train_acc = accuracy_score(y_train, y_train_pred)
    train_f1 = f1_score(y_train, y_train_pred)
    train_roc = roc_auc_score(y_train, y_train_proba)
    test_acc = accuracy_score(y_test, y_test_pred)
    test_f1 = f1_score(y_test, y_test_pred)
    test_roc = roc_auc_score(y_test, y_test_proba)
    print(f"\nMetrics: <20 {train_acc:<15.4f} {test_acc:<15.4f}")
    print(f"\nAccuracy: <20 {train_acc:<15.4f} {test_acc:<15.4f}")
    print(f"\nF1 Score: <20 {train_f1:<15.4f} {test_f1:<15.4f}")

```

```

print(f"{'ROC-AUC':<20} {train_roc:<15.4f} {test_roc:<15.4f}")
print(f"Confusion Matrix Set Confusion Matrix")
cm = ConfusionMatrix(y_test, y_test_pred)
print(f"True Negatives: {cm[0,0]}")
print(f"False Negatives: {cm[0,1]}")
print(f"False Positives: {cm[1,0]}")
print(f"True Positives: {cm[1,1]}")
# Classification Report
print(classification_report(y_test, y_test_pred, target_names=['No Flood', 'Flood']))

train_acc = train_acc,
test_acc = test_acc,
train_f1 = train_f1,
test_f1 = test_f1,
train_roc = train_roc,
test_roc = test_roc,
y_test_pred = y_test_pred,
y_test_proba = y_test_proba

def plot_confusion_matrix(y_test, y_test_pred):
    plt.figure(figsize=(8, 6))
    cm = ConfusionMatrix(y_test, y_test_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=['No Flood', 'Flood'],
                yticklabels=['Predicted No Flood', 'Predicted Flood'])
    plt.title('Confusion Matrix - Test Set', fontsize=14, fontweight='bold')
    plt.xlabel('Predicted Label')
    plt.ylabel('Actual Label')
    output_path = Path(file_dir, parent.parent / 'data' / 'outputs' / 'confusion_matrix.png')
    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    print(f"Saved: {output_path}")
    plt.close()

def plot_roc_curve(y_test, y_test_proba):
    fpr, tpr, thresholds = roc_curve(y_test, y_test_proba)
    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC Curve (AUC = {roc_auc:.3f})')
    plt.plot([0, 1], [0, 1], color='red', lw=2, linestyle='--', label='Random Classifier')
    plt.xlabel('False Positive Rate', fontsize=12)
    plt.ylabel('True Positive Rate', fontsize=12)
    plt.legend(loc='lower right', fontsize=14, fontweight='bold')
    plt.grid(alpha=0.3)
    output_path = Path(file_dir, parent.parent / 'data' / 'outputs' / 'roc_curve.png')
    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    print(f"Saved: {output_path}")
    plt.close()

def plot_feature_importance(model):
    importance = FeatureImportance({
        'importance': model.feature_importances_,
        'sort_values': 'importance', ascending=False})
    print(f"FEATURE IMPORTANCE")
    importance_df = importance.to_dataframe()
    print(importance_df.to_string(index=False))
    plt.figure(figsize=(10, 6))
    colors = sns.color_palette("viridis", len(FEATURE_COLUMNS))
    plt.bar(importance_df['importance'], importance_df['importance'], color=colors)
    plt.xlabel('Feature Importance Score', fontsize=12)
    plt.ylabel('Feature Importance - Random Forest', fontsize=14, fontweight='bold')
    plt.grid(alpha=0.3)
    output_path = Path(file_dir, parent.parent / 'data' / 'outputs' / 'feature_importance.png')
    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    print(f"Saved: {output_path}")
    plt.close()

def save_model(model):
    output_path = Path(file_dir, parent.parent / 'data' / 'models' / 'random_forest_flood_model.pkl')
    joblib.dump(model, output_path)
    file_size = os.path.getsize(output_path) / 1024 # KB
    print(f"File Size: {file_size:.2f} KB")

def save_training_report(metrics):
    report = f"""
    PREDICTION MODEL - TRAINING REPORT
    Model: Random Forest Classifier
    Training Samples: {len(X_train)}
    Test Samples: {len(X_test)}
    Hyperparameters:
    - n_estimators: {RF.n_estimators}
    - max_depth: {RF.max_depth}
    - min_samples_split: {RF.min_samples_split}
    - min_samples_leaf: {RF.min_samples_leaf}
    Performance Metrics:
    - Train Accuracy: {metrics['train_acc']:.4f}
    - Test Accuracy: {metrics['test_acc']:.4f}
    - Train F1 Score: {metrics['train_f1']:.4f}
    - Test F1 Score: {metrics['test_f1']:.4f}
    - Train ROC-AUC: {metrics['train_roc']:.4f}
    - Test ROC-AUC: {metrics['test_roc']:.4f}
    The model shows good generalization if abs(metrics['train_acc'] - metrics['test_acc']) < 0.05.
    """
    report = report.strip()
    output_path = Path(file_dir, parent.parent / 'data' / 'outputs' / 'training_report.txt')
    with open(output_path, 'w', encoding='utf-8') as f:
        f.write(report)
    print(f"Report saved: {output_path}")

def main():
    print(f"Phase 3: FLOOD PREDICTION - MODEL TRAINING")
    # Load data
    df = load_data()
    if df is None:
        return
    # Prepare features and labels
    X, y = prepare_features_labels(df)
    # Split data
    X_train, X_test, y_train, y_test = split_data(X, y)
    # Train model
    model = train_model(X_train, y_train)
    # Evaluate model
    metrics = evaluate_model(model, X_train, X_test, y_train, y_test)
    # Visualizations
    plot_confusion_matrix(metrics['y_test'], metrics['y_test_pred'])
    plot_roc_curve(metrics['y_test'], metrics['y_test_proba'])
    plot_feature_importance(model)
    # Save model
    save_model(model)
    # Save training report
    save_training_report(metrics)
    print(f"TRAINING COMPLETE!")
    print(f"Generated Files:")
    print(f"- confusion_matrix.png")
    print(f"- roc_curve.png")
    print(f"- feature_importance.png")
    print(f"- training_report.txt")
    print(f"- random_forest_flood_model.pkl")
    print(f"\nReady for Phase 4: GEE Integration & Testing")

```

```
if __name__ == "__main__":
    main()
```

src/utils.py

```
"""
Utility Functions
Helper functions for the flood prediction system
"""
import numpy as np
import pandas as pd

def classify_risk_level(probability):
    """
    Classify flood probability into risk levels
    Probability (float): Flood probability (0-1)
    Returns: Risk level
    """
    if probability >= 0.7:
        return "HIGH"
    elif probability >= 0.4:
        return "MEDIUM"
    else:
        return "LOW"

def validate_coordinates(lat, lon):
    """
    Validate latitude and longitude
    Args: (float): Latitude
           (float): Longitude
    Returns: bool: True if valid
    """
    if not (-90 <= lat <= 90):
        return False
    if not (-180 <= lon <= 180):
        return False
    return True

def format_prediction_response(probability, features=None):
    """
    Format prediction response for API
    Probability (float): Flood probability
    Features (dict): Optional feature values
    Returns: Formatted response
    """
    response = {
        'probability': round(probability, 4),
        'confidence': 'high' if abs(probability - 0.5) > 0.3 else 'medium'
    }
    if features:
        response['features'] = features
    return response

def print_banner(text):
    """
    Print a formatted banner
    Args: (str): Text to display
    """
    width = (len(text) * 4)
    print(f"{text}{' ' * width}")
    print(f"{'=' * width}")

if __name__ == "__main__":
    # Test utilities
    print(classify_risk_level(0.85)) # HIGH
    print(classify_risk_level(0.55)) # MEDIUM
    print(classify_risk_level(0.25)) # LOW
    print(validate_coordinates(17.385, 78.486)) # True
    print(validate_coordinates(100, 200)) # False
```

start_server.bat

```
@echo off
REM =====
REM Start Flood Prediction System Server
REM =====

echo.=====
echo. STARTING FLOOD PREDICTION SERVER
echo.=====

REM Check if venv exists
echo [ERROR] Virtual environment not found!
echo Please run: setup.bat first
echo.
pause
exit /b 1

REM Activate virtual environment
call Venv\Scripts\activate.bat

REM Start Flask server
echo. Starting Flask server
echo. Press Ctrl+C to stop the server
echo. Server will be available at:
echo. - http://localhost:5000
echo. - http://127.0.0.1:5000
python api\flask_app.py
```

static/css/style.css

```
:root {
  --ink: #111827;
  --muted: #556272;
  --panel: #f7f8fc;
  --line: #d9d9d9;
  --olive: #808080;
  --leaf: #00766b;
  --green: #16a34a;
  --amber: #ffc107;
  --shadow: 0 18px 50px rgba(15, 23, 42, 0.12);
}

* {
  margin: 0;
  padding: 0; border-box;
}

body {
  height: 100vh;
  font-family: inherit, "Segoe UI", Arial, sans-serif;
  color: var(--ink);
  background: radial-gradient(180deg, rgba(11, 31, 51, 0.96), rgba(11, 31, 51, 0.88) 320px, #edf2f7 321px), #edf2f7;
}

button,
input,
select {
  font: inherit;
}

.app-shell {
  width: min(1500px, calc(100% - 32px));
  padding: 24px 0 28px;
```

```

topbar {
  flex: 1;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 24px;
  color: white;
  padding: 18px 0 28px;
}

brand-lockup {
  display: flex;
  align-items: center;
  gap: 18px;
}

brand-mark {
  width: 58px;
  height: 58px;
  display: flex;
  align-items: center;
  justify-content: center;
  border: 1px solid rgba(255, 255, 255, 0.22);
  border-radius: 8px;
  background: linear-gradient(135deg, #0f766e, #1d4ed8);
  letter-spacing: 0;
}

eyebrow {
  color: #0f766e;
  font-size: 0.74rem;
  font-weight: 800;
  letter-spacing: 0.12em;
}

topbar-eyebrow {
  color: #8bd9d1;
}

topbar-h1 {
  margin-top: 40px;
  font-size: clamp(2rem, 4vw, 4.4rem);
  height: 90px;
  letter-spacing: 0;
}

subtitle {
  max-width: 760px;
  color: #cbd5e1;
  font-size: 1rem;
}

topbar-actions {
  display: flex;
  align-items: center;
  gap: 12px;
}

.notification-link,
.logout-button {
  width: 140px;
  height: 40px;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 9px;
  padding: 10px 14px;
  text-decoration: none;
  font-weight: 800;
}

.notification-link {
  position: relative;
  color: white;
  background: linear-gradient(135deg, rgba(255, 255, 255, 0.25), rgba(255, 255, 255, 0.18));
}

.notification-badge {
  min-width: 22px;
  height: 22px;
  display: flex;
  align-items: center;
  justify-content: center;
  border: 1px solid #f08080;
  border-radius: 50%;
  background: var(--red);
  font-size: 0.75rem;
}

user-chip {
  display: flex;
  align-items: center;
  gap: 18px;
  padding: 8px 20px;
  border: 1px solid rgba(255, 255, 255, 0.18);
  border-radius: 8px;
  background: linear-gradient(135deg, rgba(255, 255, 255, 0.08), rgba(255, 255, 255, 0.04));
}

avatar {
  width: 32px;
  height: 32px;
  display: flex;
  align-items: center;
  justify-content: center;
  border: 1px solid #000000;
  background: var(--heavy);
  font-weight: 900;
}

.user-chip-strong {
  display: block;
}

user-chip-small {
  overflow: hidden;
  color: #cbd5e1;
  text-overflow: ellipsis;
  white-space: nowrap;
}

logout-button {
  background: var(--red);
}

status-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 14px;
  margin-bottom: 18px;
}

.status-tile {
  border: 1px solid #000000;
  border-radius: 8px;
  background: linear-gradient(135deg, #000000, #000000);
  padding: 18px;
}

.status-tile-strong,
.status-tile-small,
.status-label {
  display: block;
}

.status-label {
  color: var(--muted);
  font-size: 0.88em;
  text-transform: uppercase;
}

.status-tile-strong {
  margin-top: 8px;
}

```

```

font-size: 1.45rem;
}
status-tile-small {
margin-top: 50px;
color: var(--muted);
}
status-tile-accent {
border-top: 4px solid var(--teal);
}
command-layout {
display: grid;
grid-template-columns: 420px minmax(0, 1fr);
gap: 18px;
}
:operations-panel,
:operations-section {
padding: 22px;
}
panel-header {
margin-bottom: 18px;
}
panel-header h2 {
margin-top: 50px;
font-size: 1.45rem;
}
panel-header.compact {
margin-bottom: 12px;
}
map-heading {
display: flex;
align-items: flex-end;
justify-content: space-between;
gap: 20px;
}
input-grid {
display: grid;
gap: 14px;
}
input-group label {
margin-bottom: 6px;
color: #334155;
font-size: 0.86rem;
font-weight: 800;
}
input-group input,
input-group select {
width: 100%;
height: 48px;
padding: 12px 10px;
border: 1px solid #cbd5e1;
border-radius: 8px;
color: var(--ink);
transition: border-color 0.2s, box-shadow 0.2s;
}
input-group input:focus,
input-group select:focus {
border-color: var(--blue);
box-shadow: 0 0 4px rgba(29, 78, 216, 0.12);
}
btn-primary,
btn-secondary {
border: 1px solid #cbd5e1;
border-radius: 8px;
font-weight: 900;
}
btn-primary {
width: 100%;
height: 58px;
margin-top: 18px;
background: linear-gradient(135deg, var(--blue), var(--teal));
}
btn-primary:hover {
filter: brightness(1.04);
}
quick-locations {
margin-top: 22px;
border-top: 1px solid var(--line);
}
quick-locations p {
margin-bottom: 10px;
color: #334155;
font-weight: 900;
}
quick-location-grid {
display: grid;
grid-template-columns: repeat(2, minmax(0, 1fr));
gap: 10px;
}
btn-secondary {
min-height: 42px;
padding: 10px 12px;
border: 1px solid #cbd5e1;
background: #f8f9fc;
}
btn-secondary:hover {
border-color: var(--blue);
background: #eff6ff;
}
loading-card {
margin-top: 18px;
padding: 8px;
box-shadow: none;
}
loading-card {
text-align: center;
}
loading-spinner {
width: 42px;
height: 42px;
border: 4px solid #000000;
border-top: 4px solid #000000;
border-top-color: var(--blue);
animation: spin 0.9s linear infinite;
}
@keyframes spin {
transform: rotate(360deg);
}
risk-indicator {
margin-bottom: 16px;
}
risk-percentage {
margin-bottom: 28px;
font-size: 0.9rem;
font-weight: 900;
line-height: 1;
}

```



```

}
risk-gauge {
  overflow: hidden;
  border-radius: 999px;
  background: #e2e8f0;
}
.gauge-fill {
  width: 0%;
  height: 100%;
  border-radius: 999px;
  background: linear-gradient(90deg, var(--green), var(--amber), var(--red));
  transition: width 0.8s ease;
}
risk-badge span {
  display: inline-block;
  text-align: center;
  min-height: 36px;
  padding: 8px 14px;
  border-radius: 8px;
  font-weight: 800;
}
risk-low {
  color: #166534;
  background: #d8fce7;
}
risk-medium {
  color: #24409e;
  background: #f3f3c7;
}
risk-high {
  color: #b11b1b;
  background: #fee2e2;
}
features-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 16px;
  margin-top: 16px;
}
feature-item {
  padding: 16px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #f8fafc;
}
feature-label {
  color: var(--muted);
  font-weight: 600;
  text-transform: uppercase;
}
feature-value {
  margin-top: 7px;
  font-size: 1.25rem;
  font-weight: 900;
}
date-range {
  margin-top: 14px;
  padding-top: 1px;
  border-radius: 8px;
  color: #475569;
  background: #eef2f7;
  font-size: 0.9rem;
}
map-panel {
  height: 680px;
  display: flex;
  flex-direction: column;
}
#map {
  height: 590px;
  overflow: hidden;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #dbe4ef;
}
map-legend {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}
legend-item {
  display: inline-block;
  align-items: center;
  color: #334155;
  font-size: 0.86rem;
  font-weight: 800;
}
legend-color {
  width: 16px;
  height: 16px;
  border-radius: 3px;
}
legend-color.low {
  background: var(--green);
}
legend-color.medium {
  background: var(--amber);
}
legend-color.high {
  background: var(--red);
}
threshold-section {
  margin-top: 18px;
}
threshold-grid {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 12px;
}
threshold-card {
  padding: 16px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel);
}
.threshold-card span {
  display: block;
  font-weight: 800;
}
.threshold-card.strong {
  color: var(--muted);
  font-size: 0.88rem;
  font-weight: 800;
  text-transform: uppercase;
}
.threshold-card.strong {
  font-size: 1.6rem;
}
.threshold-card.small {
  color: var(--muted);
}
footer {
  margin-top: 18px;
}

```

```
padding: 18px;
color: gray;
box-shadow: none;
}
leaflet-popup-content-wrapper {
border-radius: 8px;
}
custom-marker {
border: 0;
background: transparent;
}
.marker-dot {
width: 20px;
height: 20px;
border: 2px solid white;
border-radius: 50%;
box-shadow: 0 10px 24px rgba(15, 23, 42, 0.34);
}
.marker-default {background: var(--blue);}
.marker-low {background: var(--green);}
.marker-medium {background: var(--amber);}
.marker-high {background: var(--red);}
.popup-content {
min-width: 180px;
text-align: center;
}
.popup-content h3 {
margin: 0;
color: var(--ink);
}
.popup-risk {
display: inline-block;
padding: 6px 12px;
border-radius: 999px;
font-weight: 900;
}
@media (max-width: 1100px) {
topbar {
display: flex;
flex-direction: column;
}
topbar-actions {
width: 100%;
flex-wrap: wrap;
}
.status-grid {
grid-template-columns: repeat(2, minmax(0, 1fr));
}
command-layout {
grid-template-columns: 1fr;
}
}
@media (max-width: 680px) {
app-shell {
width: min(100%, 20px, 1500px);
padding-top: 14px;
}
.brand-lockup {
display: flex;
flex-direction: column;
}
brand-mark {
width: 50px;
height: 50px;
}
user-chip {
width: 100%;
min-width: 0;
}
.status-grid,
:feature-grid,
quick-location-grid {
grid-template-columns: 1fr;
}
operations-panel,
map-panel {
padding: 16px;
}
}
#map {
min-height: 420px;
}
}
```

static/js/app.js

```
// Initialize Map
let map;
let currentCircle;
// Initialize map on page load
document.addEventListener('DOMContentLoaded', function () {
initMap();
// Add event listener to predict button
document.getElementById('predictBtn').addEventListener('click', predictFlood);
// Allow Enter key in input fields
document.getElementById('latitude').addEventListener('keypress', function (e) {
if (e.key === 'Enter') predictFlood();
});
document.getElementById('longitude').addEventListener('keypress', function (e) {
if (e.key === 'Enter') predictFlood();
});
});
function initMap() {
// Create map centered on India
map = L.Map('map').setView([20.5937, 78.9629], 5);
// Add OpenStreetMap tiles
map.addLayer(L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {
maxZoom: 18,
attribution: '&copy; OpenStreetMap contributors',
})).addTo(map);
// Add click event to map
map.on('click', function (e) {
document.getElementById('latitude').value = e.latlng.lat.toFixed(4);
document.getElementById('longitude').value = e.latlng.lng.toFixed(4);
updateMarker(e.latlng.lat, e.latlng.lng);
});
// Set initial marker
const initialLat = parseFloat(document.getElementById('latitude').value);
const initialLon = parseFloat(document.getElementById('longitude').value);
updateMarker(initialLat, initialLon);
function updateMarker(lat, lon, riskLevel = null) {
```

```

// Remove existing marker and circle
map.removeLayer(marker);
if (currentCircle) {
  map.removeLayer(currentCircle);
}
// Determine marker color based on risk level
let markerColor = '#d4d8d8';
let iconHtml = '<span class="marker-dot marker-default"></span>';
if (riskLevel === 'HIGH') {
  markerColor = '#dc2626';
  iconHtml = '<span class="marker-dot marker-high"></span>';
} else if (riskLevel === 'MEDIUM') {
  markerColor = '#d97706';
  iconHtml = '<span class="marker-dot marker-medium"></span>';
} else if (riskLevel === 'LOW') {
  markerColor = '#1f77b4';
  iconHtml = '<span class="marker-dot marker-low"></span>';
}
// Create custom icon
const customIcon = L.divIcon({
  className: 'custom-marker',
  iconSize: [20, 20],
  iconAnchor: [11, 11]
});
// Add new marker
marker = new L.Marker([lat, lon], { icon: customIcon }).addTo(map);
// Add circle to show analysis area (10km radius)
const radius = 10000; // 10 km
const circle = L.Circle([lat, lon], {
  color: markerColor,
  fillColor: markerColor,
  opacity: 0.1,
  weight: 2
}).addTo(map);
// Center map on marker
map.setView([lat, lon], 10);

function selectLocation(lat, lon, name) {
  document.getElementById('latitude').value = lat;
  document.getElementById('longitude').value = lon;
  updateMarker(lat, lon);
  // Add popup with location name
  marker.bindPopup(`<b>${name}</b><br>Ready for risk assessment`).openPopup();
}

async function predictFlood() {
  const lat = parseFloat(document.getElementById('latitude').value);
  const lon = parseFloat(document.getElementById('longitude').value);
  // Validate inputs
  if (isNaN(lat) || isNaN(lon)) {
    alert('Please enter valid coordinates');
    return;
  }
  if (lat < -90 || lat > 90 || lon < -180 || lon > 180) {
    alert('Invalid coordinates! Latitude must be between -90 and 90, Longitude between -180 and 180');
    return;
  }
  // Show loading, hide results
  document.getElementById('loadingCard').style.display = 'block';
  try {
    // Make API request
    const response = await fetch('/predict', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      credentials: 'include', // Include session cookies
      body: JSON.stringify({
        latitude: lat,
        longitude: lon
      })
    });
    if (!response.ok) {
      throw new Error(`API Error: ${response.status}`);
    }
    const data = await response.json();
    // Hide loading
    document.getElementById('loadingCard').style.display = 'none';
    // Display results
    displayResults(data, lat, lon);
  } catch (error) {
    console.error('Error:', error);
    document.getElementById('loadingCard').style.display = 'none';
    alert('Failed to get prediction. Make sure the API server is running.');
  }

  function displayResults(data, lat, lon) {
    document.getElementById('resultsCard').style.display = 'block';
    // Update risk percentage and gauge
    const percentage = data.floodProbability * 100;
    document.getElementById('gaugeFill').style.width = percentage + '%';
    // Update risk level badge
    const riskBadge = document.getElementById('riskLevel');
    riskBadge.parentElement.className = 'risk-badge';
    if (data.risk_level === 'HIGH') {
      riskBadge.className = 'risk-high';
    } else if (data.risk_level === 'MEDIUM') {
      riskBadge.className = 'risk-medium';
    } else {
      riskBadge.className = 'risk-low';
    }
    // Update features
    document.getElementById('rainfall').textContent = data.features.rainfall_7d_mm.toFixed(2) + ' mm';
    document.getElementById('soilMoisture').textContent = data.features.soil_moisture.toFixed(4) + ' m³/m³';
    document.getElementById('elevation').textContent = data.features.elevation_m.toFixed(2) + ' m';
    document.getElementById('slope').textContent = data.features.slope_deg.toFixed(2) + ' deg';
    // Update date range
    document.getElementById('dateInfo').textContent = `
    ${data.date_range.start} to ${data.date_range.end}
  `;
    // Update marker on map
    updateMarker(lat, lon, data.risk_level);
    // Add popup to marker
    const popupContent = `
    <div class="popup-content">
      <div class="popup-risk">
        <div class="risk-level">
          ${data.risk_level === 'HIGH' ? 'risk-high' : data.risk_level === 'MEDIUM' ? 'risk-medium' : 'risk-low'}
        </div>
        <div class="risk-level">
          Risk
        </div>
        <div style="margin-top: 10px;">
          Probability: ${percentage}%
        </div>
      </div>
    </div>
  `;
    marker.bindPopup(popupContent).openPopup();
  }

  // Scroll to results
  document.getElementById('resultsCard').scrollIntoView({ behavior: 'smooth', block: 'nearest' });
  // Check if we need to create a notification (probability >= 50%)

```

```

if (data.flood_probability >= 0.5) {
  saveNotification(data.lat, lon);
  showAlertMessage(data.flood_probability, data.risk_level);
}

function saveNotification(data, lat, lon) {
  const notification = {
    timestamp: new Date().toISOString(),
    flood_probability: data.flood_probability,
    risk_level: data.risk_level,
    features: data.features,
    email_sent: data.email_sent || false
  };
  // Get existing notifications
  const notifications = JSON.parse(localStorage.getItem('floodNotifications') || '[]');
  // Add new notification at the beginning
  notifications.unshift(notification);
  // Keep only last 50 notifications
  notifications.splice(50);
  // Save back to local storage
  localStorage.setItem('floodNotifications', JSON.stringify(notifications));
  // Update notification badge
  updateNotificationBadge();
  console.log('Notification saved:', notification);
}

function showAlertMessage(probability, riskLevel) {
  let severityLabel = (probability > 0.75 ? 'HIGH' : probability > 0.5 ? 'MEDIUM' : 'LOW');
  let color = '#dc2626';
  if (riskLevel === 'MEDIUM') {
    color = '#d97706';
  }
  // Create alert message element
  const alertDiv = document.createElement('div');
  alertDiv.setAttribute('id', 'flood-alert-popup');
  alertDiv.style.position = 'fixed';
  alertDiv.style.top = '20px';
  alertDiv.style.left = '50px';
  alertDiv.style.backgroundColor = 'white';
  alertDiv.style.border = '1px solid ' + color;
  alertDiv.style.padding = '10px';
  alertDiv.style.zIndex = '1000';
  alertDiv.style.maxWidth = '400px';
  alertDiv.style.animation = 'slideDownRight 0.3s ease-out';

  alertDiv.innerHTML = `
    <div style="width: 140px; height: 140px; border-radius: 4px; background: ${color}; margin: 6px 15px 0 0;">
      <div style="flex: 1; display: flex; align-items: center; justify-content: center; color: #1e293b;">
        FLOOD ALERT
      </div>
      <div style="margin: 0 10px; color: #475569;">
        Flood probability: <strong style="color: ${color};">${probability}</strong>
      </div>
      <div style="margin: 0 15px; color: #64748b; font-size: 14px;">
        alert has been saved to your notifications.
      </div>
      <div style="display: flex; gap: 10px;">
        <div style="background: ${color}; color: white; padding: 8px 16px; border-radius: 6px; font-size: 14px; font-weight: bold;">
          Notifications
        </div>
        <div style="background: white; color: #333; padding: 8px 16px; border-radius: 6px; font-size: 14px; font-weight: bold;">
          Dismiss
        </div>
      </div>
    </div>
  `;
  alertDiv.onclick = "this.parentElement.parentElement.remove()" style="";
  // Add CSS animation
  const style = document.createElement('style');
  style.textContent = `
    @keyframes slideInRight {
      from {
        transform: translateX(100%);
        opacity: 0;
      }
      to {
        transform: translateX(0);
        opacity: 1;
      }
    }
  `;
  document.head.appendChild(style);
  document.body.appendChild(alertDiv);
  // Auto remove after 10 seconds
  setTimeout(() => alertDiv.remove(), 10000);
}

function updateNotificationBadge() {
  const notifications = JSON.parse(localStorage.getItem('floodNotifications') || '[]');
  const badge = document.getElementById('notificationBadge');
  if (notifications.length > 0) {
    badge.textContent = notifications.length;
    badge.style.display = 'block';
  } else {
    badge.style.display = 'none';
  }
}

// Update badge on page load
document.addEventListener('DOMContentLoaded', function () {
  updateNotificationBadge();
});

map.on('click', function(e) {
  const lat = e.latlng.lat.toFixed(6);
  document.getElementById('latitude').value = lat;
});

```

```
map.on('click', function(e) {
document.getElementById('latitude').value =
e.latlng.lat.toFixed(6);
document.getElementById('longitude').value =
e.latlng.lng.toFixed(6);
});
```

templates/admin_dashboard.html

```
<h1>Admin Dashboard</h1>
```

templates/citizen_dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css">
</head>
<body class="app-shell">
<header class="topbar">
<div class="brand-mark">FW</div>
<div class="eyebrow">Citizen Flood Awareness Network</p>
<h1 class="title">Real-time flood awareness, alerts and emergency assistance for citizens.</p>
</div>
<div class="topbar-actions">
<a class="notification-link" href="/notifications" title="Open alerts">
<span>Alerts</span>
<span id="notificationBadge" class="notification-badge">0</span>
</a>
<div class="user-chip">
<span class="avatar">{{ user.username[0]|upper if user and user.username else 'U' }}</span>
<strong>{{ user.username }}</strong>
<small>{{ user.email }}</small>
</div>
<a class="logout-button" href="/logout">Logout</a>
</div>
</header>
<section class="status-grid" aria-label="Citizen status">
<div class="status-tile">
<strong>CURRENT RISK</strong>
<small>Area Status</small>
</div>
<div class="status-tile">
<strong>MY LOCATION</strong>
<small>{{ user.status-area }}</small>
</div>
<div class="status-tile">
<strong>NEARBY SHELTERS</strong>
<small>Authority Integration Required</small>
</div>
<div class="status-tile accent">
<strong>EMERGENCY CONTACTS</strong>
<small>Authority Integration Required</small>
</div>
</section>
<main class="command-layout">
<section class="operations-panel">
<div class="panel-header">Citizen Information</p>
<h2>Location</h2>
</div>
<div class="card">
<div class="card">
<div class="card">
<div class="panel-header compact">
<h2>Report Incident</h2>
</div>
<label>Incident Type</label>
<select style="width:100%;padding:10px;">
<option>Water logging</option>
<option>Road block</option>
<option>Tree Fall</option>
<option>Rescue Required</option>
</select>
<br><br>
<label>Description</label>
<textarea
rows="4"
style="width:100%;"
placeholder="Describe the incident">
</textarea>
<br><br>
<label>Upload Evidence</label>
<input type="file"
accept="image/*"
capture="environment">
<br><br>
<button class="btn-primary">
<button>Report
</button>
</div>
</div class="card">
<div class="panel-header compact">
<h2>Local Authority Contacts</h2>
</div>
<div class="feature-item">
<div class="feature-label">Ward Officer</div>
</div>
<div class="feature-item">
<div class="feature-label">Municipal Control Room</div>
</div>
<div class="feature-item">
<div class="feature-label">Disaster Response Cell</div>
</div>
<div class="feature-item">
<div class="feature-label">Community Volunteer Coordinator</div>
</div>
</div>
<div class="card">
<div class="feature-item">
```


templates/government_dashboard.html

```
<h1>Government Dashboard</h1>
<p>{{ user.username }}</p>
<p>{{ user.sub_role }}</p>
```

templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta viewport="width=device-width, initial-scale=1.0">
<title>NGO Disaster Intelligence Network</title>
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
<div class="app-shell">
<div class="header">
<div class="brand-markup">
<div class="eyebrow">NGO Disaster Intelligence Network</div>
<div class="eyebrow">AI-assisted flood risk operations using satellite signals, terrain intelligence, and
alerts.</div>
</div>
<div class="topbar-actions">
<a href="/notifications" title="Open alerts">
<span id="notificationBadge" class="notification-badge">0</span>
</a>
<div class="user-chip">
<span class="avatar">{{ user.username[0]|upper if user and user.username else 'U' }}</span>
<strong>{{ user.username }}</strong>
<small>{{ user.email }}</small>
</div>
<a class="logout-button" href="/logout">Logout</a>
</div>
</div>
<section class="status-grid" aria-label="Platform status">
<div class="status-label">Program Status</div>
<strong>Active</strong>
<small>NGO-ready flood response platform</small>
<div class="status-label">Model</div>
<strong>Random Forest</strong>
<small>Satellite-derived feature pipeline</small>
<div class="status-label">Coverage</div>
<strong>95%</strong>
<small>Coordinate-based assessment</small>
<div class="status-label accent">Alert Trigger</div>
<strong>On Hold</strong>
<small>Email and notification logging</small>
</section>
<main class="command-layout">
<section class="operations-panel">
<div class="eyebrow">Assessment Console</div>
<div>
<div class="input-group">
<label for="latitude">Latitude</label>
<input type="number" id="latitude" placeholder="17.3850" step="0.0001" value="17.385">
</div>
<div class="input-group">
<label for="longitude">Longitude</label>
<input type="number" id="longitude" placeholder="78.4867" step="0.0001" value="78.4867">
</div>
<button id="predictBtn" class="btn-primary">Run Risk Assessment</button>
</div>
<div class="quick-locations">
<div class="location-grid">
<button class="btn-secondary" onclick="selectLocation(17.385, 78.4867, 'Hyderabad')">Hyderabad</button>
<button class="btn-secondary" onclick="selectLocation(19.076, 72.8777, 'Mumbai')">Mumbai</button>
<button class="btn-secondary" onclick="selectLocation(28.7041, 77.1025, 'Delhi')">Delhi</button>
</div>
</div>
</div>
<div class="card-loading-card" id="loadingCard" style="display: none;">
<p>Processing satellite and terrain indicators...</p>
</div>
<div class="card-results-card" id="resultsCard" style="display: none;">
<div class="panel-header compact">
<div>Assessment Result</div>
</div>
<div class="risk-indicator" id="riskIndicator">--</div>
<div class="risk-gauge">
<div class="gauge-fill" id="gaugeFill"></div>
</div>
<div class="risk-badge" id="riskBadge">
<span id="riskLevel">LOADING</span>
</div>
<div class="features-grid">
<div class="feature-label">7-Day Rainfall</div>
<div class="feature-value">id="rainfall">-- mm</div>
<div class="feature-item">Soil Moisture</div>
<div class="feature-value">id="soilMoisture">-- m3/m3</div>
<div class="feature-item">Elevation</div>
<div class="feature-value">id="elevation">-- m</div>
<div class="feature-item">Terrain Slope</div>
<div class="feature-value">id="slope">-- deg</div>
</div>
<div class="date-range" id="dateRange">
<div>
<div>
</div>
</div>
</div>
<section class="map-panel">
<div class="panel-header map-heading">
<div class="eyebrow">Geospatial Monitoring</div>
<div class="assessment-map">
<div class="map-legend">
<div class="legend-item"><span class="legend-color low"></span><span>Low</span></div>
<div class="legend-item"><span class="legend-color high"></span><span>High</span></div>
</div>
<div id="map"></div>
</div>
</div>
</div>
<section class="threshold-section">
<div class="eyebrow">Risk Governance</div>
</div>
```



```

}
form-group label {
display: block;
margin-bottom: 7px;
font-size: 0.8rem;
font-weight: 900;
}
form-group input {
width: 100%;
height: 50px;
padding: 12px 40px;
border: 1px solid #cbd5e1;
border-radius: 8px;
font: inherit;
}
form-group input:focus {
outline: none;
box-shadow: 0 0 4px rgba(29, 78, 216, 0.12);
}
btn-primary {
width: 100%;
height: 52px;
margin-top: 4px;
border: 1px solid #000000;
background: linear-gradient(135deg, var(--blue), var(--teal));
font-weight: 900;
cursor: pointer;
}
btn-primary.disabled {
cursor: not-allowed;
opacity: 0.7;
}
auth-footer {
padding-top: 24px;
border-top: 1px solid var(--line);
color: var(--muted);
text-align: center;
}
auth-footer a {
color: blue;
font-weight: 900;
text-decoration: none;
}
.error-message {
display: none;
padding: 12px 40px;
border: 1px solid #f0f0f0;
font-size: 0.92rem;
}
.success-message {
display: none;
padding: 12px 40px;
border: 1px solid #bbf7d0;
background-color: #bbf7d0;
font-size: 0.92rem;
}
.error-message.show {
display: block;
}
.loading-spinner {
display: none;
width: 28px;
height: 28px;
margin: 3px auto 16px;
border: 2px solid #deeaf6;
border-top: 2px solid var(--blue);
border-radius: 50%;
animation: spin 0.9s linear infinite;
}
.loading-spinner.show {
display: block;
}
@keyframes spin {
transform: rotate(360deg);
}
@media (max-width: 900px) {
body {
padding: 14px;
}
auth-shell {
grid-template-columns: 1fr;
}
brand-panel {
padding: 30px;
}
eyebrow {
margin-top: 42px;
}
trust-row {
grid-template-columns: 1fr;
margin-top: 30px;
}
auth-panel {
padding: 30px;
}
}
</style>
</head>
<body>
<section class="auth-shell">
<div class="brand-panel">
<div class="eyebrow">Floodwatch Command Center</div>
<p>Secure Access for NGO partners, government authorities, community coordinators, and field volunteers.</p>
<div class="trust-row">
<span>Trustworthy</span>
<span>Data Privacy</span>
<span>Email Alerts</span>
</div>
</section>
<section class="auth-panel">
<div class="login-form">
<div class="form-group">
<label for="username">Username or Email</label>
<input type="text" id="username" placeholder="name@example.org" required>
</div>
<div class="form-group">
<label for="password">Password</label>

```



```

margin-bottom: 16px;
border-top: 5px solid var(--blue);
}
notification-alert-high {
border-top-color: var(--red);
}
notification-alert-medium {
border-top-color: var(--amber);
}
notification-alert-low {
border-top-color: var(--green);
}
notification-header {
display: flex;
justify-content: space-between;
padding: 16px 22px;
border-bottom: 1px solid var(--line);
}
notification-title h2 {
font-size: 1.35rem;
}
notification-title p {
color: var(--muted);
}
notification-badge {
display: inline-flex;
align-items: center;
padding: 8px 34px;
border-radius: 999px;
font-weight: 900;
text-transform: uppercase;
}
notification-badge.high {
background: #fee2e2;
}
notification-badge.medium {
background: #fef3c7;
}
notification-badge.low {
background: #dcfce7;
}
probability-meter {
padding: 20px 22px 0;
}
probability-bar {
height: 16px;
border-bottom: 1px solid #e2e8f0;
background: #e2e8f0;
}
probability-fill {
height: 100%;
color: transparent;
}
probability-fill.high {
background: var(--red);
}
probability-fill.medium {
background: var(--amber);
}
probability-fill.low {
background: var(--green);
}
notification-body {
display: grid;
grid-template-columns: 1.2fr 0.8fr;
padding: 22px;
}
notification-section {
padding: 18px;
border: 1px solid var(--line);
background: #f8f9fc;
}
notification-section h3 {
margin-bottom: 14px;
font-size: 1rem;
}
detail-item {
display: flex;
justify-content: space-between;
padding: 8px 10px 0;
border-bottom: 1px solid #e2e8f0;
}
detail-item:last-child {
border-bottom: 0;
}
detail-label {
font-weight: 800;
}
detail-value {
font-weight: 900;
text-align: right;
}
feature-box {
border: 1px solid var(--line);
border-radius: 8px;
background: white;
}
feature-box-label {
color: var(--muted);
font-weight: 900;
text-transform: uppercase;
}
feature-box-value {
margin-top: 16px;
font-weight: 900;
}
recommendations {1}
recommendations ul {
grid-template-columns: repeat(2, minmax(0, 1fr));
gap: 8px 18px 18px;
color: #334155;
}
no-notifications {
text-align: center;
}
no-notifications h2 {
font-size: 1.8rem;
}

```

```

no-notifications p {
color: var(--muted);
}

.no-notifications a {
color: inherit;
text-decoration: none;
}

.dismiss-row {
padding: 0 22px 22px;
text-align: right;
}

@media (max-width: 840px) {
.alerts-header {
align-items: flex-start;
flex-direction: column;
}

.notification-body,
.recommendations {
grid-template-columns: 1fr;
}
}
</style>
<div id="alerts-shell">


>logged medium and high probability flood assessments for operational follow-up.</p>


>Alerts will appear here after a location crosses the configured risk threshold.</p>
</div>
</div>
</script>
function loadNotifications() {
const notifications = JSON.parse(localStorage.getItem('floodNotifications') || '[]');
const container = document.getElementById('notificationsList');
if (notifications.length === 0) {
container.innerHTML = 'No alerts';
return;
}
notifications.sort((a,b) => new Date(b.timestamp) - new Date(a.timestamp));
container.innerHTML = notifications.map((notif,index) => {
const probability = notif.riskLevel.probability(100).toFixed(1);
const date = new Date(notif.timestamp).toLocaleDateString();
const formattedDate = date.toLocaleString();
return `
<div class="notification-alert ${riskClass}">
<div class="notification-header">
<p>Location: ${notif.location.lat.toFixed(4)}, ${notif.location.lon.toFixed(4)} | ${formattedDate}</p>
<span class="notification-badge ${riskClass}">${probability}% probability</span>
</div>
<div class="probability-meter">
<div class="probability-fill ${riskClass}" style="width: ${probability}%;">${probability}%</div>
</div>
<div class="notification-body">
<div class="notification-section">
<div class="location-details">
<div class="feature-box">Rainfall 7-day</div>
<div class="feature-box-value">${notif.features.rainfall_7d_mm.toFixed(1)} mm</div>
<div class="feature-box">Soil Moisture</div>
<div class="feature-box-value">${notif.features.soil_moisture.toFixed(3)}</div>
<div class="feature-box">Elevation</div>
<div class="feature-box-value">${notif.features.elevation_m.toFixed(1)} m</div>
<div class="feature-box">Slope</div>
<div class="feature-box-value">${notif.features.slope_deg.toFixed(1)} deg</div>
</div>
<div class="notification-section">
<div class="location-details">
<div class="detail-label">Latitude</span><span>
<div class="detail-label">Longitude</span><span>
<div class="detail-label">Risk Level</span><span>
<div class="detail-label">Data Source</span><span>
</div>
<div class="notification-section recommendations">
<div class="recommended-actions">
<li>Monitor official weather and flood advisories.</li>
<li>Prepare emergency supplies and key documents.</li>
<li>Establish communication and contact with local authorities.</li>
</div>
</div>
</div>
<div class="dismiss-row">
button class="clear-btn" onclick="removeNotification(${index})">Dismiss</button>
</div>
</div>
</div>
</script>


```

</body>

templates/register.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Register - Floodwatch Command Center</title>
</head>
<style>
  :root {
    --muted: #111827;
    --blue: #00728b;
    --gray: #808080;
    --navy: #001f33;
    --blue: #00728b;
    --red: #d9534f;
    --green: #004d00;
    --amber: #d97706;
  }
  * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
  }
  body {
    min-height: 100vh;
    padding: 18px;
    font-family: Inter, "Segoe UI", Arial, sans-serif;
    background: linear-gradient(135deg, rgba(11, 31, 51, 0.96), rgba(15, 118, 110, 0.86)),
    #001f33;
  }
  .auth-shell {
    width: 1180px;
    height: 100%;
    display: flex;
    grid-template-columns: 420px minmax(0, 1fr);
    margin-top: 0 auto;
    border: 1px solid rgba(255, 255, 255, 0.16);
    border-radius: 10px;
    background: white;
    box-shadow: 0 30px 80px rgba(0, 0, 0, 0.28);
  }
  .brand-panel {
    padding: 14px;
    background: white;
    border: 1px solid linear-gradient(180deg, rgba(11, 31, 51, 0.22), rgba(11, 31, 51, 0.82)),
    #001f33, linear-gradient(circle at 20% 16%, rgba(45, 212, 191, 0.24), transparent 32%),
    #001f33;
  }
  .brand-mark {
    width: 80px;
    height: 58px;
    display: flex;
    align-items: center;
    justify-content: center;
    border: 1px solid rgba(255, 255, 255, 0.2);
    border-radius: 8px;
    background: linear-gradient(135deg, var(--teal), var(--blue));
    font-weight: 900;
  }
  .eyebrow {
    font-size: 0.8rem;
    font-weight: 900;
    text-transform: uppercase;
    letter-spacing: 0.12em;
  }
  .brand-panel .eyebrow {
    color: #80d9d1;
  }
  .brand-panel h1 {
    font-size: clamp(2.4rem, 4.8vw, 4.4rem);
    line-height: 0.98;
  }
  .brand-panel p:not(.eyebrow) {
    margin-top: 0.5em;
    line-height: 1.7;
  }
  .role-list {
    display: grid;
    gap: 12px;
    margin-top: 42px;
  }
  .role-list span {
    padding: 14px;
    border: 1px solid rgba(255, 255, 255, 0.18);
    border-radius: 8px;
    color: #e2e8f0;
    background: rgba(255, 255, 255, 0.08);
    font-weight: 800;
  }
  .auth-panel {
    padding: 38px 42px;
  }
  .auth-header {
    margin-bottom: 24px;
  }
  .auth-header h2 {
    font-size: 2rem;
  }
  .auth-header p {
    color: var(--muted);
  }
  .form-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 16px;
  }
  .form-group.full {
    grid-column: 1 / 1;
  }
  .form-group label {
    margin-bottom: 7px;
    color: #33415a;
    font-weight: 800;
  }
  .form-group select {
    width: 100%;
    height: 50px;
    padding: 12px 10px;
    border: 1px solid #cbd5e1;
    border-radius: 8px;
    color: var(--ink);
    background: white;
    font: inherit;
  }
  .form-group input:focus {
    outline: none;
    border-color: var(--blue);
    box-shadow: 0 0 0 4px rgba(29, 78, 216, 0.12);
  }
```

```

form-group {
  border-color: var(--red);
  padding: 10px;
  margin-bottom: 10px;
}

form-group input {
  width: 100%;
  height: 40px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

form-group .field-error {
  display: none;
  margin-top: 5px;
  color: #e28f00;
  font-size: 0.8rem;
}

form-group .field-error.show {
  display: block;
}

form-group .strength-bar {
  height: 10px;
  border: 1px solid #ccc;
  border-radius: 5px;
  background: #e28f00;
}

form-group .strength-bar-fill {
  height: 10px;
  width: 0%;
  transition: width 0.2s, background 0.2s;
}

form-group .strength-weak {
  width: 33%;
  background: var(--red);
}

form-group .strength-medium {
  width: 66%;
  background: var(--amber);
}

form-group .strength-strong {
  width: 100%;
  background: var(--green);
}

form-group .btn-primary {
  width: 100%;
  height: 40px;
  margin-top: 10px;
  border: 1px solid #ccc;
  border-radius: 5px;
  color: white;
  background: linear-gradient(135deg, var(--blue), var(--teal));
  font-weight: bold;
  cursor: pointer;
}

form-group .btn-primary.disabled {
  cursor: not-allowed;
  opacity: 0.7;
}

form-group .auth-footer {
  margin-top: 20px;
  border-top: 1px solid var(--line);
  color: var(--muted);
  text-align: center;
}

form-group .auth-footer a {
  color: var(--blue);
  text-decoration: none;
}

form-group .error-message {
  display: none;
  margin-top: 10px;
  border: 1px solid #fecaca;
  background: #fff2cc;
  padding: 5px;
  font-size: 0.92rem;
}

form-group .error-message.show {
  display: block;
}

form-group .success-message {
  display: none;
  margin-top: 10px;
  border: 1px solid #bbf7d0;
  background: #d1ecf1;
  padding: 5px;
  font-size: 0.92rem;
}

form-group .success-message.show {
  display: block;
}

form-group .loading-spinner {
  display: none;
  width: 20px;
  height: 20px;
  margin: 0 auto;
  border: 2px solid var(--blue);
  border-radius: 50%;
  animation: spin 0.9s linear infinite;
}

form-group .loading-spinner.show {
  display: block;
}

@keyframes spin {
  transform: rotate(360deg);
}

@media (max-width: 980px) {
  form-group {
    padding: 14px;
  }

  form-group .auth-shell {
    grid-template-columns: 1fr;
  }

  form-group .brand-panel {
    padding: 30px;
  }

  form-group .brand-panel .eyebrow {
    margin-top: 42px;
  }

  form-group .auth-panel {
    padding: 30px;
  }

  form-group .auth-panel .eyebrow {
    margin-top: 42px;
  }

  form-group .auth-panel .form-group {
    margin-top: 20px;
  }

  form-group .auth-panel .btn-primary {
    margin-top: 20px;
  }

  form-group .auth-panel .auth-footer {
    margin-top: 20px;
  }

  form-group .auth-panel .error-message {
    margin-top: 10px;
  }

  form-group .auth-panel .success-message {
    margin-top: 10px;
  }

  form-group .auth-panel .loading-spinner {
    margin-top: 10px;
  }
}

</style>
</head>
<body>
  <div class="auth-shell">
    <div class="brand-panel">
      <div class="eyebrow">
        <h1>Secure Registration</h1>
      </div>
      <div class="form-group">
        <input type="text" value="Name" />
        <input type="password" value="Password" />
        <input type="password" value="Confirm Password" />
        <input type="checkbox" value="Remember Me" />
        <input type="checkbox" value="I agree with the Terms and Conditions" />
        <input type="button" value="Register" />
      </div>
      <div class="auth-footer">
        <p>© 2023 Secure Registration. All rights reserved.</p>
      </div>
    </div>
    <div class="auth-panel">
      <div class="eyebrow">
        <h2>Secure Registration</h2>
      </div>
      <div class="form-group">
        <input type="text" value="Name" />
        <input type="password" value="Password" />
        <input type="password" value="Confirm Password" />
        <input type="checkbox" value="Remember Me" />
        <input type="checkbox" value="I agree with the Terms and Conditions" />
        <input type="button" value="Register" />
      </div>
      <div class="auth-footer">
        <p>© 2023 Secure Registration. All rights reserved.</p>
      </div>
    </div>
  </div>
</body>
</html>

```

```

<p>Set up your operational profile for FloodWatch access.</p>
<div class="loading-spinner" id="loading"></div>
<div class="success-message" id="success"></div>
<div id="registerForm">
<div class="form-group">
<label for="username">Username</label>
<input type="text" id="username" name="username" placeholder="Choose username" required>
<div class="field-error" id="usernameError"></div>
</div>
<div class="form-group">
<label for="email">Email</label>
<input type="text" id="email" name="email" placeholder="name@example.org" required autocomplete="email">
<div class="field-error" id="emailError"></div>
</div>
<div class="form-group">
<label for="phone">Phone Number</label>
<input type="text" id="phone" name="phone" placeholder="Mobile number" required>
</div>
<div class="form-group">
<label for="role">Role</label>
<select id="role" name="role" required>
<option value="citizen">Citizen</option>
<option value="government">Government Authority</option>
</select>
</div>
<div class="form-group">
<label for="subRole">Sub Role</label>
<select id="subRole" name="subRole">
<option value="">Select</option>
</select>
</div>
<div class="form-group">
<label for="district">District</label>
<input type="text" id="district" name="district" placeholder="District">
</div>
<div class="form-group">
<label for="city">City</label>
<input type="text" id="city" name="city" placeholder="City">
</div>
<div class="form-group">
<label for="area">Area</label>
<input type="text" id="area" name="area" placeholder="Area or locality">
</div>
<div class="form-group">
<label for="password">Password</label>
<input type="password" id="password" name="password" placeholder="Minimum 6 characters" required>
<div class="strength-bar-fill" id="strengthBar"></div>
<div class="field-error" id="passwordError"></div>
</div>
<div class="form-group">
<label for="retypePassword">Confirm Password</label>
<input type="password" id="retypePassword" name="retype_password" placeholder="Re-enter password" required>
<div class="field-error" id="retype_passwordError"></div>
</div>
<button type="submit" class="btn-primary" id="registerButton">Create Account</button>
</form>
<div class="auth-footer">
<p>Already registered? <a href="/login">Login here</a></p>
</div>
</main>
</script>
const registerButton = document.getElementById('registerForm');
const loading = document.getElementById('loading');
const errorDiv = document.getElementById('error');
const successDiv = document.getElementById('success');
const passwordInput = document.getElementById('password');
function showErrorMessage(message) {
errorDiv.classList.add('show');
successDiv.classList.remove('show');
}
function showSuccessMessage(message) {
successDiv.classList.add('show');
errorDiv.classList.remove('show');
}
function hideMessages() {
successDiv.classList.remove('show');
}
function showFieldError(fieldId, message) {
const errorElement = document.getElementById(fieldId) + 'Error';
const inputElement = document.getElementById(fieldId);
errorElement.textContent = message;
errorElement.classList.add('show');
inputElement.classList.add('error-input');
}
function clearFieldErrors() {
document.querySelectorAll('input').forEach(el => el.classList.remove('error-input'));
}
passwordInput.addEventListener('input', (e) => {
const password = e.target.value;
const strength = calculatePasswordStrength(password);
const strengthBar = document.getElementById('strengthBar');
if (strength === 'weak') strengthBar.classList.add('strength-weak');
else if (strength === 'medium') strengthBar.classList.add('strength-medium');
else if (strength === 'strong') strengthBar.classList.add('strength-strong');
});
function calculatePasswordStrength(password) {
let strength = 0;
if (password.length >= 6) strength++;
if (password.length >= 8) strength++;
if (/./test(password)) strength++;
if (/0-9/.test(password)) strength++;
if (/a-z/.test(password)) strength++;
if (password.length >= 12) return 'strong';
return 'medium';
}
const roleSelect = document.getElementById('role');
roleSelect.addEventListener('change', function() {
const role = roleSelect.value;
const subRoleSelect = document.getElementById('subRole');
subRoleSelect.innerHTML = '';
if (role === 'community_partner') {
subRoleSelect.innerHTML += `
<option value="ngo">NGO</option>
<option value="voluntary_group">Voluntary Organisation</option>
<option value="self_volunteer">Self Volunteer Group</option>
`;
} else if (role === 'government') {
subRoleSelect.innerHTML += `
<option value="municipal">Municipal Authority</option>
<option value="disaster_management_dept">Disaster Management Dept</option>
<option value="revenue">Revenue Department</option>
`;
}
});

```

```

    } else {
        subRoleSelect.innerHTML = `<option value="citizen">Citizen</option>`;
    });
    roleSelect.dispatchEvent(new Event('change'));
    registerForm.addEventListener('submit', async (e) => {
        e.preventDefault();
        clearFieldErrors();
        const username = document.getElementById('username').value.trim();
        const email = document.getElementById('email').value.trim();
        const password = document.getElementById('password').value;
        const retypePassword = document.getElementById('retype_password').value;
        if (password !== retypePassword) {
            showFieldError('retype_password', 'Passwords do not match');
            return;
        }
        if (password.length < 6) {
            showFieldError('password', 'Password must be at least 6 characters');
            return;
        }
        registerButton.disabled = true;
        loading.classList.add('show');
        try {
            const response = await fetch('/register', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json'
                },
                body: JSON.stringify({
                    username: username,
                    email: email,
                    phone: document.getElementById('phone').value,
                    sub_role: document.getElementById('sub_role').value,
                    district: document.getElementById('district').value,
                    area: document.getElementById('area').value,
                    password: password,
                    retype_password: retypePassword
                })
            });
            const data = await response.json();
            if (data.success) {
                showSuccess(data.message || 'Registration successful. Redirecting...');
                setTimeout(() => {
                    window.location.href = data.redirect || '/';
                }, 3000);
            } else {
                if (data.errors) {
                    if (data.errors.username) showFieldError('username', data.errors.username);
                    if (data.errors.email) showFieldError('email', data.errors.email);
                    if (data.errors.password) showFieldError('password', data.errors.password);
                    if (data.errors.general) showFieldError('general', data.errors.general);
                }
                showError('Registration failed. Please try again.');
```

tests/test_api.py

```

"""
Flask API Test Script
Tests the Flood Prediction API endpoints
"""
import requests

BASE_URL = 'http://localhost:5000'

def test_health_check():
    """Test the health check endpoint"""
    print("\n--- TESTING HEALTH CHECK ---")
    try:
        response = requests.get(f'{BASE_URL}/')
        data = response.json()
        print(f"API is online!")
        print(f"Status: {data.get('status')}")
        print(f"GEE Status: {data.get('gee_status')}")
        return True
    except (f"Health check failed: {response.status_code}") as e:
        return False
    except Exception as e:
        print(f"Cannot connect to API: {e}")
        print(f"Is flask on api/flask_app.py is running?")
        return False

def test_prediction(lat, lon, location_name):
    """Test prediction for a location"""
    print(f"Testing: {location_name} ---")
    payload = {
        "coordinates": ({lat}, {lon})
    }
    payload["latitude"] = lat
    payload["longitude"] = lon
    try:
        response = requests.post(
            f'{BASE_URL}/predict',
            headers={
                'Content-Type': 'application/json'
            },
            json=payload
        )
        if response.status_code == 200:
            data = response.json()
            print(f"Prediction successful!")
            print(f"Flood Probability: {data['flood_probability']:.4f} ({data['flood_probability']*100:.2f}%)")
            print(f"Confidence: {data['confidence']:.4f}")
            print(f"Features Used:")
            print(f"Rainfall: {data['features']['rainfall_7d_mm']:.2f} mm")
            print(f"Moisture: {data['features']['soil_moisture_12h']:.2f} m")
            print(f"Elevation: {data['features']['elevation_m']:.2f} m")
            print(f"Slope: {data['features']['slope_deg']:.2f} °")
            print(f"Date Range: {data['date_range']['start']} to {data['date_range']['end']}")
            return True
        else:
            print(f"Prediction failed: {response.status_code}")
            print(f"Response: {response.text}")
            return False
    except Exception as e:
        print(f"Request failed: {e}")
        return False

def test_invalid_coordinates():
    """Test invalid coordinates"""
    print("\n--- TESTING ERROR HANDLING ---")
    try:
        payload = {
            "coordinates": ({200}, {300})
        }
        payload["latitude"] = 200
        payload["longitude"] = 300
        print(f"Testing: Invalid Coordinates ---")
        response = requests.post(
            f'{BASE_URL}/predict',
            headers={
                'Content-Type': 'application/json'
            },
            json=payload
        )
        if response.status_code == 400:
            print(f"Invalid coordinates error: {response.json()}")
            return True
        else:
            print(f"Unexpected status code: {response.status_code}")
            return False
    except Exception as e:
        print(f"Request failed: {e}")
        return False
```



```

response = requests.post(
    f'{BASE_URL}/predict',
    headers={ 'Content-Type': 'application/json' }
)
if response.status_code == 400:
    print(f"Correctly rejected invalid coordinates")
    return True
except Exception as e:
    print(f"Unexpected response: {response.status_code}")
    print(f"Test failed: {e}")
    return False

def main():
    print("\n API Tests")
    print("\n FLOOD PREDICTION API - TEST SUITE")
    health_ok = health_check()
    print("\n Health Check")
    if not health_ok:
        print("\n API is not running. Exiting tests.")
        return
    # Test 2: Multiple Predictions
    print("\n TESTING PREDICTIONS")
    test_locations = [
        { "name": "Hyderabad", "lat": 17.3850, "lon": 78.4867 },
        { "name": "Mumbai", "lat": 19.0760, "lon": 72.8777 },
        { "name": "Chennai", "lat": 13.0827, "lon": 80.2271 },
        { "name": "Delhi", "lat": 28.7041, "lon": 77.1025 }
    ]
    results = []
    for loc in test_locations:
        success = test_prediction(loc['lat'], loc['lon'], loc['name'])
        results.append(success)
    # Test 3: Error Handling
    error_handling_ok = test_invalid_coordinates()
    print("\n TEST SUMMARY")
    health_check = f'{" " if health_ok else ""}'
    predictions = f'{" " if sum(results) / len(results) == 1 else ""}'
    error_handling = f'{" " if error_handling_ok else ""}'
    print(f"Health Check and Predictions and Error Handling: {health_check} {predictions} {error_handling}")
    if health_ok and *all(results) and error_handling_ok:
        print("\n ALL TESTS PASSED!")
        print("\n Flask API is working perfectly!")
        print("\n You can now test manually: http://localhost:5000")
        print("\n 2: Make predictions via POST /predict")
        print("\n 3: Integrate with frontend or ArcGIS")
    else:
        print("\n Some tests failed. Check the API logs.")

if __name__ == "__main__":
    main()

```